

Grado Universitario en Ingeniería en Tecnologías de  
Telecomunicación (2025-2026)

*Trabajo Fin de Grado*

“Diseño e Implementación de un Sistema  
UAV Autónomo con Comunicaciones 4G,  
Fusión de Sensores y Arquitectura IoT para  
Misiones de Búsqueda y Rescate”

---

Nerea Gorostidi García

Tutor

Daniel Díaz Sánchez

Leganés, Septiembre 2026





## RESUMEN

Las operaciones de búsqueda y rescate (SAR) en entornos naturales exigen localizar con rapidez a personas desaparecidas en áreas de difícil acceso y en condiciones que comprometen la seguridad de los equipos de tierra. Aunque los vehículos aéreos no tripulados (UAV) permiten ampliar el radio de acción y reducir ese riesgo, su eficacia depende de que operen de forma autónoma, procesen las imágenes en tiempo real y transmitan la telemetría y el vídeo con total fiabilidad al puesto de mando.

Este Trabajo de Fin de Grado describe el diseño y la construcción de Drone-SAR (Guardian Eye), una plataforma UAV autónoma orientada a misiones de emergencia. El proyecto abarca desde la selección de la estructura y el cálculo de la propulsión hasta la integración eléctrica de la electrónica de potencia, coordinando una controladora de vuelo Pixhawk 6X (con ArduPilot/MAVLink) y una Raspberry Pi 5 equipada con el acelerador neuronal Hailo-8L. El proyecto organiza sus aportes de ingeniería en tres áreas complementarias:

1. **Electrónica, control de vuelo y percepción a bordo:** Abarca el ensamblaje mecánico del dron, el cableado de los buses de datos (UART, I2C, CSI, PCIe) y la integración de una Raspberry Pi 5 con un acelerador de IA (NPU Hailo-8L) para ejecutar modelos YOLO optimizados y detectar personas en tiempo real durante el vuelo.
2. **Arquitectura IoT y almacenamiento distribuido:** Recopila métricas ambientales (con el sensor BME680) y datos de telemetría divididos en cuatro áreas. Incorpora un búfer local en SQLite (*store-and-forward*) para asegurar que ningún dato se pierda ante cortes de cobertura, sincronizándolo posteriormente mediante MQTT con una infraestructura en la nube basada en AWS EC2, InfluxDB y Grafana.
3. **Comunicaciones multicanal y supervisión remota:** Utiliza una red redundante que combina radiocontrol manual, telemetría local a baja frecuencia, conectividad móvil 4G/LTE cifrada mediante una VPN segura (Tailscale/WireGuard) y vídeo de baja latencia (RTSP/WebRTC con MediaMTX). Todo ello posibilita el control remoto más allá de la línea de visión (BVLOS) mediante un panel web progresivo (PWA), recordando que cualquier operación en un entorno real queda sujeta a las autorizaciones pertinentes de AESA.

Finalmente, la viabilidad del sistema se comprueba experimentalmente en varias etapas: primero, con una simulación de guiado autónomo en *Software-in-the-Loop* (SITL) conectada a Mission Planner; luego, con pruebas de hardware en banco en tierra (conectando la computadora de a bordo a la electrónica de vuelo sin hélices para verificar la cadena de mando, la ingesta IoT y la inferencia acelerada); y por último, con ensayos de vuelo real en un campo de aeromodelismo autorizado (LECMUAV090), que incluyeron una demostración del sistema integrado con inferencia y grabación a bordo.

**Palabras clave:** Búsqueda y Rescate (SAR), YOLO, Edge-AI, *Store-and-Forward*, ArduPilot.



## DEDICATORIA

A mis padres, por su apoyo incondicional y por creer en mí incluso cuando yo dudaba. Y a mis hermanos, mis primeros referentes: crecer detrás de vosotros, fijándome en cada paso que dabais, me ha enseñado más de lo que imagináis.

A mis amigos y compañeros, con quienes he vivido cada etapa de este camino. Hemos compartido esfuerzo, agobios y alegrías, y tengo claro que llego hasta aquí gracias a haberos tenido cerca.

A todos los profesores que, a lo largo de estos años, me han enseñado no solo lo que sé, sino también a querer seguir aprendiendo. Y a Dani, mi tutor, por guiarme en este último tramo con paciencia y confianza.

A todas aquellas personas que, a través de la web del proyecto, me enviaron vídeos o me prestaron su ayuda para hacerlo realidad.

Al Club Alas de Galapagar, por abrirme sus instalaciones y hacer posibles las pruebas de vuelo de este proyecto.

A quienes me enseñaron a mirar al cielo.



# ÍNDICE DE CONTENIDOS

|                                                                                     |    |
|-------------------------------------------------------------------------------------|----|
| 1. INTRODUCCIÓN .....                                                               | 1  |
| 1.1. Motivación personal y justificación del trabajo .....                          | 1  |
| 1.2. Objetivos del trabajo .....                                                    | 2  |
| 1.3. Metodología de trabajo .....                                                   | 2  |
| 1.4. Estructura del documento.....                                                  | 3  |
| 2. ESTADO DEL ARTE .....                                                            | 5  |
| 2.1. UAV en misiones de búsqueda y rescate .....                                    | 5  |
| 2.2. Comunicaciones más allá de la línea de visión en UAV .....                     | 6  |
| 2.3. Detección de personas con visión artificial.....                               | 8  |
| 3. CONCEPTO NORMATIVO.....                                                          | 10 |
| 3.1. Marco europeo: Reglamento (UE) 2019/947.....                                   | 10 |
| 3.2. Normativa española: Real Decreto 517/2024 y AESA.....                          | 11 |
| 3.3. Normativa de comunicaciones y espectro .....                                   | 12 |
| 3.4. Privacidad y protección de datos .....                                         | 13 |
| 3.5. Estándares técnicos aplicables.....                                            | 14 |
| 3.6. Reglamento de Inteligencia Artificial (AI Act, UE 2024/1689) .....             | 14 |
| 4. DISEÑO Y CONSTRUCCIÓN DEL SISTEMA UAV .....                                      | 16 |
| 4.1. Arquitectura general del sistema.....                                          | 16 |
| 4.1.1. Plataforma: el kit Holybro X500 V2. ....                                     | 17 |
| 4.1.2. Controladora de vuelo.....                                                   | 20 |
| 4.1.3. Raspberry Pi y HAT de inferencia .....                                       | 22 |
| 4.1.4. Enlaces de radio a bordo. ....                                               | 23 |
| 4.1.5. Interconexión entre procesadores: enlace MAVLink por UART.....               | 23 |
| 4.2. Montaje e integración del sistema.....                                         | 25 |
| 4.2.1. Ensamblaje de la estructura y de la unidad de control de vuelo .....         | 25 |
| 4.2.2. Integración de la unidad de procesamiento embarcada (nodo <i>edge</i> )..... | 29 |
| 4.3. Configuración y calibración inicial.....                                       | 31 |
| 5. CONTROL DE VUELO Y TELEMETRÍA.....                                               | 33 |
| 5.1. Telemetría de vuelo con MAVLink .....                                          | 34 |
| 5.2. Estaciones de control en tierra (GCS).....                                     | 36 |
| 5.3. Control autónomo con pymavlink.....                                            | 37 |
| 5.4. Simulación SITL con ArduPilot.....                                             | 39 |
| 5.5. Vídeo FPV y enlace VTX .....                                                   | 41 |
| 6. ARQUITECTURA IOT Y ADQUISICIÓN DE DATOS .....                                    | 42 |

|                                                                                           |    |
|-------------------------------------------------------------------------------------------|----|
| 6.1. Infraestructura AWS EC2.....                                                         | 42 |
| 6.2. Arquitectura de adquisición multidominio. ....                                       | 43 |
| 6.3. Los cuatro dominios de datos .....                                                   | 44 |
| 6.3.1. Dominio ambiental.....                                                             | 45 |
| 6.3.2. Dominio de sistema.....                                                            | 46 |
| 6.3.3. Dominio de vuelo .....                                                             | 46 |
| 6.3.4. Dominio de detección .....                                                         | 47 |
| 6.3.5. Comunicación entre procesos: la posición compartida .....                          | 48 |
| 6.4. Comunicación y flujo de datos .....                                                  | 48 |
| 6.4.1. Patrón store-and-forward .....                                                     | 49 |
| 6.4.2. Store-and-forward propio frente al bridge de Mosquitto .....                       | 50 |
| 6.4.3. Recepción en el servidor: puente dronsar/#.....                                    | 51 |
| 6.4.4. Almacenamiento en InfluxDB.....                                                    | 52 |
| 6.5. Ejecución autónoma con systemd .....                                                 | 53 |
| 7. SERVICIOS WEB Y CONTROL REMOTO .....                                                   | 55 |
| 7.1. Dominio y gestión DNS con Cloudflare .....                                           | 55 |
| 7.2. Servidor web nginx y virtual hosts .....                                             | 55 |
| 7.3. Seguridad del canal: cifrado TLS.....                                                | 56 |
| 7.4. API REST y panel de control .....                                                    | 57 |
| 7.5. La cadena de mando completa .....                                                    | 59 |
| 7.6. Acceso remoto y red privada (VPN) .....                                              | 61 |
| 7.7. Transmisión de vídeo en directo .....                                                | 62 |
| 8. DETECCIÓN DE PERSONAS: MACHINE LEARNING CON YOLO11 .....                               | 65 |
| 8.1. Introducción a la detección de objetos con YOLO .....                                | 65 |
| 8.2. Metodología – Pipeline .....                                                         | 65 |
| 8.3. Dataset: obtención y construcción.....                                               | 68 |
| 8.3.1. Recopilación de imágenes propias.....                                              | 68 |
| 8.3.2. Ampliación con fuentes de datos existentes .....                                   | 68 |
| 8.3.3. Etiquetado y particionado del dataset.....                                         | 69 |
| 8.3.4. Estadísticas del conjunto de datos .....                                           | 71 |
| 8.4. Entrenamiento del modelo YOLO11 .....                                                | 72 |
| 8.5. Evaluación del modelo: métricas de referencia.....                                   | 73 |
| 8.5.1. Métricas de evaluación .....                                                       | 73 |
| 8.5.2. Modelo inicial .....                                                               | 74 |
| 8.5.3. Mejora del modelo mediante ampliación dirigida del conjunto de entrenamiento ..... | 77 |
| 8.5.4. Comparativa de variantes ( <i>nano vs small</i> ) .....                            | 81 |



|                                                                                                   |     |
|---------------------------------------------------------------------------------------------------|-----|
| 8.6. Comparativa de motores de inferencia en CPU y despliegue final con NPU Hailo-8L.....         | 82  |
| 9. VALIDACIÓN EXPERIMENTAL .....                                                                  | 86  |
| 9.1. Metodología y entorno de ensayos .....                                                       | 86  |
| 9.2. Validación de la cadena de mando y teleoperación .....                                       | 87  |
| 9.3. Validación funcional de las comunicaciones y la plataforma IoT.....                          | 87  |
| 9.4. Evaluación de la Detección Edge-AI sobre Vídeo y Resultados del Modelo .....                 | 89  |
| 9.5. Matriz de Cumplimiento de Objetivos .....                                                    | 91  |
| 10. MARCO REGULADOR.....                                                                          | 94  |
| 10.1. Cumplimiento operativo de la normativa de la AESA o de la EASA. ....                        | 94  |
| 10.2. Procedimientos Operativos en el Espacio Aéreo (LECMUAV090) .....                            | 94  |
| 10.3. Cumplimiento del Reglamento de Inteligencia Artificial (AI Act) y Protección de Datos ..... | 95  |
| 10.4. Vía regulatoria para una operación real de búsqueda y rescate .....                         | 95  |
| 11. ENTORNO SOCIO-ECONÓMICO y OBJETIVOS ODS.....                                                  | 97  |
| 11.1. Planificación temporal (Gantt, hitos, Kanban). ....                                         | 97  |
| 11.1.1. Fases del proyecto.....                                                                   | 97  |
| 11.1.2. Diagrama de Gantt .....                                                                   | 99  |
| 11.1.3. Hitos principales.....                                                                    | 100 |
| 11.1.4. Seguimiento del proyecto y gestión de desviaciones .....                                  | 100 |
| 11.2. Presupuesto del TFG .....                                                                   | 102 |
| 11.3. Impacto Socio-Económico .....                                                               | 105 |
| 11.4. Objetivos ODS.....                                                                          | 106 |
| 12. CONCLUSIONES Y TRABAJO FUTURO .....                                                           | 109 |
| 12.1. Conclusiones .....                                                                          | 109 |
| 12.2. Lecciones Aprendidas.....                                                                   | 110 |
| 12.3. Deuda Técnica Asumida .....                                                                 | 111 |
| 12.4. Trabajo futuro y líneas de mejora .....                                                     | 113 |
| E.1. Repositorios de código .....                                                                 | 10  |
| E.2. Cuaderno de entrenamiento del modelo (Google Colab).....                                     | 12  |
| E.3. Conjunto de datos público (Roboflow) .....                                                   | 13  |



## ÍNDICE DE FIGURAS

|                                                                                                                                 |    |
|---------------------------------------------------------------------------------------------------------------------------------|----|
| Figura 3.1. Categorías de operación de UAS según el nivel de riesgo. Adaptado de: [12].                                         | 10 |
| Figura 4.1. Arquitectura general del sistema en tres planos.                                                                    | 17 |
| Figura 4.2. Comprobación en báscula del peso final del UAV: 1.817 g.                                                            | 18 |
| Figura 4.3. Esquema de conexión de Pixhawk 6X, Raspberry Pi 5 y BME680.                                                         | 24 |
| Figura 4.4. Despiece inicial de los componentes del kit Holybro X500 V2.                                                        | 25 |
| Figura 4.5. Preparación y premontaje de los brazos del chasis.                                                                  | 26 |
| Figura 4.6. Ensamblaje del módulo inferior y de la placa PDB.                                                                   | 27 |
| Figura 4.7. Montaje del tren de aterrizaje y del cableado de potencia.                                                          | 27 |
| Figura 4.8. Integración de la controladora Pixhawk 6X en el chasis.                                                             | 28 |
| Figura 4.9. Integración de módulos GNSS, telemetría SiK y receptor RC.                                                          | 29 |
| Figura 4.10. Plataforma de vuelo ensamblada con el grupo motopropulsor.                                                         | 29 |
| Figura 4.11. Integración del computador Raspberry Pi 5 con Hailo-8L.                                                            | 30 |
| Figura 5.1. Arquitectura de los cuatro enlaces de comunicación del UAV.                                                         | 34 |
| Figura 5.2. Topología de clientes concurrentes MAVLink.                                                                         | 37 |
| Figura 5.3. Secuencia de la misión autónoma ejecutada por el script de control.                                                 | 39 |
| Figura 5.4. Misión de vuelo planificada en Mission Planner sobre el campo del Club Alas de Galapagar.                           | 40 |
| Figura 6.1. Arquitectura de adquisición multidominio en el nodo <i>edge</i> .                                                   | 44 |
| Figura 6.2. Conexión del sensor BME680 a la Raspberry Pi 5.                                                                     | 46 |
| Figura 6.3. Diagrama de flujo del mecanismo de <i>store-and-forward</i> implementado en el nodo <i>edge</i> .                   | 50 |
| Figura 6.4. Visualización de las medidas de varios dominios en el explorador de datos de InfluxDB.                              | 53 |
| Figura 7.1. Interfaz del panel web de control remoto para el envío de comandos de vuelo                                         | 59 |
| Figura 7.2. Cadena de mando completa, desde el panel del operador hasta el autopiloto.                                          | 60 |
| Figura 7.3. Topología de la red privada virtual mallada (Tailscale).                                                            | 61 |
| Figura 7.4. Interfaz web de supervisión visual remota                                                                           | 63 |
| Figura 7.5. Flujo de vídeo en directo desde la cámara del dron al panel del operador.                                           | 64 |
| Figura 8.1. Pipeline completo de detección de personas, desde la construcción del <i>dataset</i> hasta la inferencia embarcada. | 67 |
| Figura 8.2. Ejemplos de anotación manual en la plataforma Roboflow para el entrenamiento del modelo.                            | 70 |
| Figura 8.3. Evolución de las funciones de pérdida y de las métricas de evaluación a lo largo de las 70 épocas de entrenamiento. | 75 |
| Figura 8.4. Matriz de confusión del modelo inicial.                                                                             | 76 |
| Figura 8.5. Detecciones del modelo inicial en imágenes de <b>escenas variadas</b> .                                             | 77 |

|                                                                                                                                                                               |     |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|
| Figura 8.6. Detecciones del modelo inicial en imágenes de prueba <b>de personas a distancia</b> . ..                                                                          | 77  |
| Figura 8.7. Ejemplos de imágenes añadidas en Roboflow durante la ampliación del conjunto de entrenamiento, con personas a mayor distancia y menor resolución en píxeles. .... | 78  |
| Figura 8.8. Matriz de confusión del modelo tras la ampliación. ....                                                                                                           | 79  |
| Figura 8.9. Detecciones del modelo ampliado sobre imágenes de prueba <b>de personas a distancia</b> . ....                                                                    | 80  |
| Figura 8.10. Detecciones del modelo ampliado sobre imágenes de <b>escenas variadas</b> .....                                                                                  | 80  |
| Figura 9.1. Demostración en vuelo del sistema integrado (vídeo), con enlace directo al mismo. ....                                                                            | 90  |
| Figura 11.1. Diagrama de Gantt del proyecto (junio–septiembre de 2026). Barra oscura: tarea acotada. Barra clara: actividad continua o transversal. ....                      | 99  |
| Figura 11.2. Ejemplo de tablero Kanban para el seguimiento de tareas en una fase intermedia del proyecto. Cada tarjeta lleva una etiqueta de área.....                        | 102 |
| Figura D.1. Ubicación y zona de vuelo del club en ENAIRE drones. ....                                                                                                         | 8   |
| Figura E.1. Perfil público de GitHub del proyecto ( <a href="https://github.com/nereagorostidi">github.com/nereagorostidi</a> ), con los repositorios destacados. ....        | 11  |
| Figura E.2. Portal técnico y documentación en abierto de Guardian Eye ( <a href="https://guardian-eye.gorostiditfg.com">guardian-eye.gorostiditfg.com</a> ). ....             | 11  |
| Figura E.3. Página web de difusión pública y presentación del proyecto ( <a href="https://drone-sar.gorostiditfg.com">drone-sar.gorostiditfg.com</a> ) .....                  | 12  |
| Figura E.4. Entrenamiento del modelo YOLO11 en Google Colab con aceleración por GPU .....                                                                                     | 13  |
| Figura E.5. Estructura y particionado del conjunto de datos en la plataforma Roboflow Universe. ....                                                                          | 13  |



## ÍNDICE DE TABLAS

|                                                                                                                                                      |     |
|------------------------------------------------------------------------------------------------------------------------------------------------------|-----|
| Tabla 2.1. Casos representativos de uso de UAV en operaciones SAR.....                                                                               | 6   |
| Tabla 3.1. Estándares técnicos aplicables al sistema .....                                                                                           | 14  |
| Tabla 4.1. Balance de masas de la aeronave (weight budget) .....                                                                                     | 19  |
| Tabla 4.2. Balance de potencia y estimación de autonomía (power budget) .....                                                                        | 20  |
| Tabla 5.1. Arquitectura jerárquica de enlaces de comunicación del sistema uav .....                                                                  | 34  |
| Tabla 5.2. Estructura de la trama binaria MAVLink V2.0 .....                                                                                         | 35  |
| Tabla 6.1. Dominios de datos del sistema.....                                                                                                        | 45  |
| Tabla 8.1. Distribución inicial del conjunto de datos .....                                                                                          | 71  |
| Tabla 8.2. Métricas del modelo inicial sobre el conjunto de prueba .....                                                                             | 75  |
| Tabla 8.3. Comparación entre el modelo inicial y el modelo tras la ampliación del conjunto de<br>entrenamiento .....                                 | 78  |
| Tabla 8.4. Comparación entre las variantes small y nano de YOLO11 .....                                                                              | 81  |
| Tabla 8.5. Comparativa del rendimiento, la latencia y el consumo de recursos de los motores<br>de inferencia en la Raspberry Pi 5 (CPU Y Npu). ..... | 82  |
| TABLA 8.6. Submuestreo mínimo necesario por motor para no acumular retraso frente al ritmo<br>de captura de la cámara (29,97 FPS).....               | 83  |
| Tabla 9.1. Matriz de cumplimiento de objetivos del TFG. ....                                                                                         | 91  |
| Tabla 9.2. Cobertura de subsistemas según la fase de validación experimental.....                                                                    | 92  |
| Tabla 11.1. Fases del proyecto y periodo aproximado de ejecución. ....                                                                               | 97  |
| Tabla 11.2. Hitos relevantes alcanzados durante el desarrollo.....                                                                                   | 100 |
| Tabla 11.3. Presupuesto desglosado de componentes hardware e instrumental .....                                                                      | 103 |
| Tabla 11.4. Presupuesto de servicios cloud, gestión de dominio y plataformas de cómputo..                                                            | 103 |
| Tabla 11.5. Amortización del equipo de uso general.....                                                                                              | 104 |
| Tabla 11.6. Presupuesto consolidado y coste total de ejecución del proyecto.....                                                                     | 104 |
| Tabla 11.7. Costo estimado para replicar una unidad adicional del sistema.....                                                                       | 104 |
| Tabla 12.1. Deuda técnica asumida durante el desarrollo del TFG y previsión de cierre.....                                                           | 112 |
| Tabla D.1. Datos del campo de vuelo.....                                                                                                             | 8   |
| Tabla E.1. Repositorios públicos del proyecto en github. ....                                                                                        | 10  |
| Tabla E.2. Acceso al entorno de entrenamiento del modelo.....                                                                                        | 12  |



# 1. INTRODUCCIÓN

## 1.1. Motivación personal y justificación del trabajo

Desde pequeña siempre me han fascinado los aviones y cualquier aparato que pueda volar. Más allá de lo impresionante que es ver cómo la tecnología humana logra crear máquinas tan precisas y útiles que unen mundos y culturas en pocas horas, lo que de verdad me atrapó fue la cultura de la ingeniería que hace posible cada vuelo. Me llama la atención la búsqueda constante de fiabilidad, la coordinación exacta entre piezas que deben comunicarse en tiempo real, la redundancia como principio de diseño y la necesidad de que cada parte funcione de manera predecible. Es un sector en el que la innovación técnica y la seguridad van siempre de la mano. Entender esa disciplina me hizo ver, ya en los primeros años de la carrera, que este campo era el lugar ideal para aplicar mis estudios en telecomunicaciones.

Los sistemas aéreos no tripulados (UAS) pronto se convirtieron en el punto de unión entre mi interés por la aviación y mi deseo de crear cosas nuevas. Toda esa curiosidad dio un gran salto el día en que volé por primera vez con un DJI Neo 2. Ver cómo un equipo tan pequeño podía mantener una sustentación exacta, notar el entorno y responder al momento fue un antes y un después para mí. Entendí que teníamos ante nosotros verdaderas plataformas autónomas de telecomunicaciones y de recopilación de datos. Están lejos de ser simples juguetes con control remoto. Al final, un dron reúne en un tamaño reducido toda la dificultad de una aeronave normal. Esto incluye la estabilidad, el control de la energía y el estricto cumplimiento de las normas del espacio aéreo. Esto me confirmó que su desarrollo práctico apenas empieza.

Lo más sorprendente fue ver cómo un solo vehículo aéreo abarca casi todo el plan de estudios del Grado en Ingeniería en Tecnologías de Telecomunicación. La electrónica de potencia controla la energía y los motores. Las radiocomunicaciones y los protocolos de enlace mantienen el control sin cortes. El tratamiento de la señal, junto con la fusión de sensores, establece la orientación y el recorrido. Sobre esta base física, se suman las redes IP, el Internet de las Cosas y los servicios en la nube para la telemetría remota. El ciclo termina con inteligencia artificial a bordo que permite la percepción visual en tiempo real. Pocos sistemas hacen que tantas ramas de la ingeniería trabajen juntas con un nivel de sincronía tan alto.

Esta variedad fue la principal razón por la que elegí este proyecto. No quería enfocarme en una sola especialidad de la carrera. Quise aprovechar la combinación de varias áreas para resolver un problema de impacto social real. Las misiones de búsqueda y rescate (SAR) eran el reto de diseño perfecto. Una aeronave útil en emergencias no puede depender de la cobertura móvil para procesar vídeo en servidores remotos. Tampoco puede usar un solo canal de comunicación, ya que podría fallar o perder datos importantes. Se necesitan autonomía de vuelo, procesamiento rápido con Edge AI a bordo, enlaces de comunicación redundantes y almacenamiento local seguro. Encontrar personas en peligro sin poner en riesgo a los equipos de rescate en tierra es el objetivo que respalda cada decisión de ingeniería.



A esto se sumó la oportunidad de profundizar en otra de mis áreas de interés, el Internet de las Cosas. Convertir la aeronave en un sensor móvil que, mientras recorre el terreno, mida variables como temperatura, humedad, concentración de gases o presencia de humo y las envíe en tiempo real al puesto de mando añade una capa de inteligencia táctica que resulta clave en incendios forestales o desastres naturales. Conocer las condiciones del entorno antes de enviar personal puede marcar la diferencia entre el éxito y el riesgo.

A largo plazo, quiero enfocar mi carrera en el sector aeroespacial y en la gestión de sistemas críticos. La formación en telecomunicaciones es clave para integrar sistemas autónomos en el espacio aéreo compartido. En nuestro país hay líderes claros en este campo. Empresas como Indra o divisiones como Airbus Defence and Space trabajan directamente en la intersección entre telecomunicaciones, inteligencia artificial y sistemas autónomos críticos. Ser parte de equipos que enfrentan retos tan grandes es mi mayor meta profesional. Confío en que la visión amplia y la base técnica que he desarrollado en este proyecto sean mi mejor carta de presentación para lograrlo.

## **1.2. Objetivos del trabajo**

El objetivo principal de este TFG es diseñar e implementar un sistema de UAV autónomo orientado a misiones de búsqueda y rescate, integrando comunicaciones 4G/LTE, una arquitectura IoT en AWS, la fusión de sensores y la detección de personas mediante visión artificial con YOLO11.

Los objetivos específicos son:

- Analizar el marco normativo europeo y nacional aplicable a la operación de UAS en misiones SAR.
- Comparar y seleccionar la plataforma de hardware más adecuada para la misión.
- Diseñar el sistema de comunicaciones 4G/LTE para el control remoto y la telemetría en tiempo real.
- Implementar una arquitectura IoT en AWS EC2 con el broker MQTT Mosquitto para la adquisición y el procesamiento de métricas ambientales.
- Entrenar y desplegar un modelo YOLO11 en Raspberry Pi 5 (con un HAT de aceleración) para la detección de personas en tiempo real.
- Construir y validar el *dataset* de entrenamiento propio.
- Validar el sistema completo mediante pruebas de vuelo en un entorno controlado.

## **1.3. Metodología de trabajo**

Este Trabajo de Fin de Grado lo ha desarrollado una sola persona e integra disciplinas muy distintas —construcción del vehículo, comunicaciones, arquitectura IoT y en la nube, visión artificial y redacción de la memoria—. Por este motivo, no se han aplicado marcos de gestión pensados para equipos, como Scrum, cuyo valor radica en coordinar a varias personas con roles diferenciados y sincronizarlas mediante reuniones periódicas. En su lugar, se ha seguido una forma de trabajo iterativa e incremental: de cada subsistema se ha construido primero una versión mínima, se ha probado de extremo a

extremo y se ha refinado en sucesivas iteraciones a medida que aumentaba el conocimiento del problema.

Para organizar y priorizar las tareas, se ha utilizado un tablero Kanban, del tipo que ofrecen herramientas como Trello o GitHub Projects, con cuatro columnas, «Pendiente», «En curso», «En pruebas / Bloqueada» y «Hecho», y etiquetas por área (Hardware, Sistema y nube, Comunicaciones, Inteligencia artificial, Vuelo y Memoria). De forma complementaria, se ha mantenido un diario de proyecto, fechado, actualizado casi a diario y publicado en tiempo real en la web del proyecto (<https://drone-sar.gorostiditfg.com/journal.html>); cada entrada recoge el trabajo realizado, los hitos alcanzados y las incidencias, con resúmenes semanales y filtros por área, y ha funcionado como registro de trazabilidad del proyecto. Todo el código se ha mantenido bajo control de versiones en GitHub, lo que, además, ha actuado como una copia de seguridad continua.

El rasgo más característico del método de trabajo ha sido el solapamiento intencional de las líneas de trabajo. Mientras se depuraba el guiado autónomo en simulación (SITL), se desplegaba en paralelo la infraestructura en la nube; la construcción del *dataset* avanzaba de forma continua desde etapas tempranas, principalmente en sesiones de fin de semana; y el montaje físico del vehículo progresaba en segundo plano, condicionado por la llegada escalonada de los componentes, sin llegar a bloquear el resto del proyecto. La redacción de la memoria también se abordó desde las primeras semanas, capítulo a capítulo. Cada validación intermedia realimentó las decisiones de diseño posteriores. La planificación temporal resultante —fases, cronograma en formato de diagrama de Gantt e hitos— se detalla en la sección 11.1.

#### 1.4. Estructura del documento

Este Trabajo de Fin de Grado está organizado en doce capítulos que siguen un orden lógico para cubrir todas las fases del diseño, la implementación y la validación del sistema *Drone-SAR (Guardian Eye)*.

- **Capítulo 1 (Introducción):** Presenta la motivación del proyecto, explica su importancia en el área de salvamento y define los objetivos generales y específicos.
- **Capítulo 2 (Estado del arte):** Analiza el uso de drones en situaciones de emergencia, las comunicaciones celulares para operaciones BVLOS y los avances en visión artificial para la detección de personas.
- **Capítulo 3 (Concepto normativo):** Explica el marco legal aeronáutico europeo (Reglamento (UE) 2019/947) y nacional (RD 517/2024 y AESA), las directivas de telecomunicaciones, la normativa de protección de datos y el Reglamento Europeo de Inteligencia Artificial (AI Act).
- **Capítulo 4 (Diseño y construcción del sistema UAV):** Explica cómo se seleccionaron los componentes, se equilibraron los pesos y la potencia, y se ensamblaron el chasis, la controladora Pixhawk 6X y la computadora Raspberry Pi 5.

- **Capítulo 5 (Control de vuelo y telemetría):** Describe la estructura de los enlaces de comunicación, el protocolo MAVLink v2.0, los scripts de guiado autónomo con pymavlink y el entorno de simulación SITL con ArduPilot.
- **Capítulo 6 (Arquitectura IoT y adquisición de datos):** Explica el nodo *edge* multidominio, la mensajería MQTT con Mosquitto, el sistema de almacenamiento local de tipo *store-and-forward* en SQLite y el guardado de datos en series temporales en InfluxDB en AWS EC2.
- **Capítulo 7 (Servicios web y control remoto):** Expone la infraestructura en la nube, la red privada virtual (Tailscale/WireGuard), la API REST en Flask con un proxy inverso de nginx (TLS), el panel de control web PWA y la transmisión de vídeo en directo con WebRTC.
- **Capítulo 8 (Detección de personas: Machine Learning con YOLO11):** Aborda la creación y curación del *dataset* propio, el entrenamiento mediante *transfer learning* del modelo YOLO11, la comparativa de variantes y su despliegue acelerado en la NPU Hailo-8L.
- **Capítulo 9 (Validación experimental y evaluación de objetivos):** Evalúa el sistema paso a paso mediante simulación en SITL, integración de hardware en un banco de pruebas (semi-HIL) y pruebas de vuelo reales, comprobando si se cumplen los objetivos establecidos.
- **Capítulo 10 (Marco regulador):** Resume cómo el prototipo y los protocolos de prueba se ajustan a las limitaciones operativas del espacio aéreo.
- **Capítulo 11 (Entorno socioeconómico y objetivos ODS):** Presenta el presupuesto del proyecto, analiza su impacto social en las misiones SAR y su aporte a los Objetivos de Desarrollo Sostenible.
- **Capítulo 12 (Conclusiones y trabajo futuro):** Resume los aportes técnicos del trabajo, las lecciones aprendidas durante la integración y las principales ideas para el desarrollo futuro.

El documento incluye un glosario de abreviaturas (Anexo A) y los certificados oficiales de AESA de registro como operador de UAS y piloto a distancia en las subcategorías A1/A3 (Anexos B y C).

## 2. ESTADO DEL ARTE

### 2.1. UAV en misiones de búsqueda y rescate

Los sistemas UAV, de origen fundamentalmente militar, se han incorporado en los últimos años a las operaciones civiles de emergencia. En el ámbito SAR, su capacidad para acceder a zonas de difícil acceso, cubrir grandes áreas en poco tiempo y transmitir imágenes en directo los ha convertido en un recurso de gran valor. La detección automática de personas en imágenes aéreas, en particular, se ha convertido en una línea de investigación activa dentro de la visión por computador aplicada al rescate, con un número creciente de publicaciones en los últimos años [1].

Esta incorporación se aprecia en numerosos casos reales documentados en los que drones equipados con distintos sensores se han empleado para localizar a personas desaparecidas en entornos montañosos y costeros. La Tabla 2.1 recoge una selección representativa que permite observar la evolución del campo, desde los sistemas basados en la inspección visual humana de las imágenes hasta los más recientes, que incorporan la detección automática mediante inteligencia artificial.

Los casos recogidos permiten distinguir dos generaciones tecnológicas. En la primera, el dron aporta la vista aérea, pero el análisis lo realiza una persona. Es el caso de los drones térmicos con los que distintos equipos de rescate han localizado a personas desaparecidas revisando en directo la imagen de la cámara, como el hallazgo de un hombre de 78 años con demencia en Malibú en diciembre de 2024 [2]. En esta misma generación encajan soluciones de asistencia al rescate como el Auxdron LFG (España), un dron diseñado para playas que se aproxima a una persona en riesgo de ahogamiento y le entrega un chaleco salvavidas, lo que reduce el tiempo de respuesta del socorrista [3].

En la segunda generación, el dron sigue capturando imágenes, pero es un algoritmo de visión por computador el que las analiza y señala automáticamente lo que no encaja con el entorno. Programas como Eagle Eyes, desarrollado en Canadá, combinan la cámara del dron con visión por computador entrenada para detectar anomalías en el paisaje, de modo que un color fuera de lugar, como puede ser la chaqueta de una persona, basta para activar una alerta [4]. El ejemplo más ilustrativo es el rescate del Monviso, en los Alpes italianos: en julio de 2025, diez meses después de la desaparición de un montañero y tras suspenderse la búsqueda inicial sin resultado, el CNSAS empleó dos drones que cubrieron 183 hectáreas en apenas cinco horas, capturando unas 2.600 imágenes de alta resolución. El análisis manual de todo ese material habría llevado semanas; el software de IA lo procesó en una sola tarde, detectando el casco del desaparecido a partir de un grupo de píxeles rojos que destacaban sobre el terreno [5].

Ahora bien, la detección de personas desde el aire sigue siendo un problema difícil. La investigación en este campo muestra que, a diferencia de los objetos grandes y rígidos habituales en imágenes aéreas, como vehículos o embarcaciones, las personas aparecen como objetos diminutos, dispersos y, con frecuencia, parcialmente ocultos por la vegetación, lo que dificulta su detección, especialmente en terrenos no urbanos como montañas o bosques [1]. Para abordar este problema, la investigación se apoya en

conjuntos de datos específicos, como HERIDAL, referencia habitual para el entrenamiento y la evaluación de detectores de personas en entornos naturales desde UAV [6].

A estas dificultades se añade otra, la más relevante para este trabajo: aunque existen sistemas de detección automática cada vez más precisos, su procesamiento se realiza casi siempre fuera del dron, ya sea sobre las imágenes descargadas o en una estación en tierra, y no a bordo durante el vuelo. Los propios estudios reconocen que integrar estos modelos en el hardware embarcado de los drones sigue siendo un problema sin resolver, debido a las limitaciones de cómputo y de ancho de banda de estos dispositivos [1].

Es precisamente en esta línea donde se sitúa la aportación de este TFG: integrar la detección de personas embarcada, mediante un modelo YOLO ejecutado a bordo, cuya elección y fundamento se detallan en el apartado 2.3, dentro de un sistema SAR completo y funcional, ejecutándola sobre un acelerador de inteligencia artificial de bajo consumo montado en el propio dron, de modo que la identificación de posibles víctimas se produzca en tiempo real durante la misión.

TABLA 2.1. CASOS REPRESENTATIVOS DE USO DE UAV EN OPERACIONES SAR.

| Caso / Sistema               | Año  | Análisis de la imagen               | Dónde se procesa                       |
|------------------------------|------|-------------------------------------|----------------------------------------|
| Auxdron LFG (España)         | 2015 | Localización humana (piloto)        | Entrega de chaleco                     |
| Rescate en Malibú (EE. UU.)  | 2024 | Inspección humana de cámara térmica | En tiempo real, revisión visual        |
| Eagle Eyes (Canadá)          | 2025 | Detección automática (IA)           | Sobre las imágenes capturadas          |
| Rescate del Monviso (Italia) | 2025 | Detección automática (IA)           | En estación de tierra, tras la captura |
| Drone-SAR (este TFG)         | 2026 | Detección automática (IA, YOLO)     | A bordo del dron, en tiempo real       |

## 2.2. Comunicaciones más allá de la línea de visión en UAV

El alcance al que puede operar un dron depende directamente de su sistema de comunicaciones. En una misión SAR, donde hay que cubrir zonas amplias y con frecuencia de difícil acceso, ese alcance se convierte en un factor determinante: cuanto más lejos pueda llegar la aeronave sin perder el enlace, mayor es el área que puede rastrear.

El enlace de radio convencional (2,4 GHz / 5,8 GHz) limita el alcance a la línea de visión directa (VLOS, *Visual Line of Sight*), ya que el piloto debe mantener contacto visual con la aeronave. Para superar esta limitación, existen dos enfoques: uno que mejora la operación a corto alcance y otro que permite salir de ella.

El primero es el vídeo FPV (*First Person View*). El dron incorpora un transmisor de vídeo (VTX) que envía en tiempo real la imagen de su cámara a la estación en tierra, donde el operador la visualiza en un monitor o en unas gafas de realidad virtual. De este modo se

pilota tomando como referencia la propia cámara, lo que facilita la operación en misiones SAR. No obstante, este enlace de vídeo emplea radio de corto alcance en la banda de 5,8 GHz, lo que mejora el pilotaje, pero no resuelve el problema de fondo: la distancia de operación sigue siendo limitada.

La segunda vía sí lo hace, al conectar el dron a la red celular (4G/LTE) para operar más allá de la línea de visión (BVLOS, *Beyond Visual Line of Sight*). En lugar de establecer un enlace punto a punto entre el dron y el piloto, la aeronave pasa a ser un cliente más de la red móvil y aprovecha la cobertura de los operadores de telecomunicaciones. Esta idea, conocida en la literatura como *cellular-connected UAV*, ha recibido una atención creciente como forma de proporcionar cobertura amplia y continua para el control del dron, aunque su viabilidad a distintas alturas sigue siendo objeto de estudio [7].

La cuestión clave es si el enlace celular introduce una latencia lo suficientemente baja como para controlar el dron en tiempo real. Los estudios disponibles indican que sí. En un sistema desarrollado para búsqueda y rescate marítimo sobre LTE se han medido latencias del orden de pocos milisegundos en el envío de comandos críticos, suficientes para el control en tiempo real, aunque para ello es necesario reservar capacidad adicional de la red [8]. En esta misma línea, se ha comprobado en prototipos de dron conectados por LTE que es posible controlar la aeronave y transmitir datos a través de una red comercial [8], [9]. No obstante, las mediciones reales muestran que ese rendimiento se degrada con la altura: al pasar del nivel del suelo a los 170 m, la latencia llega a aumentar hasta 94 ms y la tasa de datos en el enlace descendente se reduce en torno al 30 % [9].

Esta degradación tiene una causa estructural: las redes celulares están pensadas para usuarios situados en tierra, con las antenas de las estaciones base inclinadas hacia el suelo, de manera que un dron que vuela por encima de ellas queda servido por los lóbulos secundarios de la antena, es decir, por la parte más débil de su señal. Los estudios de cobertura celular para drones confirman, además, un efecto contraintuitivo: al ganar altura, el dron tiene línea de visión con más estaciones base a la vez, lo que aumenta la interferencia entre ellas y empeora tanto la cobertura como el rendimiento [7].

A esta limitación se suman otras dos. La primera es que el vídeo y los comandos de control comparten el mismo enlace e interfieren entre sí: el retardo de uno reduce el margen disponible para el otro. Garantizar, a la vez, un control rápido y una buena calidad de vídeo supone, por tanto, un compromiso de diseño [10]. La segunda es de carácter regulatorio: operar en BVLOS requiere una autorización específica de la AESA.

No obstante, conviene señalar que también existe la opción de programar el dron mediante misiones, de modo que pueda cubrir automáticamente áreas predeterminadas, sin necesidad de control manual. Este mecanismo resulta especialmente útil cuando hay que cubrir zonas extensas o cuando las áreas de búsqueda ya están definidas de antemano, ya que permite abarcar regiones que se extienden más allá del alcance de un control manual, siempre que se cuente con la autorización legal correspondiente.

### 2.3. Detección de personas con visión artificial

En una misión de búsqueda y rescate, la cámara aérea ayuda poco si solo depende de la vista de una persona en tierra. Revisar a mano un flujo de vídeo continuo, fotograma por fotograma, cansa la vista en pocos minutos. Además, retrasa la respuesta en situaciones en las que cada segundo importa. Por eso, el sistema debe identificar automáticamente a posibles víctimas mientras vuela.

Sin embargo, detectar personas desde un dron plantea retos muy concretos que no aparecen en aplicaciones terrestres convencionales:

1. **El tamaño de la persona en la imagen.** A la altura normal de rastreo, que suele situarse entre 20 y 50 metros, una persona tumbada o sentada ocupa solo una pequeña parte del encuadre. A veces, se reduce a un grupo de apenas 20 o 30 píxeles.
2. **La perspectiva y el terreno.** Desde el aire no vemos la silueta clásica de pie. La vista desde arriba hace que la persona se mezcle con facilidad con piedras, sombras, ramas o desniveles del suelo.
3. **El consumo y el peso a bordo.** No podemos instalar un ordenador potente en la aeronave. La batería es limitada y el peso reduce la autonomía de vuelo. Por eso, el algoritmo debe ser muy eficiente y funcionar en hardware embebido ligero.

Dentro de la visión artificial moderna existen principalmente dos formas de plantear la detección de objetos:

- Modelos en dos etapas, como la familia Faster R-CNN, primero localizan regiones donde creen que puede haber algo. Después clasifican qué hay en cada una. Son modelos exactos, pero resultan demasiado pesados y lentos para procesar vídeo en tiempo real en una placa pequeña.
- Modelos en una sola etapa (*single-shot*, como la familia YOLO): Analizan la imagen completa en una sola pasada de la red neuronal. Predicen simultáneamente la posición y la clase de cada objeto.

Para este proyecto, se elige YOLO (You Only Look Once) porque procesa el vídeo tan rápido como vuela el dron. Así no se pierden fotogramas durante el rastreo.

Desde la primera versión de YOLO en 2016 hasta hoy, la arquitectura ha cambiado mucho. En este trabajo se eligió YOLO11, la versión más reciente de la familia. Se prefirieron a generaciones anteriores, como YOLOv5 o YOLOv8, por dos mejoras directas para el rescate aéreo.

- **Detección directa sin cajas predefinidas (*anchor-free*):** Las versiones clásicas, como YOLOv3 o v5, usaban plantillas de tamaño fijo (*anchor boxes*) diseñadas para personas vistas de pie y de frente. En una toma aérea cenital, las proporciones cambian por completo. YOLO11 ya no usa estas plantillas rígidas y calcula directamente el centro y las dimensiones del objeto. Así se adapta mucho mejor a poses complejas en el suelo.

- **Módulos de atención espacial (C2PSA):** Son el principal aporte técnico de YOLO11 frente a YOLOv8. Esta versión incluye capas de atención espacial que ayudan a la red a fijarse en zonas con detalles compatibles con un cuerpo o con prendas de vestir. Ignora el fondo repetitivo del bosque o de la montaña. Además, la arquitectura reduce el número de operaciones de cálculo. Esto permite mejorar la precisión en figuras pequeñas sin aumentar significativamente la temperatura ni el consumo de la placa.

En el proyecto se cuenta con un enlace 4G y un servidor en AWS. Una opción teórica sería enviar el vídeo en directo a la nube y ejecutar allí el modelo de detección. Sin embargo, se descarta esa arquitectura por dos razones de diseño en el ámbito de las telecomunicaciones.

- **Inestabilidad del enlace celular al aire libre.** En zonas rurales o de montaña, la cobertura 4G presenta caídas de señal y retrasos impredecibles. Si la detección dependiera de subir videos en tiempo real a internet, un corte de cobertura justo en el momento del avistamiento haría que la víctima pasara desapercibida.
- **Ahorro de ancho de banda.** Subir un vídeo en alta definición saturaría el canal de subida, lo que interferiría con la telemetría crítica de navegación.

Por estas razones, la inteligencia se mantiene a bordo mediante el *Edge Computing*. La Raspberry Pi 5, junto con el acelerador neuronal Hailo-8L, ejecuta YOLO11 localmente y en tiempo real. El dron detecta de forma autónoma sin depender de la conexión a internet. La conexión 4G con AWS se usa solo para lo realmente importante. Sirve para enviar la alerta inmediata, la posición GPS de la víctima y una fotografía recortada de confirmación a la estación de control.



### 3. CONCEPTO NORMATIVO

#### 3.1. Marco europeo: Reglamento (UE) 2019/947

El Reglamento de Ejecución (UE) 2019/947 establece las normas y los procedimientos para la utilización de aeronaves no tripuladas en el espacio aéreo europeo [11]. Su enfoque parte del riesgo de la operación y no del uso, ya sea recreativo o profesional: todas las operaciones con drones se clasifican en tres categorías según el riesgo que suponen, tal como ilustra la Figura 3.1.



Figura 3.1. Categorías de operación de UAS según el nivel de riesgo. Adaptado de: [12].

La categoría abierta agrupa las operaciones de bajo riesgo. No necesita autorización previa ni declaración del operador, siempre que se respeten sus limitaciones: volar a un máximo de 120 metros sobre el terreno, mantener el dron dentro del alcance visual del piloto (VLOS) y no sobrevolar concentraciones de personas. La categoría específica cubre las operaciones de riesgo medio, como los vuelos más allá del alcance visual o a mayor altura, y requiere una autorización de AESA basada en un análisis de riesgos, o una declaración del operador cuando la operación se ajusta a un escenario estándar ya definido. Por último, la categoría certificada abarca las operaciones de alto riesgo, equiparables a las de la aviación tripulada, como el transporte de personas, y exige certificar tanto la aeronave como al operador.

Junto con este reglamento, el Reglamento Delegado (UE) 2019/945 define siete clases de drones, de la C0 a la C6, identificadas mediante un marcado que acredita que la aeronave cumple determinados requisitos técnicos [13]. Ese marcado lo llevan los drones fabricados y comercializados en Europa, pero no las aeronaves de construcción propia, como la de este proyecto, que quedan al margen de esa clasificación. Esto no impide volar, pero sí condiciona en qué subcategorías puede operar el dron y qué obligaciones le corresponden.

Dentro de la categoría abierta, existen tres subcategorías, A1, A2 y A3, que se distinguen por la proximidad con la que se puede volar a personas y zonas urbanas. La más restrictiva en cuanto a distancias es la A3, pensada para el menor riesgo: obliga a mantener una distancia de al menos 150 metros respecto de zonas residenciales, comerciales, industriales o recreativas, y permite operar drones de hasta 25 kilogramos. También es la

subcategoría en la que encajan las aeronaves de construcción propia que superan los 250 gramos, siempre que se respeten dichas distancias.

El UAV de este proyecto es una aeronave de construcción propia, con una masa aproximada de 1817 g, diseñada para operar en un campo de vuelo alejado de núcleos urbanos. Por sus características, la operación se enmarca en la subcategoría A3 de la categoría abierta. De forma coherente con ello, el piloto dispone del certificado de competencia A1/A3, que se obtiene tras superar un examen teórico en línea de AESA y que habilita precisamente para volar en las subcategorías A1 y A3.

Un requisito adicional que introduce la normativa es la identificación remota directa, conocida como Remote ID. Consiste en un sistema que difunde en tiempo real por radio datos como el número de registro del operador y la posición del dron, de modo que la aeronave pueda identificarse a distancia. Esta obligación se ha implantado por fases: desde el 1 de enero de 2024, para los drones con marcado de clase (C1, C2 y C3) en categoría abierta y en toda la categoría específica, y, de forma general, a partir de 2026, cuando finaliza el plazo de adaptación para las aeronaves sin marcado de clase. En cualquier caso, el UAV de este proyecto queda sujeto a esta exigencia: al superar los 250 gramos y ser de construcción propia, carece de marcado de clase, por lo que necesita incorporar un módulo externo de identificación remota para cumplirla.

### **3.2. Normativa española: Real Decreto 517/2024 y AESA**

El uso civil de drones en España se rige por el Real Decreto 517/2024, de 4 de junio, que adapta el marco europeo al ámbito nacional y sustituye al anterior Real Decreto 1036/2017 [14]. Su finalidad es desarrollar y concretar, en el ordenamiento jurídico español, las disposiciones que los reglamentos europeos establecen de manera general.

En línea con la normativa europea, el decreto no distingue entre el uso recreativo y el profesional, sino que se aplica a cualquier persona que opere un dron en territorio español. La autoridad encargada de regular y supervisar estas operaciones es la Agencia Estatal de Seguridad Aérea (AESA).

El Real Decreto ha sufrido una modificación desde su entrada en vigor: en junio de 2025, el Tribunal Supremo anuló la parte relativa a las sanciones por defectos en su tramitación [15], de modo que las multas vuelven a regirse por la Ley 21/2003 de Seguridad Aérea [16]. El resto del decreto sigue plenamente vigente.

Aunque el Real Decreto 517/2024 [14] contempla las labores de búsqueda y salvamento, conviene distinguir dos situaciones muy diferentes. Cuando se lleva a cabo un rescate real, la operación no se rige por las normas habituales de la categoría abierta; al tratarse de una emergencia pública, queda fuera del reglamento europeo ordinario y pasa al marco de las actividades de Protección Civil reguladas por los artículos 12 y 44 del propio decreto [14], así como por lo establecido en la Ley 17/2015 del Sistema Nacional de Protección Civil [17]. Los detalles sobre cómo encajaría el sistema en ese marco operativo se analizan en el apartado 10.4.

En cambio, los vuelos realizados durante este proyecto se han considerado ensayos para el desarrollo de un prototipo. Por ello, se han llevado a cabo dentro del marco civil

estándar: en la subcategoría A3 de la categoría abierta, sin perder nunca el contacto visual directo con el dron (VLOS) y despegando dentro de un campo de vuelo federado y reconocido en la documentación oficial de ENAIRE, siguiendo punto por punto los requisitos establecidos por AESA.

Como parte del desarrollo se ha obtenido el certificado de piloto de drones A1/A3, necesario para operar en la categoría abierta.

Además, al tratarse de un dron de más de 250 gramos, la normativa exige una doble inscripción, la cual también se ha completado. Por un lado, el registro como operador de UAS en la sede electrónica de AESA, cuyo número identificativo debe figurar de forma visible en la aeronave. Por otro lado, el registro de la propia aeronave en el Registro de Aeronaves del Ministerio del Interior, introducido por el Real Decreto 517/2024 para garantizar la trazabilidad de cada aparato.

Para la selección del espacio aéreo de las pruebas se ha empleado la herramienta ENAIRE Drones, de consulta obligatoria antes de cada vuelo, que muestra las zonas geográficas de UAS (zonas GEO) y sus limitaciones [18]. El campus de la UC3M en Leganés se descartó para los vuelos al exterior por encontrarse bajo la CTR del aeródromo de Getafe, un espacio aéreo controlado cuyo uso exigiría coordinación adicional con el control de tránsito aéreo, así como la solicitud de autorizaciones específicas para cada operación.

Las pruebas de vuelo finalmente se han realizado en el campo de vuelo del Club de Aeromodelismo Alas de Galapagar (Madrid), donde se ha tramitado el alta como socio. Este campo es una zona de vuelo de aeromodelismo publicada oficialmente en la Publicación de Información Aeronáutica (AIP) de ENAIRE, con el identificador LECMUAV090 y un límite de vuelo de hasta 120 m sobre el terreno [19]. Ser socio del club permite realizar las pruebas dentro de un marco reconocido por la normativa, sin tener que solicitar una autorización distinta para cada vuelo. A esto se suma que la ubicación cumple los criterios buscados de seguridad y ausencia de restricciones, al ser un espacio pensado específicamente para el aeromodelismo y al ubicarse fuera de los espacios aéreos controlados. En el Anexo D se recoge la ficha del campo de vuelo y su ubicación.

### **3.3. Normativa de comunicaciones y espectro**

El UAV desarrollado incorpora varios equipos de radiocomunicación, lo que sitúa el proyecto también en el ámbito de la normativa de telecomunicaciones, regulada en España por la Ley 11/2022, de 28 de junio, General de Telecomunicaciones [20]. El uso del espectro radioeléctrico, por su parte, se rige por el Cuadro Nacional de Atribución de Frecuencias (CNAF), que determina qué servicio está reservado para cada banda y en qué condiciones puede utilizarse [21].

Los enlaces de radio de corto alcance del sistema (el control por radiofrecuencia en la banda de 2,4 GHz y el vídeo FPV en la de 5,8 GHz) operan en bandas de uso común. Esto quiere decir que no necesitan licencia individual ni autorización previa, siempre que se respeten las condiciones técnicas establecidas en las Notas de Utilización Nacional del

CNAF, en especial los límites de potencia de emisión. En este caso, la obligación del usuario no es solicitar un permiso, sino emplear equipos que ya cumplan esas condiciones.

Para garantizarlo, todo equipo radioeléctrico que se comercializa en la Unión Europea debe llevar el marcado CE y cumplir la Directiva 2014/53/UE, conocida como Directiva RED (Radio Equipment Directive) [22], que asegura que el dispositivo utiliza de manera eficiente el espectro y respeta la compatibilidad electromagnética.

El enlace celular (4G/LTE) es un caso distinto porque no utiliza el espectro de uso común, sino el espectro licenciado por el operador de telecomunicaciones. Aun así, esto no añade ninguna obligación al UAV: el espectro está concedido al operador móvil, que es su titular, y el dron se comporta simplemente como un terminal más de su red, igual que un teléfono. Por tanto, llevar a bordo un módulo 4G no exige ninguna habilitación específica de espectro, siempre que el módulo disponga de marcado CE y se utilice en la red de un operador autorizado.

En consecuencia, el conjunto de equipos de radiocomunicación embarcados no requiere autorizaciones de espectro adicionales; basta con que cada dispositivo esté homologado conforme a la Directiva RED. Esta condición se ha tenido en cuenta como criterio para seleccionar los componentes de comunicaciones del sistema.

### **3.4. Privacidad y protección de datos**

El UAV desarrollado incorpora una cámara como sensor principal para la detección de personas, lo que sitúa el proyecto en el ámbito de la protección de datos personales. Esta materia se regula a nivel europeo por el Reglamento General de Protección de Datos (RGPD, Reglamento (UE) 2016/679) [23] y, a nivel nacional, por la Ley Orgánica 3/2018, de Protección de Datos Personales y Garantía de los Derechos Digitales [24].

Una imagen constituye un dato personal cuando permite identificar a una persona, ya sea de forma directa o indirecta. En un sistema de búsqueda y rescate, esta cuestión cobra especial relevancia, ya que la finalidad misma del sistema (localizar e identificar a personas) implica el tratamiento de datos personales de forma inherente. Las imágenes en las que una persona resulte reconocible quedan, por tanto, sujetas a la normativa de protección de datos.

El RGPD exige una base legal para tratar datos personales. En la fase de pruebas de este proyecto, la base es el consentimiento de las personas que participan en los ensayos: no se graba a personas ajenas a las pruebas, y quienes colaboran lo hacen de forma voluntaria e informada. En un despliegue operativo real, la base legal sería distinta, ya que el propio Reglamento contempla el interés vital de la persona afectada y el cumplimiento de una misión de interés público como fundamentos para tratar datos sin consentimiento previo, supuestos que encajan de forma natural en una emergencia en la que está en juego la vida de una persona desaparecida.

Durante el desarrollo del proyecto se han aplicado medidas técnicas y organizativas acordes con el principio de minimización de datos: se ha evitado captar imágenes de personas no participantes, las grabaciones realizadas durante las pruebas se han limitado

a lo estrictamente necesario para validar el sistema de detección, y los datos se han eliminado una vez concluido su análisis, sin conservarse más allá de su finalidad.

### 3.5. Estándares técnicos aplicables

Además del marco legal descrito en los apartados anteriores, el sistema desarrollado debe ajustarse a una serie de estándares técnicos. A diferencia de la normativa jurídica, que impone obligaciones legales, los estándares técnicos son especificaciones definidas por organismos internacionales de normalización que establecen cómo deben comportarse los equipos y los protocolos para garantizar su interoperabilidad, es decir, que los distintos componentes puedan comunicarse entre sí y con sistemas externos de forma fiable.

En un proyecto como este, que integra comunicación celular, control de vuelo, telemetría de sensores y transmisión de vídeo, el cumplimiento de estos estándares permite que todos los subsistemas funcionen de manera coordinada y compatible con la tecnología existente. La Tabla 3.1 recoge los principales estándares aplicables, su función en el sistema y el organismo responsable de su definición.

TABLA 3.1. ESTÁNDARES TÉCNICOS APLICABLES AL SISTEMA

| Estándar                                                                            | Función en el sistema                                                        | Organismo             |
|-------------------------------------------------------------------------------------|------------------------------------------------------------------------------|-----------------------|
| 3GPP Release 16 (4G/LTE) <b>¡Error! No se encuentra el origen de la referencia.</b> | Enlace de comunicación celular para la operación BVLOS.                      | 3GPP                  |
| MAVLink v2.0 [26]                                                                   | Protocolo de telemetría y de comandos entre el dron y la estación de tierra. | Dronecode / ArduPilot |
| MQTT v5.0 [27]                                                                      | Protocolo de mensajería para la telemetría de sensores (IoT).                | OASIS                 |
| RTSP [28]                                                                           | Transmisión en tiempo real de vídeo desde la cámara del dron.                | IETF                  |
| IEEE 802.11 (Wi-Fi) [29]                                                            | Conexión local con la estación de control en tierra (GCS).                   | IEEE                  |

### 3.6. Reglamento de Inteligencia Artificial (AI Act, UE 2024/1689)

El Reglamento (UE) 2024/1689, conocido como AI Act, es la primera ley que regula de forma integral la inteligencia artificial [30]. Su planteamiento se basa en el riesgo: divide las aplicaciones de IA en cuatro niveles (inaceptable, alto, limitado y mínimo) y exige requisitos más estrictos a medida que dicho riesgo aumenta. El reglamento afecta a este trabajo porque el sistema desarrollado se apoya en un modelo de detección de personas basado en visión artificial, que la norma considera un sistema de IA.

Lo que debe determinarse, por tanto, es en qué nivel de riesgo se encuadra el sistema. El reglamento sitúa en la categoría de alto riesgo los sistemas de identificación biométrica y los que se aplican a infraestructuras críticas. En particular, la identificación biométrica remota en tiempo real en espacios públicos (es decir, la que permite reconocer la identidad de una persona a distancia) queda fuertemente restringida y llega a considerarse de riesgo inaceptable, salvo en casos muy concretos. Entre esas excepciones se encuentra,

precisamente, la búsqueda de una persona desaparecida, uno de los escenarios de uso previstos.

Aun así, el factor determinante para clasificar el sistema es que no realiza identificación biométrica. El modelo detecta que hay una persona en la imagen, pero no reconoce su identidad ni extrae rasgos que permitan diferenciar a un individuo de otro: únicamente indica dónde se encuentra una persona, no quién es. Esta es la distinción clave, ya que lo que el reglamento considera de alto riesgo es precisamente el reconocimiento de la identidad. Al no producirse, el sistema queda fuera de las categorías vinculadas a la biometría.

A ello se añaden otros factores que lo sitúan en el nivel de riesgo mínimo: el sistema opera en entornos despoblados y con un piloto que supervisa en todo momento; las detecciones solo aportan información y no desencadenan ninguna acción sobre las personas; las imágenes se emplean únicamente para la detección en tiempo real y no se conservan al finalizar la misión; y el sistema no se conecta a ninguna infraestructura crítica. Esta clasificación en el nivel de riesgo mínimo implica que el sistema no está sujeto a las obligaciones de evaluación, documentación y supervisión que el reglamento reserva a las categorías de mayor riesgo, y puede desarrollarse y operar sin requisitos regulatorios específicos.

Conviene señalar que esta clasificación podría cambiar si el sistema evolucionara en determinadas direcciones. Sería el caso, por ejemplo, de un despliegue en entornos con público no informado, o de la integración de las detecciones con sistemas automáticos de aviso a los servicios de emergencia. Ambos escenarios se contemplan en el apartado de trabajo futuro.

## 4. DISEÑO Y CONSTRUCCIÓN DEL SISTEMA UAV

### 4.1. Arquitectura general del sistema

Antes de entrar en el detalle de cada componente, conviene ofrecer una visión de conjunto del sistema. La arquitectura se estructura en tres planos físicos claramente diferenciados:

- **El campo de vuelo:** reúne los elementos de operación local. Incluye el transmisor de radiocontrol (RC), que garantiza en todo momento la posibilidad de asumir el control manual por seguridad, y la estación de control en tierra (GCS), desde donde se supervisa la misión de vuelo.
- **La aeronave:** constituye el núcleo del sistema e incluye a bordo dos unidades de procesamiento complementarias: por un lado, la controladora de vuelo (Pixhawk), encargada de las tareas críticas de tiempo real como mantener el dron estable y ejecutar las órdenes de navegación; y por otro, una Raspberry Pi 5 que actúa como nodo *edge* para asumir las tareas de aplicación, tales como adquirir las medidas del sensor ambiental, procesar el vídeo en local para detectar personas y gestionar las comunicaciones con el exterior.
- **La nube:** consiste en una infraestructura desplegada sobre una instancia de AWS EC2 encargada de recibir, registrar y gestionar todo el flujo remoto del sistema. Centraliza las comunicaciones mediante un broker MQTT (Mosquitto), almacena las series temporales y las lecturas ambientales en InfluxDB, y ofrece una API REST mediante un servidor nginx con cifrado TLS para permitir la teleoperación y el envío de comandos desde un panel web.

La interacción física y lógica entre estos tres planos se resume en la Figura 4.1. Dado que las comunicaciones, la gestión de datos y los servicios cloud se analizan en los próximos capítulos, el resto de este capítulo se centra exclusivamente en el diseño, la selección e integración del hardware embarcado en el dron.

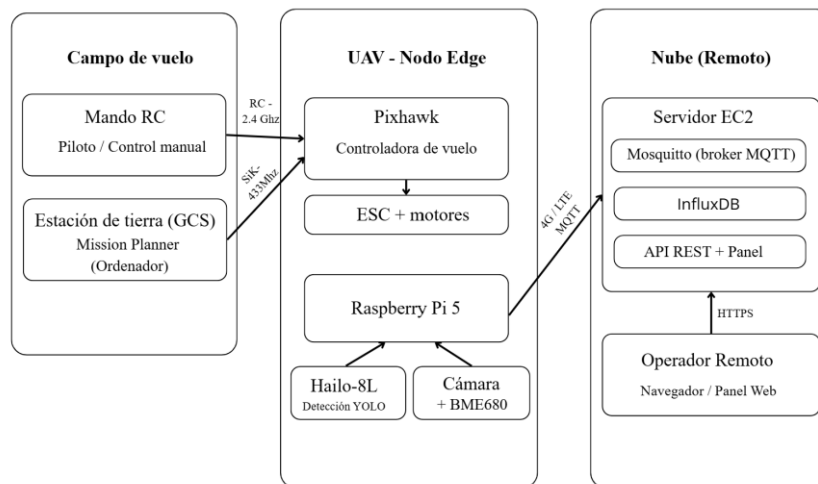


Figura 4.1. Arquitectura general del sistema en tres planos.

#### 4.1.1. Plataforma: el kit Holybro X500 V2.

La elección de la estructura mecánica y del conjunto de potencia no fue inmediata, sino el resultado de un proceso en el que se valoraron tres vías distintas antes de llegar a la solución final:

1. **Montaje a partir de piezas sueltas:** Se evaluaron chasis clásicos como el F450, el F550 o el Tarot 650. Aunque los hexacópteros ofrecían mayor capacidad de carga y redundancia ante el fallo de un motor, esta opción obligaba a comprar cada elemento por separado, validar manualmente las compatibilidades y asumir un gran número de soldaduras y ajustes mecánicos, con un alto riesgo de desajustes.
2. **Reutilización de un dron existente:** Se estudió reutilizar un dron disponible en el club de aeromodelismo, pero se descartó porque utilizaba una controladora cerrada (DJI Naza) sin firmware abierto ni protocolos estándar que permitieran recibir órdenes desde un ordenador externo.
3. **Adquisición de un kit integrado (solución adoptada):** Se optó finalmente por el kit Holybro X500 V2. Esta alternativa reúne en un único paquete componentes validados por el fabricante para operar conjuntamente, lo que reduce el número de soldaduras y garantiza la compatibilidad. Además, incluía el núcleo de control del sistema: la controladora Pixhawk 6X, el módulo GPS/brújula M10 y el par de radios de telemetría de 433 MHz.

Esta decisión supuso asumir un compromiso de diseño: al tratarse de un cuadricóptero de 500 mm en fibra de carbono, carece de la redundancia de seis motores que ofrece un hexacóptero. No obstante, para un prototipo experimental probado en un campo de vuelo controlado y bajo supervisión, se priorizó la compatibilidad entre componentes, la disponibilidad inmediata del sistema y el uso de una arquitectura electrónica abierta. La transición hacia una configuración multirrotor redundante queda planteada como una línea de desarrollo futura.



A partir de esta base estructural, los elementos principales del conjunto de potencia quedan definidos de la siguiente manera:

- **Estructura y chasis:** Chasis cuadricóptero de 500 mm de diagonal, fabricado en fibra de carbono con brazos plegables, lo que facilita el transporte a la zona de vuelo y proporciona una base rígida para reducir las vibraciones transmitidas a la controladora.
- **Motores y hélices:** Cuatro motores *brushless* de 2216 (920 KV) combinados con hélices de 10 pulgadas (1045), equilibrados para proporcionar el empuje necesario manteniendo un consumo contenido en vuelo estacionario (*hover*).
- **Variadores de velocidad (ESC):** Cuatro controladores electrónicos de velocidad de 20 A a 30 A, encargados de modular la corriente de los motores en función de las señales PWM generadas por el autopiloto.
- **Batería de alimentación:** Batería Li-Po de 4 celdas (4S, 14,8 V) con una capacidad de 5.000 mAh, dimensionada para sostener el consumo conjunto del grupo motopropulsor (motores + hélices), de la Raspberry Pi 5 y de los sensores a bordo durante las misiones de prueba.

Una vez ensamblado todo el dron, el chasis, la electrónica de control, la Raspberry Pi 5 con el acelerador Hailo y la batería, se comprobó el peso real en la báscula (Figura 4.2). El peso total al despegue (MTOW) quedó fijado en 1.817 g.



Figura 4.2. Comprobación en báscula del peso final del UAV: 1.817 g.

En la Tabla 4.1 se presenta el desglose de masas por componente, lo que confirma que, incluso con el peso del cableado y de los anclajes estructurales, el dron se mantiene dentro del rango de sustentación nominal de la plataforma para operar de forma estable y con margen de maniobra.

TABLA 4.1. BALANCE DE MASAS DE LA AERONAVE (WEIGHT BUDGET)

| Componente                    | Modelo / Especificación                   | Cantidad | Peso unitario (g) | Peso total (g) |
|-------------------------------|-------------------------------------------|----------|-------------------|----------------|
| Estructura y brazos           | Fibra de carbono Holybro X500 V2          | 1        | 420               | 420            |
| Grupo motopropulsor           | Motores Holybro 2216 920 KV + ESC 20A     | 4        | 85                | 340            |
| Hélices                       | Palas 1045 (10" × 4.5)                    | 4        | 14                | 56             |
| Unidad de control de vuelo    | Pixhawk 6X + Power Module                 | 1        | 55                | 55             |
| Navegación y telemetría       | Holybro M10 GNSS + Radio SiK 433 MHz + RC | 1        | 62                | 62             |
| Nodo Edge de cómputo          | Raspberry Pi 5 (4 GB) + Carcasa           | 1        | 68                | 68             |
| Acelerador de IA              | Raspberry Pi AI HAT+ (Hailo-8L)           | 1        | 18                | 18             |
| Sensorización y visión        | Cámara Raspberry Pi (CSI) + BME680        | 1        | 15                | 15             |
| Comunicaciones celulares      | Módem USB 4G/LTE + Alargador              | 1        | 35                | 35             |
| Batería principal             | Li-Po 4S 14,8 V 5000 mAh (60C)            | 1        | 490               | 490            |
| Miscelánea y cableado         | Tornillería, bridas, conectores XT30/XT60 | -        | 45                | 45             |
| Masa Total al Despegue (MTOW) | Configuración completa SAR                | -        | -                 | 1817 g         |

Además de controlar el peso estructural, la viabilidad del dron en una misión real depende directamente de la autonomía que proporciona la batería Li-Po de 4 celdas 14,8 V y 5.000 mAh, equivalente a 74 Wh.

$$E = V * Ah = 14,8 \text{ V} * 5 \text{ Ah} = 74 \text{ Wh}$$

La batería debe alimentar al mismo tiempo el grupo motopropulsor en vuelo estacionario, y toda la electrónica de la misión, como la Raspberry Pi 5, el módulo de inferencia Hailo-8L, la cámara CSI, los sensores y el módem 4G.

Para un peso total en orden de vuelo (AUW) de 1.817 g, el empuje requerido por cada uno de los cuatro motores es de 454,25 g. Considerando una eficiencia motopropulsora

media de  $\eta \approx 7,4 \text{ g/W}$  [31]. Este valor se toma como referencia de un ensayo de banco realizado para un motor AIR2216 de 920 KV con una hélice T1045 a 16 V, en el que se obtuvo una eficiencia de 7,39 g/W al 55% de acelerador. Puesto que Holybro no publica una curva completa de eficiencia para su conjunto 2216 KV920–1045, este valor debe considerarse una aproximación y no una especificación oficial del fabricante. La potencia eléctrica demandada por los motores se calcula según:

$$P_{\text{motores}} = \frac{m_{\text{AUW}}}{\eta} = \frac{1.817 \text{ g}}{7,4 \text{ g/W}} \approx 245,6 \text{ W} \Rightarrow I_{\text{motores}} = \frac{245,6 \text{ W}}{14,8 \text{ V}} \approx 16,6 \text{ A}$$

Sumando el consumo del subsistema electrónico de procesamiento, percepción y comunicaciones ( $P_{\text{elec}} \approx 14,25 \text{ W}$  /  $I_{\text{elec}} \approx 0,96 \text{ A}$  a 14,8 V), la demanda continua total del sistema en estacionario asciende a  $P_{\text{total}} \approx 260 \text{ W}$  ( $I_{\text{total}} \approx 17,55 \text{ A}$ ), tal y como se desglosa en la Tabla 4.2.

Aplicando un margen de descarga segura del 80 % (DoD) sobre la capacidad nominal para preservar la vida útil de las celdas de litio y asegurar una reserva de aterrizaje, la autonomía estimada de vuelo continuo resulta:

$$\text{Autonomía calculada} = \frac{C_{\text{bat}} \times \text{DoD}}{I_{\text{total}}} \times 60 \text{ min} = \frac{5.0 \text{ Ah} \times 0.8}{17,55 \text{ A}} \times 60 \text{ min} \approx 13,7 \text{ minutos}$$

de vuelo continuo en misión SAR.

Este tiempo de vuelo es suficiente para realizar los barridos de búsqueda en el área de prueba y aterrizar con seguridad. No obstante, esta cifra es inferior a los 18 minutos declarados por el fabricante para el chasis sin carga útil. Esto refleja de manera realista el impacto que supone incorporar todo el subsistema de procesamiento y comunicaciones (Raspberry Pi 5, Hailo-8L, módem 4G y sensores) en el peso total en orden de vuelo (AUW).

TABLA 4.2. BALANCE DE POTENCIA Y ESTIMACIÓN DE AUTONOMÍA (POWER BUDGET)

| Subsistema                     | Tensión nominal (V)      | Corriente media en hover (A) | Consumo de potencia (W) |
|--------------------------------|--------------------------|------------------------------|-------------------------|
| Motores en estacionario (4×)   | 14,8 V (Li-Po 4S)        | 16,60 A (4,15 A por motor)   | 245,6 W                 |
| Pixhawk 6X + GPS + Radio 433   | 5,0 V (vía Power Module) | 0,45 A                       | 2,25 W                  |
| Raspberry Pi 5 + Hailo-8L      | 5,0 V (vía BEC dedicado) | 1,70 A                       | 8,50 W                  |
| Módem 4G + Cámara CSI + BME680 | 5,0 V (bus RPi)          | 0,7 A                        | 3,50 W                  |
| Total Sistema (Hover)          | 14,8 V                   | A                            | ~259,85 W               |

#### 4.1.2. Controladora de vuelo

La controladora de vuelo (*Flight Controller*) es el "cerebro" encargado de hacer que el dron vuele de forma estable y segura. Su trabajo principal es medir continuamente cómo se mueve la aeronave mediante sus sensores internos (giróscopos y acelerómetros) y corregir la velocidad de cada motor decenas de veces por segundo. Esta tarea es crítica:

si el procesador se quedara colgado, aunque fuera por un instante, el dron se desestabilizaría y caería. Por eso, este control básico de vuelo se ejecuta de forma aislada en la controladora y no en la Raspberry Pi, lo que deja a esta última libre para procesar la cámara, los sensores y la conexión a internet, sin interferir en la estabilidad del vuelo.

A la hora de elegir la controladora, el requisito principal era que fuese un sistema abierto que permitiese recibir órdenes desde un ordenador externo:

- Se descartaron opciones comerciales cerradas (como la DJI Naza), ya que no ofrecen código abierto ni permiten enviar comandos automáticos desde programas Python propios.
- También se descartaron las placas típicas de drones de carreras (con firmwares como Betaflight), porque están diseñadas para acrobacias manuales rápidas y no para misiones de navegación automática por puntos.
- Por estos motivos, se eligió la placa Pixhawk 6X junto con el firmware abierto ArduPilot. La Pixhawk 6X destaca por su fiabilidad física, ya que incorpora varios sensores redundantes y amortiguadores para mitigar las vibraciones del chasis. Además, ArduPilot utiliza el protocolo estándar MAVLink, lo que facilita el intercambio bidireccional de telemetría y comandos tanto con la estación de tierra como con la Raspberry Pi mediante scripts en Python.

Para que el dron conozca su posición en el mapa y su orientación, la controladora se conecta al módulo externo Holybro M10:

- **Receptor GPS:** Utiliza el chip u-blox M10, que se conecta a varias redes satelitales a la vez (GPS, Galileo, GLONASS) para proporcionar una posición rápida y precisa de latitud, longitud y altura.
- **Brújula digital (magnetómetro):** Está montada en la parte superior, sobre un mástil elevado, para alejarla de las interferencias electromagnéticas generadas por la batería y los cables de los motores al girar. Esta referencia permite determinar el rumbo (*heading*) de la aeronave en todo momento y orientar su navegación.

La Pixhawk 6X centraliza todo el cableado básico de vuelo:

- **Entrada RC (RC IN):** Conectada al receptor de radio FlySky (2,4 GHz) para que el piloto pueda tomar el control manual del mando en cualquier momento ante imprevistos.
- **Salidas de motor (PWM 1 a 4):** Conectadas a los cuatro variadores (ESC) que suministran corriente a los motores.
- **Puerto TELEM 1:** Conectado a la radio de telemetría a 433 MHz, lo que permite ver en el portátil de campo parámetros como la batería o la altura sin depender de que haya internet.
- **Puerto TELEM 3 (UART):** Conectado por cable directamente a los pines de la Raspberry Pi 5 (/dev/ttyAMA0), lo que permite el envío mutuo de datos y órdenes MAVLink entre ambas placas.

#### 4.1.3. Raspberry Pi y HAT de inferencia

Mientras que la controladora de vuelo (Pixhawk 6X) se encarga exclusivamente de estabilizar la aeronave y mantenerla en el aire mediante cálculos en tiempo real, las tareas de alto nivel de la misión se trasladan a un segundo ordenador a bordo (*Companion Computer*). Este equipo ejecuta un sistema operativo Linux (Raspberry Pi OS) y asume tres responsabilidades principales: procesar las imágenes de la cámara para detectar personas, recopilar las lecturas del sensor ambiental BME680 y gestionar la comunicación con el servidor en la nube. Separar estas funciones en dos placas independientes evita que una sobrecarga provocada por la inteligencia artificial o una pérdida momentánea de conexión a internet interfiera con el control de los motores, garantizando la estabilidad del vuelo en todo momento.

Para desempeñar este rol, se seleccionó una Raspberry Pi 5, que actúa como nodo Edge y procesa la información directamente a bordo, sin necesidad de enviarla a un servidor externo. Este modelo aporta la potencia de cómputo necesaria para ejecutar varios procesos concurrentes y cuenta con una ventaja clave: la incorporación de un puerto PCIe 2.0 nativo, una interfaz de alta velocidad que elimina los cuellos de botella de ancho de banda y de latencia propios de las conexiones USB convencionales.

Disponer de esta interfaz PCIe resulta fundamental para resolver el principal reto de procesamiento del sistema: la ejecución en tiempo real del modelo de detección de personas (YOLO) sobre el flujo de vídeo. Si esta red neuronal se procesara directamente en la CPU de la Raspberry Pi, los núcleos se saturarían al 100 %, alcanzando entre 3 y 5 fotogramas por segundo (FPS). Esta tasa es insuficiente para una búsqueda aérea eficaz, en la que el dron avanza a velocidad de crucero y debe analizar cada imagen al instante para no sobrevolar a una persona sin detectarla.

Para solucionar esta limitación sin comprometer la autonomía ni añadir un peso excesivo, se incorporó el módulo Raspberry Pi AI HAT+, conectado directamente al bus PCIe mencionado. Este HAT integra la NPU Hailo-8L, un procesador específico para redes neuronales que consume menos de 3 W y eleva la tasa de inferencia a 25–30 FPS de forma estable. De este modo, la detección visual se realiza con fluidez en tiempo real a bordo y la CPU de la Raspberry Pi queda totalmente liberada para atender la telemetría, el sensor ambiental y las comunicaciones con la nube.

En la Raspberry Pi 5 se conectan los elementos principales de la misión:

- **Cámara digital:** Montada y orientada hacia el suelo con un ángulo de inclinación aproximado de 40 a 65 grados respecto de la vertical, esta cámara envía un flujo continuo de imágenes a la Raspberry Pi. Así, el modelo de IA puede detectar posibles víctimas en tiempo real. En la campaña de validación en vuelo, este rol se llevó a cabo mediante una cámara USB instalada sobre un soporte orientable (apartados 4.2.2 y 9.1).
- **Sensor ambiental Bosch BME680:** Conectado mediante el bus I2C (usando solo dos cables de datos, SDA y SCL), mide en cada instante la temperatura, la

humedad, la presión y la calidad del aire para identificar riesgos como la presencia de humo.

- **Enlace serie con la Pixhawk:** Un cable directo entre los pines de la Raspberry Pi (/dev/ttyAMA0) y el puerto de telemetría de la Pixhawk, por el que viajan los datos de vuelo y las órdenes automáticas en formato MAVLink.
- **Módem 4G/LTE:** Conectado por USB para proporcionar acceso a internet a través de la red móvil.

#### 4.1.4. Enlaces de radio a bordo.

Además de las dos placas principales de procesamiento (la Pixhawk y la Raspberry Pi), la aeronave incorpora los equipos de radiocomunicación encargados de mantener el enlace con tierra. Este apartado describe únicamente el hardware físico instalado a bordo; la arquitectura completa de comunicaciones (los protocolos empleados, los flujos de datos y los distintos niveles de criticidad) se desarrolla en detalle en el Capítulo 5.

- **Receptor de radiocontrol (FlySky FS-iA10B):** Se conecta directamente a la entrada RC IN de la controladora de vuelo. Es el extremo embarcado del canal de mando manual: recibe las órdenes que el piloto emite desde la emisora en tierra (FlySky FS-i6X) en la banda libre de 2,4 GHz y las traslada de forma inmediata a la Pixhawk sin pasar por ningún ordenador ni requerir conexión a internet. Constituye el canal de seguridad primario y obligatorio para cumplir con la normativa de vuelo, permitiendo al operador tomar el control manual directo ante cualquier imprevisto.
- **Radio de telemetría de 433 MHz:** El kit incluye un par de módulos transceptores de radiofrecuencia para telemetría. Uno de ellos se monta a bordo conectado al puerto TELEM 1 de la Pixhawk, mientras que el módulo gemelo se conecta por USB al ordenador portátil de campo. Este enlace punto a punto transmite en tiempo real las variables de vuelo (orientación, altura, coordenadas GPS y estado de la batería) a la estación de control en tierra (GCS). Al operar en la banda ISM de 433 MHz, funciona de forma totalmente independiente del mando de radiocontrol y no depende de la cobertura móvil ni de la red.
- **Módem de comunicaciones 4G/LTE:** A diferencia de los dos sistemas anteriores, que operan mediante enlaces de radiofrecuencia directos con la estación en tierra, el módem 4G conecta la aeronave a internet a través de la red celular comercial. Se conecta por USB a la Raspberry Pi 5 y se alimenta directamente de ella. Se menciona en este apartado para completar el conjunto de transmisores de radio que lleva el dron, aunque su funcionamiento y su papel en el envío de datos a la nube se explican junto con el resto del software del nodo *edge*.

#### 4.1.5. Interconexión entre procesadores: enlace MAVLink por UART

Las dos unidades de cómputo descritas en los apartados anteriores no operan de forma aislada. Para que la Raspberry Pi pueda desempeñar su función (consultar el estado del

vuelo y enviar órdenes autónomas a la aeronave), necesita un canal de comunicación directo y estable con la controladora. Este enlace es precisamente lo que permite que la Raspberry Pi actúe como un verdadero ordenador de a bordo (*Companion Computer*) y no como un simple procesador de vídeo: al comunicarse directamente con la Pixhawk, puede consultar en todo momento la posición GPS, la altitud o el nivel de batería, y enviarle instrucciones como armar motores, iniciar una misión o activar el retorno a casa (RTL).

La unión entre ambas placas se realiza mediante un enlace serie asíncrono UART. Se trata de una interfaz punto a punto sencilla y fiable que solo requiere dos líneas de datos cruzadas: transmisión (TX) y recepción (RX), además de la referencia de masa común (GND), conectando los dos dispositivos directamente sin necesidad de circuitería adicional ni buses compartidos.

Físicamente, la conexión se realiza a través de uno de los puertos serie auxiliares de la Pixhawk 6X. Como el enlace de telemetría de 433 MHz hacia la estación de tierra ya ocupa el puerto TELEM1, la Raspberry Pi se conecta a otro puerto disponible, en este caso TELEM3, cableando sus líneas a los pines GPIO correspondientes (utilizando el dispositivo `/dev/ttyAMA0` en Linux). De este modo, ambas placas quedan unidas por cable a pocos centímetros de distancia, dentro de la propia estructura del dron y sin depender de ningún enlace de radio. El detalle de esta interconexión física entre puertos serie y pines de bus y el resto de la sensorización se presenta en la Figura 4.3.

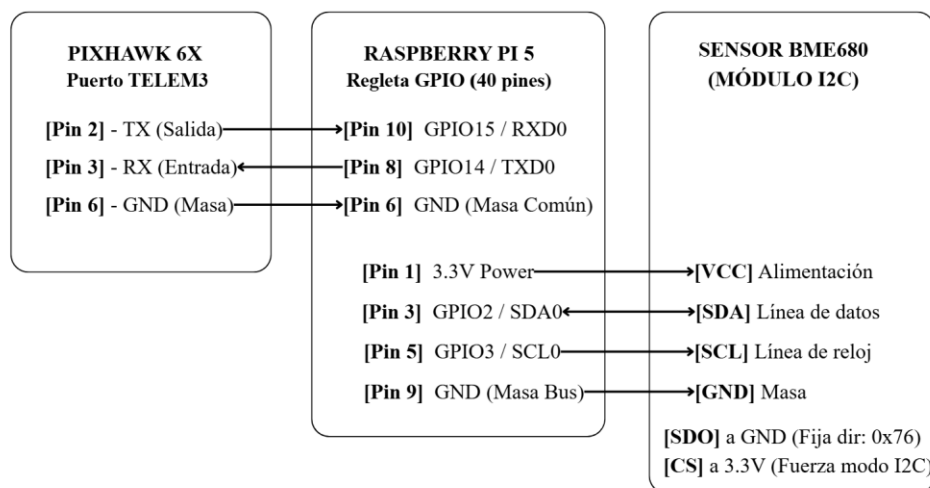


Figura 4.3. Esquema de conexión de Pixhawk 6X, Raspberry Pi 5 y BME680.

Por este cable serie circula el protocolo MAVLink, el mismo que la Pixhawk utiliza para comunicarse por radio con la estación de control en tierra. La única diferencia real está en el medio de transmisión: la estación de campo dialoga con el dron por ondas de radiofrecuencia (433 MHz), mientras que la Raspberry Pi lo hace por cable serie a bordo. Para la controladora de vuelo, ambos interlocutores son equivalentes porque hablan exactamente el mismo lenguaje. Esta equivalencia permite que la Raspberry Pi funcione,

a todos los efectos, como si fuera una estación de control embarcada en el propio vehículo, sirviendo de base para la lógica de control que se detallará en el Capítulo 5.

En la práctica, este enlace serie sostiene toda la cadena de mando del dron:

- **Envío de comandos:** Cuando se pulsa un botón en el panel web de control, la orden viaja por la red celular hasta la Raspberry Pi y, a través de este puerto UART, llega finalmente a la Pixhawk para que la ejecute.
- **Lectura de telemetría:** Del mismo modo, las coordenadas y el estado de la aeronave que luego se publican en la nube de AWS se obtienen a partir de las lecturas que la Raspberry Pi extrae de la Pixhawk por este mismo canal serie.

El enlace UART constituye así el punto de unión indispensable entre la electrónica de vuelo (la Pixhawk, encargada de mantener la estabilidad física en el aire) y la electrónica de misión (la Raspberry Pi, encargada de la IA, los sensores y las comunicaciones IP), lo que permite que ambas partes trabajen como un único sistema coordinado.

## 4.2. Montaje e integración del sistema

Una vez elegidos los componentes, se montó la aeronave con el kit Holybro X500 V2. El proceso se organizó de manera progresiva para revisar las conexiones eléctricas y la fijación física, paso a paso (Figura 4.4). A continuación, se describe cómo se estructuró el ensamblaje, desde el armado de los brazos y de la placa de potencia hasta la colocación final de la Pixhawk y de la Raspberry Pi.



Figura 4.4. Despiece inicial de los componentes del kit Holybro X500 V2.

### 4.2.1. Ensamblaje de la estructura y de la unidad de control de vuelo

El montaje comenzó con los cuatro brazos de fibra de carbono (Figura 4.5). Al tratarse del kit semiensamblado de Holybro, cada brazo ya incluye de fábrica el motor brushless y su variador de velocidad (ESC), integrados con el cableado guiado por el interior del tubo. La preparación previa consistió en montar las abrazaderas de fijación en los extremos interiores de los tubos (Figura 4.4b), dejando los conectores de alimentación y



señal listos para su conexión con la placa central y verificando que el sentido de giro asignado a cada motor (dos horarios CW y dos antihorarios CCW) coincidiese con el esquema de vuelo del cuadricóptero.

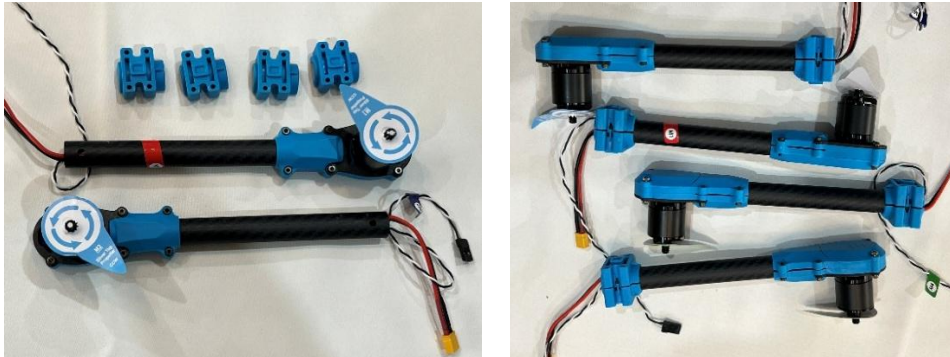


Figura 4.5. Preparación y premontaje de los brazos del chasis.

(a) Brazos preensamblados de fábrica con motores y variadores junto a los soportes de fijación; (b) Conjunto de los cuatro brazos con las abrazaderas ya montadas antes de su acople al chasis central.

A continuación, se procedió al montaje de la sección central e inferior del chasis:

1. **Estructura de raíles y bandejas inferiores:** Sobre dos tubos guía de fibra de carbono se instalaron los soportes con amortiguadores de goma y las dos plataformas auxiliares: una bandeja alargada para sujetar la batería Li-Po mediante correas de fijación y una base secundaria destinada al anclaje del mástil del módulo GNSS/brújula (Figura 4.6a).
2. **Placa inferior y distribución de potencia (PDB):** Se acopló la placa inferior del cuerpo central a los raíles y, en su zona central, se atornilló la placa de distribución de potencia (PDB) sobre separadores aislantes. En este mismo paso se conectó el cable principal de alimentación con su conector XT60, encargado de suministrar la corriente de la batería a la electrónica de a bordo (Figura 4.6b).
3. **Tren de aterrizaje y anclaje de brazos:** Se fijaron las patas verticales de fibra de carbono bajo la estructura para proporcionar la altura libre necesaria respecto al suelo (Figura 4.7a). Seguidamente, los cuatro brazos premontados se anclaron a sus soportes articulados en la placa central y se enchufaron las tomas de alimentación de cada variador de velocidad (ESC) directamente a las salidas XT30 de la PDB (Figura 4.7b).

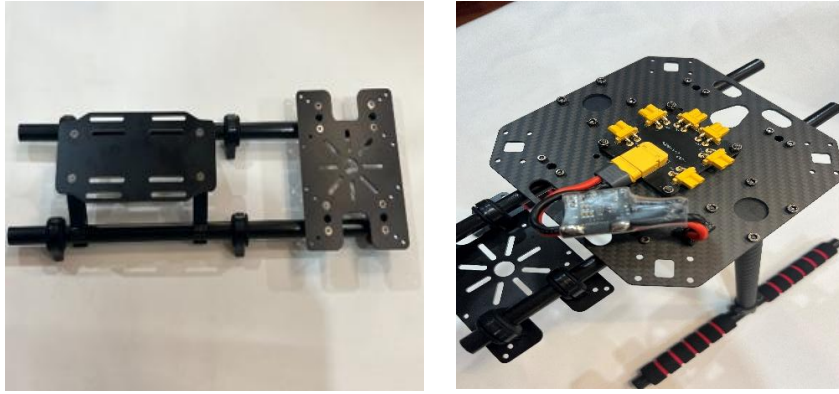


Figura 4.6. Ensamblaje del módulo inferior y de la placa PDB.

**(a)** Raíles tubulares con las bandejas para la batería y el mástil GNSS; **(b)** Fijación de la placa base inferior y montaje de la placa de distribución de potencia (PDB).



Figura 4.7. Montaje del tren de aterrizaje y del cableado de potencia.

**(a)** Fijación del tren de aterrizaje vertical de fibra de carbono; **(b)** Acople de los cuatro brazos articulados y conexión de las líneas de alimentación de los variadores (ESC) a la PDB.

Con el módulo inferior ensamblado, se completó la estructura mecánica central mediante la colocación de la placa superior de fibra de carbono (Figura 4.8a). A través de su apertura central se guió el mazo de cables de señal procedente de los cuatro variadores de velocidad, dejando el chasis cerrado y protegido frente a tensiones mecánicas.

Sobre esta placa superior se instaló la controladora de vuelo Pixhawk 6X, ubicada en el centro geométrico de la aeronave y fijada mediante una almohadilla de espuma antivibración para aislar sus giróscopos y acelerómetros internos del ruido mecánico generado por los motores (Figura 4.8b). Una vez fijada la controladora, se procedió al conexionado de sus puertos de control primarios:

- Salidas de control de motor: Las cuatro líneas de señal de los variadores de velocidad (ESC) se conectaron a las salidas PWM de la regleta principal (canales

MAIN 1 a 4), respetando la correspondencia con el orden y el sentido de giro de cada brazo.

- **Alimentación y monitorización:** Se conectó el cable procedente del módulo de potencia (Power Module) al puerto POWER1 de la Pixhawk, lo que suministró la tensión de funcionamiento regulada y permitió la lectura analógica de voltaje y corriente de la batería Li-Po.

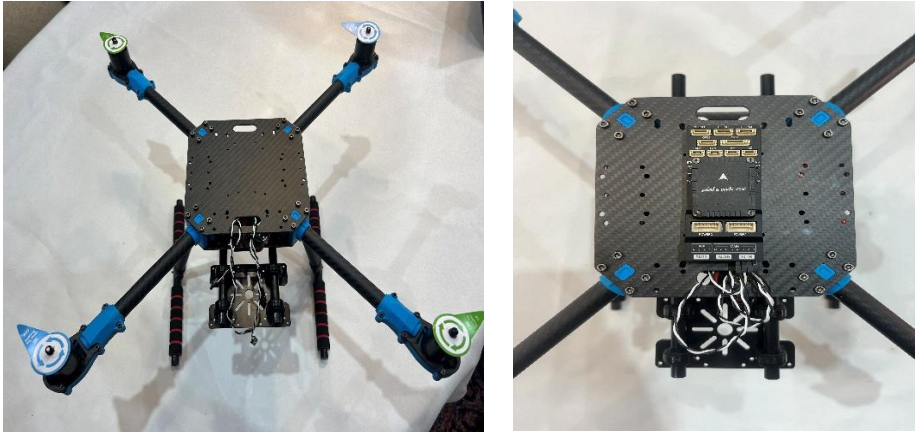


Figura 4.8. Integración de la controladora Pixhawk 6X en el chasis.

(a) Cerramiento del chasis con la placa superior de fibra de carbono y guiado del cableado de señal; (b) Fijación central de la Pixhawk 6X sobre una almohadilla antivibración y conexionado de las líneas PWM de los variadores (canales MAIN 1 a 4).

A continuación, se integraron los periféricos de navegación, el canal de telemetría local de vuelo y el enlace de radiocontrol (Figura 4.9):

- **Módulo GNSS y brújula digital (Holybro M10):** Se fijó en la bandeja secundaria mediante su soporte y se conectó al puerto GPS1 de la Pixhawk 6X. Esta disposición aleja el magnetómetro de los campos electromagnéticos generados por la batería y la placa de distribución de potencia (PDB), garantizando una lectura de rumbo fiable y una estimación de posición precisa.
- **Radioenlace de telemetría (433 MHz):** El transceptor embarcado se fijó sólidamente a un lateral del chasis mediante bridas de sujeción de nylon para amortiguar las vibraciones, conectando su interfaz serie al puerto TELEM1 de la controladora y orientando su antena hacia el exterior.
- **Receptor de radiocontrol FlySky (2,4 GHz):** Se montó el receptor en el plano superior del chasis y se conectó a la entrada RC IN de la Pixhawk, garantizando la línea de seguridad para el pilotaje manual directo por radiofrecuencia.

Con la electrónica de vuelo primaria instalada y cableada, la aeronave queda lista para proceder a la segunda fase de montaje: la integración del ordenador de a bordo Raspberry Pi 5, del acelerador Hailo-8L y de los sensores de la misión SAR.

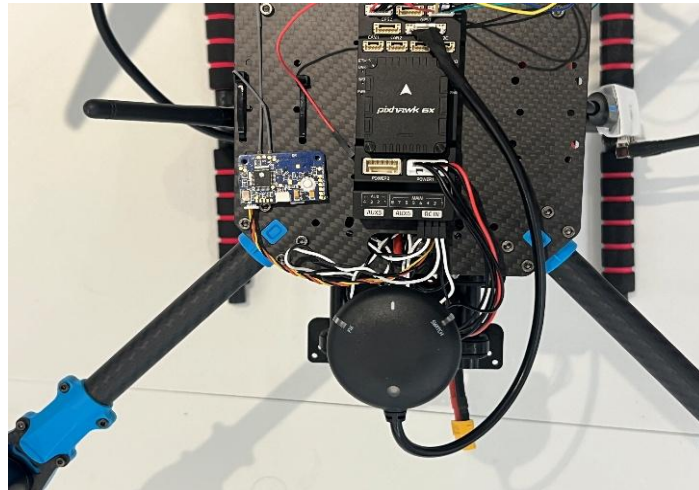


Figura 4.9. Integración de módulos GNSS, telemetría SiK y receptor RC.

Por último, con las hélices instaladas respetando el sentido de giro de cada motor (dos horarias CW y dos antihorarias CCW), la parte mecánica y de vuelo primario queda finalizada (Figura 4.10). En este punto, la aeronave dispone de la sustentación y del control básico de la estabilidad, y está lista para incorporar el ordenador a bordo.



Figura 4.10. Plataforma de vuelo ensamblada con el grupo motopropulsor.

#### 4.2.2. Integración de la unidad de procesamiento embarcada (nodo *edge*)

Una vez completada la plataforma de vuelo primaria, se procedió a instalar el nodo de cómputo embarcado, responsable del procesamiento de imágenes en tiempo real, de la ejecución de los algoritmos de inferencia y de la pasarela de comunicaciones IoT (Figura 4.11).



El núcleo de este bloque está conformado por una Raspberry Pi 5 complementada con el acelerador neuronal Hailo-8L HAT+ (Figura 4.11a). Para su acople mecánico a la aeronave, se instaló una placa de soporte auxiliar de fibra de carbono en la parte posterior del chasis, sobre la cual se fijó la carcasa protectora de la Raspberry Pi. La integración física de los diferentes periféricos y sensores sobre el ordenador de a bordo se resolvió siguiendo los siguientes criterios:

- Acelerador de IA e interfaz de sensores: El HAT Hailo-8L se acopla directamente al puerto PCIe de la Raspberry Pi mediante su cable plano dedicado, empleando una regleta extensora de GPIO para elevar el conexionado y mantener accesibles los pines de comunicación serie (UART hacia la Pixhawk) y el bus I2C para el sensor ambiental BME680 (Figura 4.11b).
- Captura de vídeo: El diseño inicial integró el módulo de cámara oficial mediante el bus MIPI-CSI, con la óptica orientada hacia abajo para proporcionar la perspectiva aérea necesaria para el barrido en cuadrícula de la misión SAR. Durante la campaña de validación, el módulo CSI presentó problemas de conexión. Se reemplazó por una cámara USB de mejor calidad, montada sobre un soporte con inclinación regulable. Se ajustó al rango de 40 a 65 grados respecto de la vertical, con el que se había capturado el conjunto de datos de entrenamiento (apartado 8.3). Esto también facilitó la orientación de la carga óptica durante las pruebas. La estabilización mecánica de la cámara (*gimbal*) se incluye como trabajo futuro (apartado 12.4).
- Enlace celular 4G/LTE: Dado que la conexión directa del módem USB a la placa colisionaba mecánicamente con el recorrido de plegado del brazo trasero, se utilizó un cable USB de extensión. El módem 4G se desplazó y se fijó mediante cinta de doble cara técnica sobre una de las patas verticales del tren de aterrizaje, una ubicación que además aleja el transceptor celular de los sensores inerciales de la Pixhawk 6X.

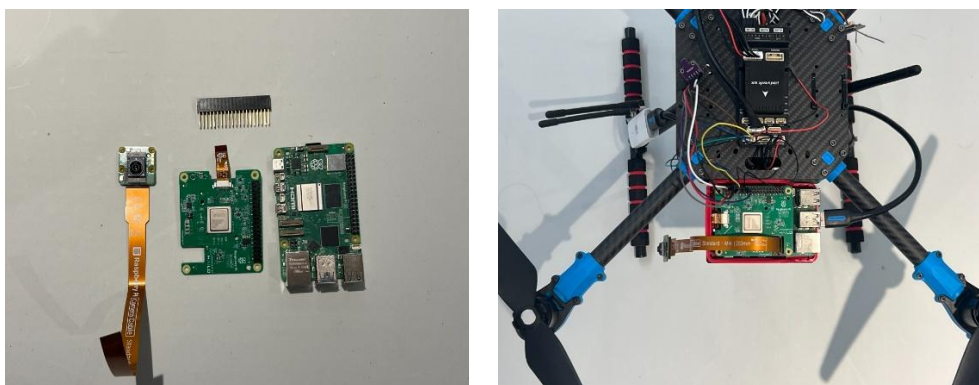


Figura 4.11. Integración del computador Raspberry Pi 5 con Hailo-8L.

Con esta disposición, el dron presenta una arquitectura física modular y compacta: la electrónica crítica de control de vuelo permanece aislada en el cuerpo central, mientras que el subsistema de cómputo, visión y comunicaciones celulares queda distribuido sin comprometer la aerodinámica ni el centro de gravedad del vehículo.

#### 4.3. Configuración y calibración inicial

Una vez montado el armazón y antes de realizar cualquier vuelo, la controladora requiere una fase inicial de configuración y calibración. Su objetivo es que el autopiloto conozca las características físicas de la aeronave y el estado de sus sensores, de modo que pueda estabilizarla correctamente y superar las comprobaciones de seguridad previas al armado (*pre-arm checks*).

Esta puesta a punto se ha realizado con QGroundControl, una de las estaciones de control terrestre del ecosistema de ArduPilot. Aunque la operación de vuelo y la simulación del sistema se llevan a cabo con Mission Planner (Capítulo 5), para esta fase de configuración inicial se ha empleado QGroundControl por la claridad de su asistente de calibración, que guía el proceso paso a paso. Una vez calibrada la aeronave, el control pasa a Mission Planner, la estación utilizada durante la operación.

El proceso de calibración comprende las siguientes etapas:

- **Selección del tipo de aeronave (*frame*).** Se configura la controladora para un cuadricóptero en configuración X, lo que indica al firmware la disposición de los motores y el sentido de giro de cada uno.
- **Calibración del acelerómetro.** Se establecen las referencias de horizontalidad y de orientación de la aeronave, situándola en distintas posiciones que el asistente va solicitando, para que el autopiloto conozca su actitud real en el espacio.
- **Calibración de la brújula (magnetómetro).** Se calibra el compás integrado en el módulo GPS, girando la aeronave en torno a sus ejes hasta que el asistente da por concluida la medición. Es imprescindible para que el dron conozca su orientación respecto al norte, necesaria para el vuelo autónomo.
- **Comprobación de motores.** Mediante la herramienta de prueba de motores de QGroundControl, se ha accionado cada motor de forma individual para verificar dos aspectos esenciales de un multirrotor: que cada motor se corresponde con la posición asignada por el software (siguiendo el esquema numerado que muestra la propia herramienta) y que su sentido de giro es el correcto. En un cuadricóptero, los motores deben girar por pares en sentidos opuestos, de modo que un motor mal posicionado o que gire en sentido contrario impediría un vuelo estable.
- **Calibración del radiocontrol.** Se sincroniza el mando con la controladora, recorriendo todo el rango de cada palanca para que el autopiloto asocie correctamente cada canal a su función (acelerador, alabeo, cabeceo y guiñada).
- **Asignación de los modos de vuelo.** Se asocian los modos de vuelo (por ejemplo, *Stabilize*, *Loiter* o *RTL*) a los interruptores del mando, de modo que el piloto pueda conmutar entre ellos durante la operación.

Durante esta fase se presentó un problema que conviene documentar para mostrar cómo funciona la integración de hardware. Al conectar la controladora a QGroundControl, la radio de telemetría no aparecía como disponible, aunque el cableado del módulo parecía correcto. Después de revisar a fondo las conexiones, se determinó que el problema no estaba en el enlace de telemetría, sino en la alimentación. La batería no estaba conectada correctamente a la placa de distribución de potencia. De esta placa se alimentan tanto la controladora como el módulo de telemetría. Una vez restablecida la alimentación, la telemetría volvió a funcionar con normalidad. El problema, aunque fácil de arreglar, ilustra un principio común en la integración de hardware. Un fallo en un subsistema, como en este caso la falta de telemetría, puede deberse a otro subsistema, como la alimentación. Por eso es necesario revisar el sistema completo y no solo una parte.

Completadas estas calibraciones, la controladora supera las comprobaciones previas al armado, momento a partir del cual la aeronave puede armar motores de forma segura. Con ello, el dron queda listo para las pruebas de vuelo, que se abordan en el Capítulo 9.

## 5. CONTROL DE VUELO Y TELEMETRÍA

La arquitectura de comunicaciones constituye uno de los pilares centrales de este trabajo desde la perspectiva de las telecomunicaciones. Para no depender de un único canal de datos, se ha diseñado un sistema redundante basado en cuatro enlaces independientes, cada uno dimensionado para un nivel específico de criticidad, alcance y latencia. Esto permite que el dron esté siempre comunicado ante cualquier caída puntual.

A continuación, se detallan los cuatro enlaces desarrollados. Se ha separado lo que hace volar al dron de lo que procesa los datos y la misión, de forma que el sistema sea robusto:

- **Enlace 1. Radio RF (control manual de emergencia):** Enlace bidireccional por radiofrecuencia en la banda ISM de 2,4 GHz, establecido entre la emisora FlySky FS-i6X del piloto y el receptor FS-iA10B a bordo. Es el canal de seguridad fundamental: por normativa de la EASA y por protocolo operacional, el piloto debe poder asumir el control manual directo en cualquier instante ante un fallo imprevisto. Al no pasar por ningún sistema operativo ni depender de capas de red IP, ofrece una latencia mínima y la máxima fiabilidad a corta distancia.
- **Enlace 2. Telemetría P2P local (monitorización en tierra):** Enlace de datos punto a punto serie mediante transceptores SiK Radio en la banda de 433 MHz. Este canal transmite de forma continua las tramas MAVLink generadas por la controladora Pixhawk 6X (coordenadas GPS, altitud barométrica, actitud, estado de la batería y modos de vuelo) directamente al ordenador de campo mediante conexión USB, garantizando la supervisión en tiempo real del estado del dron, incluso en zonas rurales sin cobertura móvil.
- **Enlace 3. Red móvil 4G/LTE (control remoto y en la nube):** Conexión a internet mediante un módem USB 4G conectado a la Raspberry Pi 5. Mediante este enlace se establece una red privada virtual (VPN) segura y cifrada que conecta el dron con el servidor EC2 de AWS. Se permite así enviar telemetría IoT, recibir órdenes de navegación desde el panel web y operar la aeronave a distancia, más allá de la línea de vista (BVLOS).
- **Enlace 4. Red Wi-Fi local (respaldo de mantenimiento y banco de pruebas):** Conexión de corto alcance en la que la Raspberry Pi 5 se asocia a la red Wi-Fi del ordenador o de un router en tierra. No está pensado para volar a larga distancia por el alcance limitado del Wi-Fi convencional, sino como una vía rápida en el banco de trabajo para conectarse por SSH, depurar el código y descargar los registros de vuelo sin consumir datos de la SIM.

La Figura 5.1 resume la topología global resultante e ilustra la coexistencia entre los canales de radio directos en el campo de vuelo y los enlaces de datos a la nube. Asimismo, las especificaciones técnicas, los protocolos y los niveles de criticidad de cada interfaz se sintetizan en la Tabla 5.1.



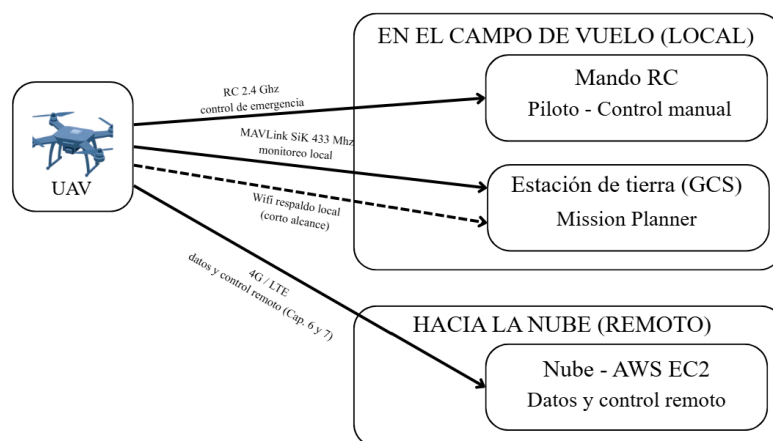


Figura 5.1. Arquitectura de los cuatro enlaces de comunicación del UAV.

TABLA 5.1. ARQUITECTURA JERÁRQUICA DE ENLACES DE COMUNICACIÓN DEL SISTEMA UAV

| Enlace / Canal                 | Banda / Frecuencia           | Protocolo / Capa               | Dispositivo a bordo             | Nodo en tierra / remoto       | Rol y nivel de criticidad                                  |
|--------------------------------|------------------------------|--------------------------------|---------------------------------|-------------------------------|------------------------------------------------------------|
| Enlace 1: Mando RC             | 2,4 GHz (AFHDS 2A)           | Propietario (PPM / i-Bus)      | Receptor FlySky FS-iA10B        | Emisora FlySky FS-i6X         | Crítico primario: control manual de seguridad (EASA).      |
| Enlace 2: Telemetría local     | 433 MHz (banda ISM)          | MAVLink v2.0 (serie asíncrona) | Transceptor SiK Radio (TELEM 1) | Módem USB SiK + PC GCS        | Supervisión directa: posición, batería y estados locales.  |
| Enlace 3: Conectividad celular | 4G/LTE (bandas B1/B3/B7/B20) | IP / VPN WireGuard / MQTT      | Módem USB LTE en RPi 5          | Instancia AWS EC2 + Dashboard | Misión remota: telemetría IoT, vídeo y telecontrol BVLOS.  |
| Enlace 4: Red Wi-Fi local      | 2,4 / 5,8 GHz (802.11ac)     | TCP/IP / SSH / MAVLink UDP     | Interfaz Wi Fi RPi 5 (wlan0)    | Router portátil / PC de campo | Mantenimiento: calibración en banco y depuración en campo. |

### 5.1. Telemetría de vuelo con MAVLink

Para comunicar la Pixhawk con el exterior, tanto con la estación de tierra en el campo como con la Raspberry Pi a bordo, se utiliza MAVLink (*Micro Air Vehicle Link*), un protocolo estándar en sistemas no tripulados abiertos. En este proyecto se emplea la versión MAVLink v2.0, que, frente a la original (1.0), aporta tres mejoras clave: amplía el espacio de identificadores de mensaje hasta 24 bits (lo que permite definir más de 16 millones de mensajes distintos), reduce dinámicamente el tamaño de la trama eliminando los ceros sobrantes de la carga útil (zero-truncation) y añade la posibilidad de firmar criptográficamente los paquetes para impedir que se cuelen órdenes falsas.

A diferencia de protocolos pensados para internet como HTTP o JSON, donde cada mensaje arrastra mucho texto, MAVLink es un protocolo binario de muy bajo consumo de ancho de banda. Esto permite transmitirlo por canales serie estrechos o por radioenlaces a 433 MHz sin saturar la línea.

TABLA 5.2. ESTRUCTURA DE LA TRAMA BINARIA MAVLINK V2.0

| Campo     | Descripción                     |
|-----------|---------------------------------|
| STX       | 0xFD (1 byte)                   |
| LEN       | (1 B) Longitud                  |
| INC       | (1 B) Flags de incompatibilidad |
| CMP       | (1 B) Flags de compatibilidad   |
| SEQ       | (1 B) N.º de serie              |
| SYS       | (1 B) Sys ID                    |
| COMP      | (1 B) Comp ID                   |
| MSG       | (3 B) Msg ID                    |
| PAYLOAD   | (0-255 B)                       |
| CRC       | (2 B) CRC16-M                   |
| SIGNATURE | (13 B, opcional)                |

Cada paquete MAVLink v2.0 se empaqueta en bytes siguiendo una estructura de la Tabla 5.2:

- **STX (0xFD, 1 byte):** Marca el inicio exacto de la trama e indica al receptor que se trata de un paquete MAVLink v2.0 (en la versión 1.0 este byte era 0xFE).
- **LEN (1 byte):** Indica cuántos bytes útiles contiene el mensaje (de 0 a 255).
- **INC / CMP (2 bytes):** Banderas de incompatibilidad y compatibilidad. Sirven para que el receptor sepa si el paquete incluye funciones especiales obligatorias (como la firma digital) antes de intentar procesarlo.
- **SEQ (1 byte):** Número de secuencia correlativo (0 a 255). Permite a la Raspberry Pi y a la estación de tierra detectar de inmediato si se ha perdido algún paquete debido a interferencias en el enlace de radio.
- **SYS ID / COMP ID (2 bytes):** Dirección del emisor. Identifica qué vehículo habla (por ejemplo, el UAV principal con ID 1) y qué parte interna genera el dato (ID 1 para el autopiloto Pixhawk o ID 190 para la Raspberry Pi).
- **MSG ID (3 bytes):** Identificador numérico que indica qué tipo de información contiene el paquete.
- **PAYLOAD (0-255 bytes):** Información útil en formato binario (posiciones, voltajes, velocidades).
- **CRC (2 bytes):** Código de redundancia cíclica (CRC-16/MCRF4XX). Garantiza que, si un byte se corrompe por ruido electromagnético o por una cobertura deficiente, el paquete se descarta de inmediato para evitar procesar datos falsos.
- **SIGNATURE (13 bytes, opcional):** Bloque de seguridad con marca temporal y firma HMAC-SHA256 para autenticar el origen del comando.

Dentro de la amplia variedad de mensajes que define el protocolo, el sistema utiliza de forma continua cinco tipos principales:

- **HEARTBEAT (1 Hz):** El latido periódico que emite la controladora para confirmar que está viva, indicando el tipo de vehículo y el modo de vuelo activo

(GUIDED, AUTO, RTL). Es el primer paquete que cualquier software espera recibir para comprobar si hay conexión.

- **GLOBAL\_POSITION\_INT (5 Hz):** Proporciona la posición tridimensional calculada mediante la fusión de sensores: latitud y longitud en grados (con precisión de 7 decimales), altitud sobre el nivel del mar y altura relativa respecto al punto de despegue.
- **ATTITUDE (10 Hz):** Informa sobre la orientación espacial de la aeronave en los tres ejes: alabeo (*roll*), cabeceo (*pitch*) y guiñada (*yaw*), junto con sus respectivas velocidades angulares.
- **BATTERY\_STATUS (2 Hz):** Reporta la tensión total, la corriente instantánea demandada y el porcentaje de capacidad restante de la batería.
- **STATUSTEXT (eventual):** Envía avisos del autopiloto, como las alertas de los chequeos previos al armado (*pre-arm checks*) o la confirmación de los modos de seguridad.

Al agrupar toda la información de vuelo en estos mensajes, el protocolo MAVLink se convierte en el punto de conexión de todo el sistema: posibilita la supervisión de la aeronave en tiempo real desde la estación terrestre, la navegación automática mediante el script en Python y, al mismo tiempo, la provisión de las coordenadas GPS requeridas para geolocalizar de inmediato cada dato ambiental del sensor y cada detección de la cámara.

## 5.2. Estaciones de control en tierra (GCS)

Una estación de control terrestre (Ground Control Station, GCS) es el software principal mediante el cual el operador monitorea y dirige la aeronave durante las operaciones en el campo. Mediante su interfaz gráfica, el piloto observa en tiempo real en el mapa la ruta del dron, supervisa las variables de estado esenciales (actitud, nivel de batería, cobertura GNSS), modifica los parámetros internos del autopiloto y organiza misiones de vuelo mediante puntos de paso (waypoints). En este sistema, la telemetría local se transmite a la GCS a través del transceptor SiK a una frecuencia de 433 MHz.

Conviene destacar que una GCS no actúa como un mero visor pasivo de datos, sino como un nodo MAVLink activo y bidireccional:

- **Canal de subida (*uplink*):** Permite al operador transmitir comandos directos al autopiloto, tales como el armado de motores, cambios de modo de vuelo o la orden de retorno al punto de origen (*Return to Launch*, RTL). Cada operación realizada desde la interfaz gráfica se traduce internamente en la creación y el envío de un paquete binario MAVLink al vehículo.
- **Canal de bajada (*downlink*):** Recibe de forma continua el flujo de telemetría emitido por la Pixhawk y procesa las variables de navegación para actualizar el panel de instrumentos y la posición en el mapa.

En este proyecto se ha seleccionado Mission Planner como GCS de referencia por su plena compatibilidad con el ecosistema ArduPilot y por incluir de serie el simulador

SITL. Esta elección cumple una doble función metodológica: durante la fase de desarrollo, sirve como interfaz visual para validar el comportamiento del vehículo en el simulador, mientras que en la operación física proporciona al piloto la conciencia situacional necesaria para una supervisión segura.

Un principio fundamental del diseño es que la estación de tierra y el script de control en Python operan como dos clientes MAVLink independientes y desacoplados sobre el mismo autopiloto. No existe jerarquía ni dependencia entre ellos: ambos dialogan de forma autónoma con la aeronave mediante el mismo protocolo.

De este modo, cuando el script ejecuta una maniobra o cambia de modo de vuelo, la actualización se refleja al instante en el panel de Mission Planner simplemente porque ambos leen el mismo flujo de telemetría de la controladora. La división de procesos, representada en la Figura 5.2, facilita el monitoreo en tiempo real de la misión autónoma desde la estación terrestre, manteniendo siempre la posibilidad de intervenir manualmente ante cualquier imprevisto.

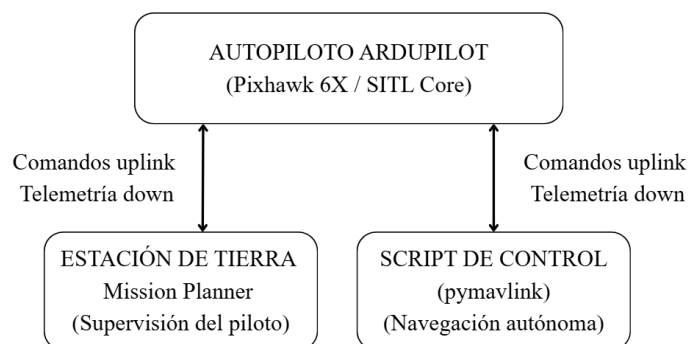


Figura 5.2. Topología de clientes concurrentes MAVLink.

### 5.3. Control autónomo con pymavlink

Para que el dron sirva realmente en una misión de búsqueda y rescate, no basta con pilotarlo a mano desde tierra. La aeronave debe poder rastrear el terreno por sí misma. Así, el piloto no tiene que estar al mando de los controles todo el tiempo. El dron puede recorrer la zona siguiendo un patrón geométrico exacto, mientras la cámara y los sensores escanean el entorno en tiempo real.

Esta lógica de guiado se ha desarrollado en Python utilizando la librería pymavlink. El script funciona como el cerebro de la misión a bordo. Abre un canal de comunicación con la Pixhawk, ya sea por el puerto serie de la Raspberry Pi o por sockets de red, durante las pruebas. Luego traduce la misión en una serie ordenada de comandos de navegación.

El programa no envía órdenes de inmediato al arrancar, sino que permanece a la escucha hasta recibir el primer paquete HEARTBEAT. Este latido periódico confirma que la controladora de vuelo está encendida y permite al script sincronizarse con ella para empezar a dialogar. A partir de ese momento, el script ejecuta la misión siguiendo cinco etapas bien diferenciadas:

1. **Carga de waypoints (*Mission Upload*):** Envía la lista de puntos de paso con sus coordenadas GPS a la memoria interna de la Pixhawk mediante el protocolo de misión de MAVLink.
2. **Comprobaciones y armado (*Pre-arm checks*):** El firmware impide que los motores giren hasta que los sensores se calibren y la señal GNSS esté completamente estable. Por eso, el script revisa estos avisos y solo da la orden de armar cuando el sistema confirma que es seguro hacerlo.
3. **Despegue guiado (*GUIDED Takeoff*):** Ordena al dron que suba verticalmente hasta la altura de seguridad establecida. En ArduPilot, cambiar al modo automático con el dron en el suelo no hace que vuele por sí solo. Primero hay que despegar en modo GUIDED para asegurar una subida estable antes de empezar a moverse horizontalmente.
4. **Ejecución de la ruta (*AUTO Mode*):** Al alcanzar la altura programada, cambia al modo AUTO. Así, el autopiloto empieza a recorrer los puntos de la cuadrícula de búsqueda.
5. **Retorno y aterrizaje (*RTL & Land*):** Al completarse el último punto, se activa automáticamente el modo RTL (Return to Launch). El dron regresa al punto de despegue, aterriza y apaga los motores por motivos de seguridad.

La Figura 5.3 muestra la máquina de estados de este control. Se puede ver que el script nunca cambia de fase sin recibir previamente la confirmación real de la controladora.

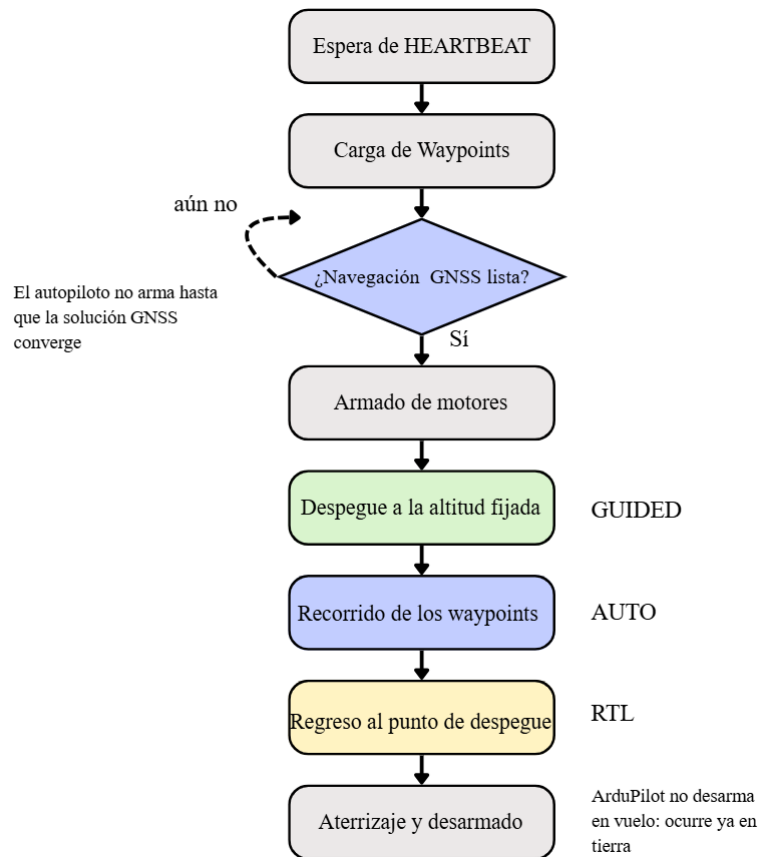


Figura 5.3. Secuencia de la misión autónoma ejecutada por el script de control.

#### 5.4. Simulación SITL con ArduPilot

Antes de transferir el control a la aeronave física, la lógica de guiado se ha desarrollado y validado en simulación por tres motivos fundamentales: reduce el riesgo de daños materiales o personales derivados de posibles errores de programación, separa el desarrollo del software de la disponibilidad del chasis y ofrece un entorno controlado en el que se puede repetir la misma misión bajo condiciones idénticas tantas veces como se requiera.

Para esto se utiliza SITL (Software-in-the-Loop), el simulador de ArduPilot. A diferencia de los simuladores genéricos que apenas replican el vuelo, SITL compila y ejecuta el código auténtico del firmware de vuelo en el procesador del ordenador. Simplemente sustituye los sensores físicos (giróscopos, acelerómetros, GPS) y los motores por representaciones dinámicas matemáticas. De este modo, el autómata y los modos de seguridad que se evalúan en la simulación son los mismos que posteriormente operarán la Pixhawk 6X real.

La simulación se inicia en Mission Planner, que descarga y ejecuta el binario del multirrotor. Para que las pruebas sean iguales al escenario real de operación, el punto de origen de la aeronave simulada se ha fijado en las coordenadas exactas del campo del Club de Aeromodelismo Alas de Galapagar usando el parámetro de inicio.

--home=40.5979097,-3.9988503,880,0

donde los dos primeros elementos indican la latitud y la longitud. El tercero indica la altura sobre el nivel del mar en metros. El cuarto indica la dirección inicial en grados. De este modo, los planes de vuelo diseñados en el simulador pueden aplicarse directamente en la vida real sin necesidad de modificar las coordenadas.

La Figura 5.4 muestra la misión de búsqueda planeada en el mapa de Galapagar en la interfaz de Mission Planner.



Figura 5.4. Misión de vuelo planificada en Mission Planner sobre el campo del Club Alas de Galapagar.

### El puente MAVLink: un autopiloto, varios clientes

Durante el desarrollo, se identifica una limitación en la capa de transporte: el núcleo SITL expone su telemetría a través de un único socket TCP en el puerto local 5760, que permite a la vez un único proceso conectado. Sin embargo, el sistema requiere la conexión simultánea de dos programas independientes: la estación de tierra Mission Planner (para monitorizar el mapa) y el script en Python (para comandar la misión).

Para resolver esta limitación en el entorno de pruebas local, Mission Planner se configuró como un puente MAVLink (*MAVLink bridge*): toma el flujo TCP exclusivo del simulador y lo replica en datagramas UDP multicliente a través de la interfaz de bucle local (*loopback*), reenviando a 127.0.0.1:14550 (para la propia interfaz de Mission Planner) y a 127.0.0.1:14551 (hacia el script de control en Python). Esta configuración permite depurar la concurrencia y la lógica de mando en una única máquina de desarrollo antes de desacoplar los procesos de la Raspberry Pi y de la red de comunicaciones.

Esta distribución de datos es totalmente bidireccional y transparente:

- **En bajada:** Ambos clientes reciben simultáneamente la telemetría periódica del dron simulado.

- **En subida:** Cuando el script en Python emite una orden de armado o de cambio de modo, el mensaje viaja por el puente hasta el simulador sin ser alterado por la estación de tierra. Mission Planner simplemente muestra el cambio de estado en pantalla al recibir la confirmación del autopiloto.

Comprender esta topología de red es fundamental, ya que anticipa fielmente la arquitectura final del hardware: cuando el sistema se despliegue a bordo, la Raspberry Pi 5 conectada a la controladora asumirá ese mismo papel de multiplexor para compartir el puerto serie físico (/dev/ttyAMA0) entre el script de vuelo autónomo, el módulo de visión con IA y el servicio de telemetría local encargado de publicar los datos en AWS.

### 5.5. Vídeo FPV y enlace VTX

En el campo de las aeronaves no tripuladas, los sistemas FPV (*First Person View*) suelen emplear un transmisor de radiofrecuencia analógico dedicado (VTX) en la banda de 5,8 GHz. Esta tecnología se caracteriza por una latencia casi nula, de entre 1 y 5 ms. Esto es clave al pilotar de forma acrobática o al pilotar drones de carreras. Sin embargo, la transmisión analógica requiere canales de radio específicos, tiene una resolución limitada y la señal se degrada por el ruido cuando hay obstáculos o a mayor distancia.

En este proyecto, el aumento es muy distinto al del pilotaje FPV tradicional. Como se trata de una misión SAR con navegación controlada por la controladora de vuelo y telemetría independiente a través de MAVLink, el control de la aeronave no depende de que el piloto vea todo en tiempo real y de inmediato. Por eso, no se integra un enlace VTX analógico adicional. En su lugar, se elige una cámara digital conectada directamente a la Raspberry Pi 5. Así, se puede ejecutar el modelo de visión artificial a bordo de forma autónoma mediante Edge Computing. De este modo, las detecciones no dependen de la calidad ni de la estabilidad de un enlace de radio externo.

Como resultado de este diseño, el flujo de vídeo enviado a tierra solo sirve para la supervisión remota desde el puesto de mando. El sistema separa el almacenamiento de alta calidad de la transmisión en directo. La grabación completa se guarda localmente sin las limitaciones de ancho de banda del canal de radio. Al mismo tiempo, se envían a la nube un flujo digital optimizado mediante WebRTC y alertas ligeras con miniaturas fotográficas, ambos a través de MQTT. Así se logra un retardo inferior a un segundo, lo cual es suficiente para la supervisión humana. El sistema se mantiene ligero, resistente y eficiente en el uso de energía.



## 6. ARQUITECTURA IOT Y ADQUISICIÓN DE DATOS

### 6.1. Infraestructura AWS EC2

La arquitectura IoT del sistema se despliega en una instancia de EC2 de AWS, que actúa como servidor central para la recepción, el almacenamiento y la visualización de los datos de los sensores del UAV.

La instancia se ha creado a partir de una AMI de Ubuntu Server del tipo t3.small, ubicada en la región de España (eu-south-2). Este tipo ofrece un equilibrio adecuado entre capacidad de cómputo y coste para los servicios que ejecuta: el broker MQTT y la base de datos de series temporales. Durante su creación, también se ha generado un par de claves (key pair) que habilita el acceso remoto por SSH mediante autenticación de clave pública y prescinde del uso de contraseñas.

El control del tráfico entrante se gestiona mediante un grupo de seguridad. Cada regla de entrada habilita el acceso a un servicio concreto:

- **Puerto 22 (TCP):** Destinado a la administración remota mediante el terminal SSH.
- **Puerto 1883 (TCP):** Reservado para la mensajería MQTT con el broker Mosquitto.
- **Puertos 80 y 443 (TCP):** Para el tráfico web HTTP y HTTPS gestionado por nginx.
- **Protocolo ICMP:** Habilitado para tareas de diagnóstico de conectividad y comprobación de retardo (*ping*).
- **Puerto 8189 (UDP):** Asignado al servidor de medios (MediaMTX / WebRTC) para el transporte del flujo de vídeo en tiempo real hacia el operador; su señalización se resuelve sobre el puerto 443 existente y el enlace de publicación del dron viaja a través del canal privado de comunicaciones, por lo que no requiere exponer puertos adicionales.

Cabe señalar que el grupo de seguridad no incluye reglas para los puertos de la API REST ni de la base de datos. Ambos servicios son accesibles desde el exterior, pero a través del servidor web, según se describe en el apartado 7.2. Los únicos puertos expuestos a internet son, por tanto, los estrictamente necesarios.

Para el acceso administrativo por SSH y la mensajería MQTT se utilizó inicialmente la regla abierta 0.0.0.0/0 durante las primeras pruebas de conectividad básica. Sin embargo, en la arquitectura final, este acceso se asegura mediante una red privada virtual (VPN), lo que permite comunicar la Raspberry Pi, el ordenador de operación y la instancia de AWS de forma directa y cifrada, sin necesidad de exponer estos servicios a internet. Por su parte, la apertura pública de los puertos 80 y 443 se debe al comportamiento natural del servidor web, que permite el acceso al panel desde el navegador.

Como broker MQTT, se ha empleado Mosquitto, instalado mediante el gestor de paquetes de Ubuntu, junto con sus herramientas de línea de comandos. Su configuración se ha definido en un archivo independiente dentro del directorio `/etc/mosquitto/conf.d/`, en lugar de modificar el fichero principal, siguiendo la práctica recomendada de mantener las personalizaciones aisladas de la configuración por defecto. En dicho archivo se han establecido dos directivas:

```
listener 1883 0.0.0.0
```

```
allow_anonymous true
```

La directiva `listener 1883 0.0.0.0` obliga a Mosquitto a escuchar en el puerto estándar en todas las interfaces de red, lo que permite recibir paquetes procedentes de la Raspberry Pi a bordo. La directiva `allow_anonymous true` autoriza conexiones sin autenticación previa durante la fase de prototipado. El correcto funcionamiento del broker se validó localmente suscribiendo el cliente de terminal al *topic* general de sensores y comprobando la recepción instantánea de los mensajes de prueba.

## 6.2. Arquitectura de adquisición multidominio.

Durante una operación de búsqueda y rescate, el UAV captura simultáneamente información de distinta naturaleza: variables meteorológicas, telemetría de navegación, métricas de carga del computador a bordo y eventos de visión artificial. Para gestionar esta diversidad sin cuellos de botella, la adquisición se ha estructurado en torno a un principio fundamental: un proceso independiente para cada dominio de datos.

Toda la arquitectura responde, además, a una filosofía *edge-first*: el nodo embarcado captura y guarda sus datos siempre de forma autónoma y local, y los envía a la nube solo cuando la conexión lo permite, sin que el estado del enlace condicione nunca la toma de decisiones. Este planteamiento es fundamental en el rescate, donde la cobertura móvil en zonas rurales o de montaña es inestable y no puede considerarse garantizada.

Cada dominio corre en su propio proceso dentro de Linux, con su propia base de datos SQLite local y su propio *topic* MQTT. Esta separación resuelve dos problemas clave:

- **Concurrencia:** Si un único programa intentase escribir en una sola base de datos desde cuatro fuentes a la vez, las escrituras competirían por el acceso al archivo y acabarían bloqueándose entre sí. Al asignar a cada dominio su propia base de datos SQLite, cada proceso escribe en la suya sin interferencias entre sí.
- **Independencia y tolerancia a fallos:** Como los procesos no comparten memoria ni ejecución, si uno de ellos falla o se reinicia por cualquier motivo, no arrastra a los demás. Esta separación entre quien genera el dato y quien lo envía responde al patrón *productor-consumidor*, en el que ambos extremos quedan desacoplados gracias al buffer local, de modo que la captura no se detiene, aunque el envío falle.

A esta robustez se suma una clara ventaja de diseño: si en el futuro se quiere añadir un nuevo sensor (como una cámara térmica o un sensor adicional), basta con crear un nuevo proceso sin tocar los que ya están en funcionamiento.

Para organizar el envío al broker, los *topics* siguen una jerarquía con la estructura `dronsar/{dron_id}/{dominio}`:

- `dronsar`: Es el espacio de nombres común de todo el proyecto.
- `{dron_id}`: Identifica a la aeronave que publica (por ejemplo, `dron-01`). Esto deja el sistema preparado para trabajar con varios drones a la vez, ya que cada uno publica con su propio identificador y el servidor los distingue sin necesidad de reconfigurar nada.
- `{dominio}`: Identifica el tipo de dato (ambiental, sistema, vuelo o deteccion). Así, el proceso ambiental del primer dron se publica en `dronsar/dron-01/ambiental`, el de vuelo en `dronsar/dron-01/vuelo`, y así con el resto.

Los cuatro procesos comparten un único archivo de configuración `.env`, donde la variable `DRON_ID` define la identidad de la aeronave. Cada colector utiliza ese valor para generar su propio identificador de cliente MQTT en el formato `{dron_id}-{dominio}` (por ejemplo, `dron-01-ambiental`). Con esto se asegura que cada conexión al broker sea única, algo obligatorio en el protocolo MQTT, ya que, si dos clientes intentasen conectarse con el mismo identificador, el broker los expulsaría en bucle.

La Figura 6.1 resume esta arquitectura: cuatro procesos independientes en el nodo *edge*, cada uno con su buffer SQLite y su *topic*, que publican en la nube a través de los enlaces disponibles.

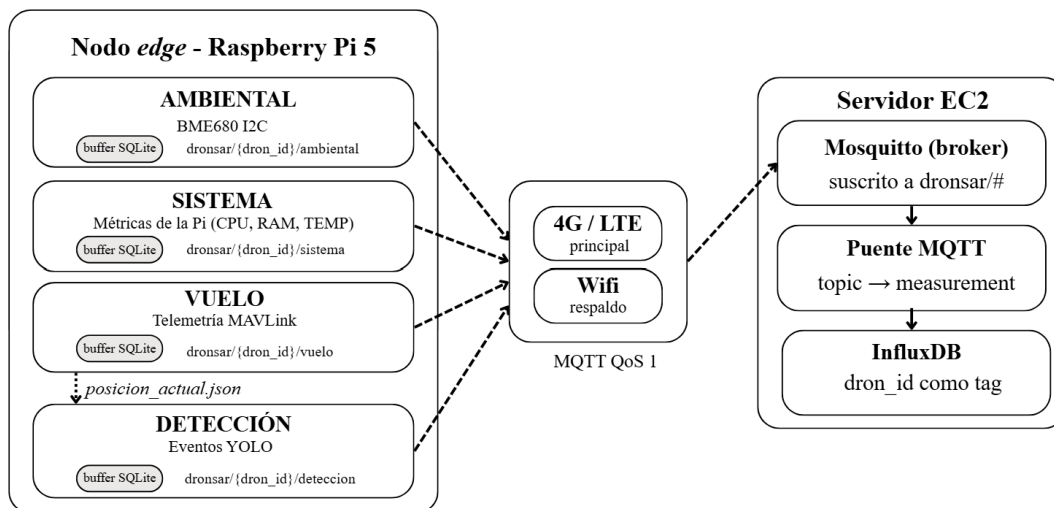


Figura 6.1. Arquitectura de adquisición multidominio en el nodo *edge*.

### 6.3. Los cuatro dominios de datos

Presentado el patrón arquitectónico común, en este apartado se desarrolla el diseño y la implementación de los cuatro dominios de adquisición que integran el nodo *edge*. La Tabla 6.1 resume sus principales características antes de abordar el detalle técnico de cada subsistema.

TABLA 6.1. DOMINIOS DE DATOS DEL SISTEMA

| Dominio   | Variables capturadas                                      | Origen del dato               | Interfaz / Protocolo                      |
|-----------|-----------------------------------------------------------|-------------------------------|-------------------------------------------|
| Ambiental | Temperatura, humedad, presión, gas                        | Sensor BME680                 | Bus I2C (0x76)                            |
| Sistema   | CPU, RAM, temperatura SoC, disco                          | Raspberry Pi 5 (kernel Linux) | API del sistema operativo (psutil)        |
| Vuelo     | Posición GPS, altitud, batería, actitud                   | Pixhawk 6X / SITL             | MAVLink v2.0 (UART/UDP)                   |
| Detección | <i>Bounding boxes</i> , confianza, fotograma testigo, GPS | Inferencia YOLO11 (Hailo-8L)  | Memoria compartida (posicion_actual.json) |

### 6.3.1. Dominio ambiental

El dominio ambiental tiene como objetivo caracterizar el entorno de la misión SAR y detectar condiciones que puedan representar un riesgo, especialmente en escenarios como incendios forestales o en espacios cerrados.

Para ello se ha empleado el BME680, un sensor integrado capaz de medir cuatro magnitudes: temperatura, humedad relativa, presión barométrica y resistencia de gas, que varía con la concentración de compuestos orgánicos volátiles (COV) presentes en el aire y permite, por tanto, estimar variaciones en la calidad del aire.

La elección del BME680 se debe precisamente a que reúne esas cuatro medidas en un único componente. Esto ahorra peso, simplifica el cableado y reduce el consumo, tres factores importantes en una plataforma aérea con capacidad de carga limitada. A esto se suma que su interfaz I2C facilita mucho la integración con la Raspberry Pi, ya que le bastan dos líneas: una de datos y otra de reloj.

La conexión al bus I2C de la Raspberry Pi 5 se ha realizado de la siguiente forma:

- **VCC:** Conectado al riel de 3,3 V de la Raspberry Pi.
- **GND:** Conectado a la masa común del sistema.
- **SDA / SCL:** Conectados a los pines GPIO2 (SDA) y GPIO3 (SCL) de la placa.
- **SDO / CS:** El pin SDO se conecta a masa para fijar la dirección I2C en 0x76, mientras que CS se conecta a 3,3 V para forzar permanentemente el modo I2C y evitar conmutaciones espurias al protocolo SPI.

La Figura 6.2 muestra el conexionado real en la Raspberry Pi 5.

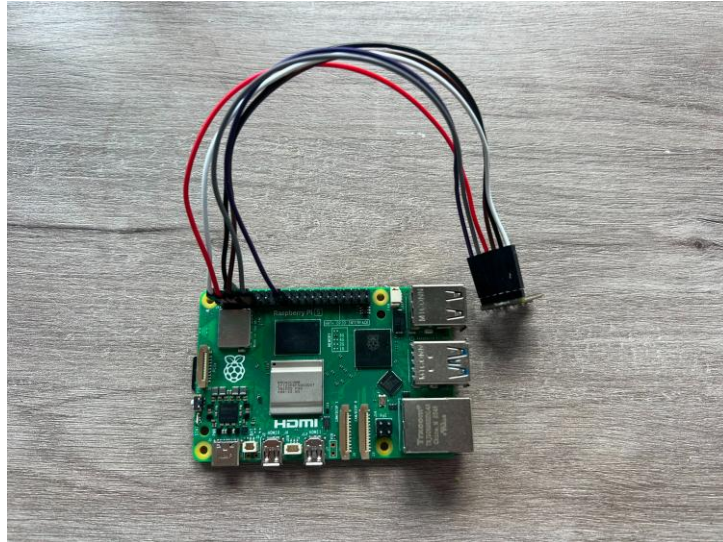


Figura 6.2. Conexión del sensor BME680 a la Raspberry Pi 5.

Una vez habilitada la interfaz I2C en la Raspberry Pi, se comprobó que el sensor respondía a la dirección 0x76. Las lecturas se obtienen periódicamente mediante un script en Python que utiliza la librería de Pimoroni y extrae los datos del bus, entregándolos al proceso de almacenamiento y envío descrito en el apartado 6.4.

### 6.3.2. Dominio de sistema

En un dron autónomo, esta telemetría interna es una medida de seguridad fundamental. Hay que tener en cuenta que la Raspberry Pi soporta una carga de trabajo enorme durante la misión: ejecuta la red neuronal de visión artificial sobre el acelerador Hailo-8L, gestiona la captura periódica de todos los sensores y mantiene el flujo de comunicaciones inalámbricas con el exterior. Sostener toda esa concurrencia en un dispositivo en el aire, sin posibilidad de reiniciarlo manualmente ni de intervenir físicamente, conlleva riesgos térmicos y de saturación de recursos.

Si la temperatura sube de forma sostenida (lo que podría provocar un estrangulamiento térmico o *thermal throttling*) o si hay un consumo de memoria anómalo, estamos ante los primeros síntomas de que algo falla a nivel de software. Disponer de estas métricas en la estación de tierra permite anticiparse al problema: el operador puede detectarlo a tiempo y ordenar un aterrizaje seguro antes de que el computador se bloquee por completo y comprometa la misión. A diferencia de los datos que describen el exterior, esta visión hacia dentro resulta clave para garantizar que todo lo demás siga funcionando.

### 6.3.3. Dominio de vuelo

El dominio de vuelo se encarga de monitorizar en tiempo real el estado de navegación y la cinemática del UAV. A través del protocolo MAVLink v2.0 sobre el enlace serie con la controladora Pixhawk (o mediante conexión UDP en el entorno de simulación SITL), este proceso extrae magnitudes críticas para la supervisión de la misión, tales como las coordenadas GNSS (latitud, longitud y altitud barométrica/elipsoidal), la actitud de la

aeronave (alabeo, cabeceo y rumbo), la velocidad respecto al suelo y el estado de la batería.

A diferencia de la telemetría directa enviada a la estación de control en tierra, este colector desacopla la información de vuelo para integrarla en la nube. Las lecturas se almacenan en su base de datos local SQLite y se publican periódicamente por MQTT en el *topic* `dronsar/{dron_id}/vuelo`, lo que permite el registro histórico del trayecto y la monitorización remota desde cualquier punto con acceso a internet. Asimismo, este proceso es responsable de mantener actualizado el archivo local de memoria compartida para etiquetar geográficamente los eventos de detección.

#### 6.3.4. Dominio de detección

El dominio de detección es la pieza que conecta toda la arquitectura de datos con el objetivo final del proyecto: localizar personas. La fuente de información aquí es el modelo YOLO ejecutándose sobre el acelerador; por tanto, en este apartado me centro exclusivamente en qué hace el proceso con cada detección, no en cómo funciona la red neuronal por dentro.

A diferencia de los demás dominios, que publican sus métricas de forma periódica y constante, el dominio de detección se guía exclusivamente por eventos. Solo emite información cuando ocurre algo: cada vez que el modelo identifica a una persona en el fotograma, el proceso genera una alerta.

Este evento no se limita a avisar de que hay alguien, sino que empaqueta toda la información necesaria para poder actuar sobre el hallazgo:

- **Datos de visión:** La posición exacta de la persona en la imagen (marcada por la caja delimitadora o *bounding box*) y el porcentaje de confianza con el que el modelo la detecta.
- **Coordenadas espaciales:** La posición GPS exacta de la aeronave en el momento del avistamiento, que este proceso lee del fichero compartido descrito en el apartado siguiente. Sin esta fusión de datos, saber que se ha visto a una persona sería inútil en una misión real, ya que los equipos de emergencia no sabrían a qué coordenadas dirigirse.

Además, para garantizar la trazabilidad, cada evento captura el fotograma exacto en el que se detectó. A esta imagen se le superponen visualmente las coordenadas y la marca temporal, y se almacena localmente como evidencia. La referencia a esta imagen se incluye en el propio mensaje de alerta, de modo que el operador en tierra pueda verificar a posteriori la prueba visual que originó el aviso.

Se ha contemplado un problema práctico: si una misma persona permanece a la vista de la cámara durante varios segundos, el modelo la detectará repetidamente en cada fotograma, lo que provocaría una avalancha de eventos idénticos. Para evitarlo, el proceso incorpora un mecanismo de control de saturación (*anti-flooding*) que impone un intervalo mínimo de tiempo entre las alertas. De este modo, un mismo hallazgo se comunica de

forma fiable, pero sin ahogar el canal de comunicación mediante detecciones repetidas del mismo sujeto.

### 6.3.5. Comunicación entre procesos: la posición compartida

El principio de procesos independientes y aislados, descrito en el apartado 6.2, admite una única excepción deliberada. En una misión de búsqueda y rescate, de nada sirve que el algoritmo de visión artificial identifique a una persona si no es capaz de asociar ese hallazgo con coordenadas geográficas exactas en el terreno. El reto arquitectónico radica en que la información de navegación pertenece exclusivamente al dominio de vuelo, no al de detección.

Para resolver este cruce de información sin romper el aislamiento funcional ni introducir un acoplamiento rígido entre procesos, se ha descartado la comunicación directa. En su lugar, se ha implementado un mecanismo de memoria compartida asíncrona basado en un archivo local (`posicion_actual.json`), que actúa como punto de encuentro entre ambos dominios.

La dinámica de actualización opera de la siguiente manera:

- **Escritura continua:** El proceso de dominio de vuelo, mientras gestiona la telemetría, sobrescribe este archivo de forma continua con la última posición conocida de la aeronave.
- **Lectura bajo demanda:** El proceso de detección no interactúa con este archivo de forma continua; únicamente lo abre en modo lectura en el instante exacto en que se registra un evento positivo, extrayendo las coordenadas para etiquetar la alerta.

Gracias a este enfoque, ambos procesos mantienen su independencia operativa. Si por cualquier motivo el dominio de vuelo sufriera una interrupción o un retraso, el dominio de detección continuaría funcionando con total normalidad, limitándose a etiquetar los nuevos avistamientos con la última posición disponible. Este diseño garantiza la contención de fallos: evita que un error puntual en la lectura de la telemetría arrastre o bloquee la ejecución del motor de inteligencia artificial, preservando la robustez global del sistema.

### 6.4. Comunicación y flujo de datos

El nodo *edge* y el servidor nunca se comunican directamente entre sí: cada uno solo se comunica con el broker, que actúa como intermediario. Cada proceso publica sus datos en su *topic* sin saber quién los va a recoger, y el servidor los recibe suscribiéndose a esos *topics*, sin saber de dónde proceden.

Esta separación constituye la principal ventaja de MQTT frente a una arquitectura punto a punto. Al no requerir conocimiento previo de las direcciones IP, cualquiera de los extremos puede cambiar de red o desconectarse temporalmente sin necesidad de reconfigurar el extremo opuesto.

#### 6.4.1. Patrón store-and-forward

En una misión de rescate, el enlace inalámbrico del dron (ya sea por red móvil 4G/LTE o por radio) está expuesto a zonas de sombra y a pérdidas de señal debido a la distancia o al relieve del terreno. Si los datos se enviaran directamente a la nube según se leen, cualquier microcorte provocaría la pérdida de medidas ambientales, de métricas de la Raspberry Pi o de alertas de detección. Para evitarlo, los cuatro procesos de adquisición emplean una arquitectura común de almacenamiento y reenvío (*store-and-forward*).

El funcionamiento sigue una secuencia en cuatro pasos:

- **Guardado local previo:** En cuanto el proceso obtiene una lectura, la guarda en su base de datos local SQLite y le asigna el valor enviado = 0.
- **Desacoplamiento de la red:** El intento de envío por MQTT solo ocurre después de que el dato esté guardado en disco. De esta forma, la captura nunca se frena ni depende del estado de la conexión.
- **Confirmación con QoS 1:** La publicación MQTT se realiza con el nivel de calidad de servicio QoS 1 (*at least once*). El programa queda a la espera del acuse de recibo del broker (PUBACK); solo cuando este llega, actualiza el registro en SQLite a enviado = 1.
- **Reintentos automáticos:** Si el PUBACK no llega debido a un fallo de cobertura, el registro permanece con enviado = 0 y el script vuelve a intentarlo en el siguiente ciclo.

Este diseño protege la información en dos escenarios habituales: cuando el dron atraviesa zonas sin cobertura durante el vuelo y cuando el servidor EC2 está apagado o en mantenimiento. En ambos casos, las lecturas se acumulan en la memoria local y se transmiten en orden al recuperarse el enlace.

La Figura 6.3 resume la lógica de este flujo implementada en el nodo *edge*.



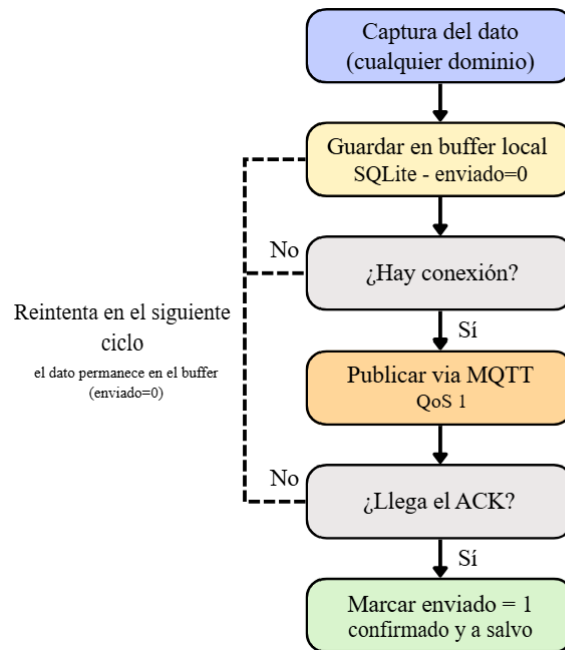


Figura 6.3. Diagrama de flujo del mecanismo de *store-and-forward* implementado en el nodo *edge*.

#### 6.4.2. Store-and-forward propio frente al bridge de Mosquitto

El almacenamiento local en SQLite y el reenvío que usa código Python no son la única opción para manejar las pérdidas de enlace en una red MQTT. El bróker Mosquitto también incluye una función llamada modo bridge, diseñada para conectar dos servidores de mensajería entre sí. Es útil comparar ambas opciones para explicar por qué se optó por un desarrollo a medida en este sistema.

Si se utilizase la opción de *bridge*, la arquitectura operaría de la siguiente manera:

- En el nodo *edge* se instalaría y ejecutaría un bróker local de Mosquitto en la Raspberry Pi 5.
- Los procesos de captura enviarían sus mensajes directamente a ese bróker a bordo. Esto ocurriría sin importar si hay conexión a internet en ese momento.
- El broker local abriría una conexión remota con la instancia EC2 de AWS. Luego, mediante la directiva de enlace, reenviaría automáticamente los temas acumulados cuando detectara cobertura. Toda la lógica de reintentos y de almacenamiento temporal quedaría a cargo directamente del servidor MQTT.

Aunque representa una alternativa directa que reduce líneas de código, se ha optado por implementar un mecanismo *store-and-forward* propio con SQLite debido a dos criterios de diseño centrados en el control del flujo y la escalabilidad del sistema:

- **Simplicidad de administración y despliegue:** Con el modo *bridge*, cada aeronave añadida a la flota requiere su propio servicio de Mosquitto, configurado y supervisado a bordo. Si se suman más drones, también aumenta la cantidad de

brokers que hay que mantener y sincronizar. Con la solución adoptada, la infraestructura de red sigue centralizada. Solo hay un bróker en la nube y cada dron funciona como un cliente normal que publica datos mediante scripts en Python. Esto hace que el mantenimiento y la réplica del sistema sean más sencillos.

- **Visibilidad e inspección del búfer:** Al guardar las lecturas en una base de datos SQLite estándar, es posible consultar en cualquier momento qué mensajes se han enviado y cuáles siguen pendientes. Esto se hace mediante una consulta SQL en el archivo local, revisando el campo enviado. En cambio, con el modo bridge, los mensajes en cola quedan guardados en estructuras internas o en archivos binarios de Mosquitto. No hay una forma directa de revisarlos. Tener una base de datos abierta y fácil de leer simplifica el diagnóstico, la depuración y la validación del funcionamiento de las colas durante las pruebas de campo, especialmente cuando se producen cortes forzados de cobertura.

#### 6.4.3. Recepción en el servidor: puente dronsar/#

En el lado del servidor, el flujo de datos se completa mediante un script en Python que actúa como cliente suscriptor frente al broker Mosquitto desplegado en la máquina EC2. En lugar de abrir una suscripción individual para cada flujo de información, este proceso se suscribe a la raíz comodín dronsar/#.

El uso del comodín # permite abarcar automáticamente todos los subniveles definidos en el espacio de nombres principal. Gracias a ello, un único script centraliza la recepción de cualquier dominio y de cualquier aeronave sin necesidad de crear suscriptores adicionales ni reiniciar los servicios del servidor al incorporar nuevos elementos al sistema.

Esta estructura aporta una gestión escalable y desacoplada:

- **Identificación dinámica por topic:** Al recibir un paquete, el proceso analiza la ruta de publicación (por ejemplo, dronsar/dron-01/ambiental), extrayendo directamente el identificador de la aeronave (dron-01) y el dominio de los datos (ambiental).
- **Persistencia estructurada:** Con estos campos y el contenido del *payload* JSON, el script estructura los datos y los escribe en el *bucket* correspondiente de la base de datos de series temporales de InfluxDB.

En este punto se aplica un criterio de diseño fundamental: se utiliza siempre como marca temporal el instante de captura generado en la Raspberry Pi 5, y no la hora de recepción en el servidor.

Esta decisión resulta indispensable en una arquitectura de tipo *store-and-forward*. Si se emplease la hora de llegada a la nube, todas las lecturas acumuladas durante un corte de cobertura 4G/LTE quedarían registradas en el instante exacto de la reconexión, lo que generaría picos artificiales y falsearía la evolución temporal de las medidas. Al respetar la marca de captura original, las series temporales reflejan con total precisión lo ocurrido durante la misión.

#### 6.4.4. Almacenamiento en InfluxDB

Para el almacenamiento persistente de las medidas se ha optado por InfluxDB, un sistema gestor de bases de datos orientado específicamente a series temporales (*Time Series Database*). Dado que los dominios del UAV generan lecturas continuas asociadas a una marca de tiempo estricta, este motor resulta más eficiente que un modelo relacional convencional: optimiza tanto las escrituras continuas a alta frecuencia como las consultas analíticas por ventanas temporales (medias móviles, agregaciones o históricos de misión). A ello se suma la disponibilidad de una librería oficial de Python y de una interfaz web integrada para la exploración de datos, sin necesidad de recurrir a herramientas externas.

El script puente normaliza cada *payload* MQTT recibido hacia el modelo de datos de InfluxDB siguiendo un criterio homogéneo para los cuatro dominios:

- **Measurement:** El dominio deducido de la ruta del *topic* determina la tabla o *measurement* de destino (ambiental, sistema, vuelo o detección), asegurando la segregación lógica de los flujos.
- **Tag Set:** El identificador del UAV (*dron\_id*) se registra como etiqueta (*tag*). Al ser un campo indexado en memoria, agiliza los filtros y las consultas por aeronave, lo que facilita la supervisión simultánea de flotas de múltiples UAV.
- **Field Set y aplanamiento de datos:** Dado que los mensajes viajan en formato JSON con estructuras anidadas por subsistema, el puente realiza un proceso de aplanamiento (*flattening*). Combina el nombre del bloque y el parámetro bajo el criterio padre\_hijo (por ejemplo, *gps\_latitud* o *bateria\_voltaje*), de modo que cada magnitud física quede almacenada como un campo independiente y directamente consultable dentro de su *measurement*.

En el servidor, los datos se organizan dentro de la organización *sar-drone* y se almacenan en el *bucket* *sensores*.

El acceso a la interfaz web de administración se realiza a través de su subdominio correspondiente, utilizando credenciales de servicio. InfluxDB no expone su puerto de escucha a internet ni abre accesos públicos en el grupo de seguridad de AWS EC2: las peticiones se canalizan a través del servidor web *nginx*, que actúa como proxy inverso y las redirige internamente cifradas con TLS.

La Figura 6.4 muestra la inspección de las medidas registradas en el explorador de datos de InfluxDB (Data Explorer), lo que confirma la recepción y la estructuración del flujo completo desde el nodo *edge* hasta la nube.

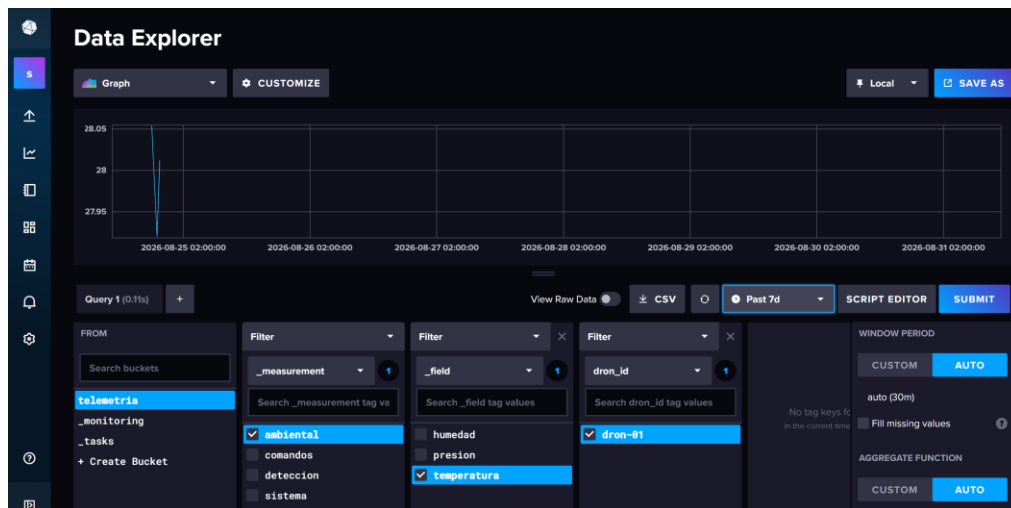


Figura 6.4. Visualización de las medidas de varios dominios en el explorador de datos de InfluxDB.

## 6.5. Ejecución autónoma con systemd

Para garantizar que el sistema opere de forma totalmente desatendida y sin requerir intervención manual tras el encendido, tanto el nodo *edge* como el servidor en la nube delegan la gestión de sus procesos en systemd, el sistema de inicialización y control de servicios estándar en Linux.

En la Raspberry Pi 5 se ejecutan de forma independiente cinco servicios del sistema:

- **Servicios de adquisición (4 servicios):** Corresponden a los cuatro dominios definidos en la arquitectura (ambiental, sistema, vuelo y detección). En cuanto el computador de a bordo recibe alimentación de la batería, cada script arranca por separado e inicia la captura y publicación de métricas al bróker, sin necesidad de abrir una sesión remota por SSH ni ejecutar comandos manualmente.
- **Servicio receptor de comandos (1 servicio):** Ejecuta el script receptor, que permanece a la escucha de las órdenes de vuelo enviadas a la aeronave desde la estación remota y las traduce en mensajes MAVLink para el autopiloto.

Esta estructura modular refuerza directamente el principio de aislamiento por dominios: si un sensor sufre un fallo puntual o un proceso de captura se detiene de forma anómala, systemd lo relanza automáticamente sin interferir en la ejecución de los demás servicios.

En la instancia AWS EC2 concurren simultáneamente cinco servicios:

- **Servicios de infraestructura base:** El bróker Mosquitto y la base de datos InfluxDB, habilitados para arrancar como servicios nativos del sistema operativo al inicio de la máquina virtual.
- **Servicios de aplicación específicos:** El script puente en Python (encargado de suscribirse a dronsar/# y de persistir las medidas en InfluxDB), la API REST para el envío de órdenes y el servidor de medios encargado de la redistribución del flujo de vídeo en directo.

La correcta ejecución del script puente resulta indispensable para la autonomía global: sin este proceso activo, los paquetes MQTT recibidos por Mosquitto no se escribirían en la base de datos, lo que interrumpiría el flujo de información en su último tramo.

Todos los servicios personalizados comparten un patrón de configuración homogéneo: se invocan mediante el intérprete de Python en su entorno virtual correspondiente (venv) y configuran directivas de reinicio automático ante cualquier terminación inesperada. Esta arquitectura garantiza que tanto el flujo de telemetría como la cadena de mando y control se restablezcan de inmediato tras un corte de alimentación o un reinicio no programado, lo cual es un requisito crítico para misiones reales de búsqueda y rescate en entornos de emergencia.

## 7. SERVICIOS WEB Y CONTROL REMOTO

### 7.1. Dominio y gestión DNS con Cloudflare

En las etapas iniciales del despliegue, el acceso a la máquina EC2 se realizaba directamente a través de su dirección IP pública. Este enfoque planteaba dos limitaciones importantes: por un lado, obligaba a memorizar direcciones numéricas y a discriminar cada aplicación únicamente por su número de puerto; por otro, las instancias de AWS EC2 asignan por defecto una IP pública dinámica que cambia tras cada reinicio, lo que rompía de inmediato la conectividad de cualquier cliente configurado con la dirección anterior. Para dotar de persistencia al servidor, se resolvió este inconveniente mediante la asignación de una dirección IP elástica (Elastic IP), fijando de forma permanente la dirección pública de la infraestructura.

Una vez asegurada la IP estática, se registró el dominio `gorostiditfg.com` y se delegó su gestión y la resolución DNS en Cloudflare. Contar con un nombre de dominio propio no solo facilita el acceso mediante identificadores legibles, sino que introduce una capa de abstracción fundamental en la ingeniería de redes: los clientes y la estación de control apuntan a nombres de dominio y no a direcciones IP físicas. Si en el futuro fuera necesario migrar los servicios a otra instancia o proveedor de cloud, bastaría con actualizar los registros DNS sin tener que reconfigurar el software a bordo ni las estaciones de tierra.

A partir de este dominio principal, se estructuró una jerarquía de subdominios para enrutar de forma independiente cada componente de la plataforma. La Tabla 7.1 describe los subdominios configurados y el servicio que cada uno de ellos atiende.

TABLA 7.1. SUBDOMINIOS DEFINIDOS Y SERVICIOS ASOCIADOS.

| Nombre                     | Servicio asociado                                                                                                      |
|----------------------------|------------------------------------------------------------------------------------------------------------------------|
| gorostiditfg.com, www, web | Sitio web del proyecto: página principal, diario de seguimiento, identificación del operador y planificación de tareas |
| control                    | Panel de control y envío de comandos al UAV                                                                            |
| api                        | API REST de mando y control                                                                                            |
| InfluxDB                   | Interfaz de consulta de la base de datos de series temporales                                                          |
| Mosquitto                  | Broker MQTT                                                                                                            |
| grafana                    | Cuadros de mando para la visualización de la telemetría sobre InfluxDB                                                 |
| stream                     | Transmisión de vídeo en directo del dron mediante WebRTC                                                               |
| drone-sar                  | Página pública de difusión del proyecto y captación de colaboraciones (vídeos aéreos y patrocinadores)                 |
| guardian-eye               | Sitio de documentación técnica del proyecto                                                                            |

### 7.2. Servidor web nginx y virtual hosts

Todos los servicios web de la plataforma se alojan en la misma instancia de EC2 de AWS y comparten una única dirección IP pública. Dado que los subdominios definidos en el apartado anterior apuntan a la misma IP, el servidor web nginx centraliza el tráfico entrante e implementa alojamiento virtual (*virtual hosting*) basado en nombres. Al recibir una petición HTTP/HTTPS, nginx inspecciona la cabecera Host y la compara con sus bloques de configuración mediante la directiva `server_name`, entregando el contenido web correspondiente o reenviando la solicitud al servicio interno adecuado. De este modo,

el usuario accede a cada entorno introduciendo su subdominio específico (como `control.gorostiditfg.com` o `influxdb.gorostiditfg.com`), sin necesidad de memorizar direcciones IP ni especificar puertos en la barra de direcciones del navegador.

En función de la naturaleza de cada servicio, los bloques de configuración de nginx operan bajo dos modalidades:

- **Servicio de contenido estático (directiva `root`):** nginx actúa como servidor de archivos convencional, leyendo archivos HTML, hojas de estilo y scripts directamente del almacenamiento local para servirlos al navegador. Este esquema se aplica tanto al portal principal del proyecto (`/var/www/html`) como a la interfaz de usuario del panel de teleoperación (`/var/www/control`).
- **Servicio de aplicaciones dinámicas (directiva `proxy_pass`):** Para los servicios que procesan información en tiempo real, como la API REST en Flask o la interfaz web de InfluxDB, no hay ficheros estáticos en disco. Nginx actúa como proxy inverso: recibe la petición externa y la redirige internamente al puerto local en el que escucha el proceso (`localhost`). Asimismo, nginx inyecta cabeceras HTTP normalizadas (`Host`, `X-Real-IP` y `X-Forwarded-For`) para que las aplicaciones internas conozcan la dirección IP de origen y el subdominio solicitado.

Esta arquitectura aporta una ventaja directa en la seguridad del sistema. Los procesos dinámicos escuchan exclusivamente en la interfaz local de la máquina y no están expuestos al exterior. Los únicos puntos de entrada públicos a la infraestructura son los puertos 80 (HTTP) y 443 (HTTPS) gestionados por nginx. Esto permite mantener cerrados en el grupo de seguridad de AWS los puertos nativos de la API y de la base de datos, reduciendo la superficie de ataque sin comprometer la accesibilidad remota a los servicios.

### 7.3. Seguridad del canal: cifrado TLS

En las primeras fases de desarrollo, los servicios web se desplegaron sobre el protocolo HTTP sin cifrado. Esta configuración resulta inviable en un sistema de teleoperación aérea: si el tráfico circula en texto plano, un atacante intermedio (*man-in-the-middle*) podría interceptar las coordenadas de la misión o, de forma crítica, inyectar y alterar comandos de vuelo en tránsito. La incorporación del protocolo TLS (*Transport Layer Security*) resuelve ambas vulnerabilidades al establecer un canal cifrado, íntegro y autenticado entre el navegador del operador y la instancia en la nube.

La gestión de los certificados digitales se ha realizado mediante Certbot, el agente cliente de la autoridad de certificación Let's Encrypt. El proceso de validación de expedición valida automáticamente el control sobre el dominio mediante un desafío HTTP: la autoridad solicita al servidor un recurso criptográfico específico y, tras verificar la respuesta, emite el certificado digital correspondiente. De forma paralela, Certbot actualiza los bloques de configuración de nginx, habilitando la escucha segura en el puerto 443 (HTTPS) y forzando la redirección automática de cualquier petición entrante al puerto 80 (HTTP).

Los certificados emitidos poseen un periodo de validez de noventa días, una directiva de seguridad que acota la ventana de riesgo ante una hipotética filtración de la clave privada. Para asegurar la continuidad del servicio sin expiraciones imprevistas, se ha configurado una tarea periódica de renovación automática gestionada por `systemd`, cuya operatividad se ha validado en fase de pruebas mediante una simulación de renovación (`certbot renew --dry-run`).

Cabe subrayar que la arquitectura implementa un esquema de terminación de TLS (*TLS Termination*) en el servidor web `nginx`. `Nginx` asume la carga criptográfica del canal exterior y redirige las peticiones descifradas a la API REST en `Flask` y a la interfaz de `InfluxDB` a través de la interfaz local (`localhost`). Al quedar este tráfico confinado dentro del propio núcleo del servidor, no existe exposición a la red pública, lo que simplifica la administración al centralizar la custodia y la renovación de certificados en un único punto de la infraestructura.

#### 7.4. API REST y panel de control

El envío de órdenes a la aeronave se realiza mediante una interfaz web accesible desde cualquier navegador estándar, lo que evita depender de una estación de control terrestre (GCS) instalada en el puesto de operación y prescinde de que el usuario conozca los detalles del protocolo MAVLink. Para facilitar su uso en campo en teléfonos o tabletas, el panel se ha estructurado en formato PWA (*Progressive Web App*). De este modo, es posible instalarlo como acceso directo en el dispositivo y ejecutarlo a pantalla completa, combinando la inmediatez de una página web con la experiencia y la comodidad de una aplicación nativa.

Como soporte de esta interfaz, se ha implementado una API REST en `Python` con el framework `Flask`, seleccionado por su ligereza y bajo consumo de recursos. Para proteger el acceso, `Flask` se ejecuta internamente en la máquina `EC2` y escucha únicamente en la interfaz local (`localhost`). Es el servidor web `nginx` el que atiende las solicitudes externas como proxy inverso, cifradas con TLS (HTTPS), y las redirige al proceso interno de la API.

Por su parte, tanto la API en el servidor como el proceso receptor de órdenes a bordo del UAV se ejecutan como servicios automáticos mediante `systemd`. Esta configuración garantiza que ambos programas arranquen automáticamente tras el encendido y se restablezcan ante caídas imprevistas, evitando que el cierre accidental de una sesión remota deje a la aeronave sin capacidad para recibir órdenes.

El núcleo de la API consiste en actuar como una pasarela que traduce peticiones HTTP en publicaciones MQTT:

- **Comandos de vuelo:** Cuando se acciona un botón en el panel (Figura 7.1), el navegador envía una solicitud HTTP al *endpoint* de control de la API. El script valida los parámetros y publica el comando en el *topic* `dronsar/{dron_id}/comandos`. A bordo del dron, el script receptor suscrito a este *topic* recibe el mensaje, lo procesa y lo traduce a la instrucción MAVLink correspondiente mediante la librería `pymavlink`, y la envía al autopiloto. La API



incorpora, además, un recurso de comprobación de estado (*health check*) que permite verificar la disponibilidad de la pasarela sin emitir órdenes de vuelo.

- **Plano de configuración dinámica:** De manera paralela al control de vuelo, la API permite ajustar en caliente los colectores de adquisición de datos desarrollados en el Capítulo 6. A través de *topics* dedicados (`dronsar/{dron_id}/{dominio}/config`), el operador puede modificar variables de funcionamiento, como el intervalo de muestreo de un colector, sin necesidad de reiniciar el servicio ni de acceder por terminal SSH a la Raspberry Pi.

La separación de estos dos planos asegura que las instrucciones de navegación se canalicen de forma prioritaria hacia la controladora, mientras que los ajustes de muestreo afecten exclusivamente a los procesos de captura de métricas, impidiendo que un flujo interfiera con el otro.

La protección del canal de comandos se fundamenta en la arquitectura de red:

- **Canal cifrado en la interfaz de usuario:** La comunicación entre el navegador y la API REST viaja íntegramente bajo HTTPS/TLS sobre el puerto 443, gestionado por nginx, lo que impide la lectura o manipulación de las órdenes en tránsito.
- **Aislamiento en red privada:** La mensajería MQTT entre la nube y el UAV no se expone a internet abierto, sino que circula dentro de la red privada virtual (Tailscale), restringiendo el intercambio de mensajes exclusivamente a los nodos autenticados en la malla.

Este esquema aísla el canal de mando de los accesos no autorizados, quedando identificada la implementación de esquemas complementarios de autenticación de usuarios (como tokens JWT en la API) como una línea de evolución en el Capítulo de Trabajo Futuro.

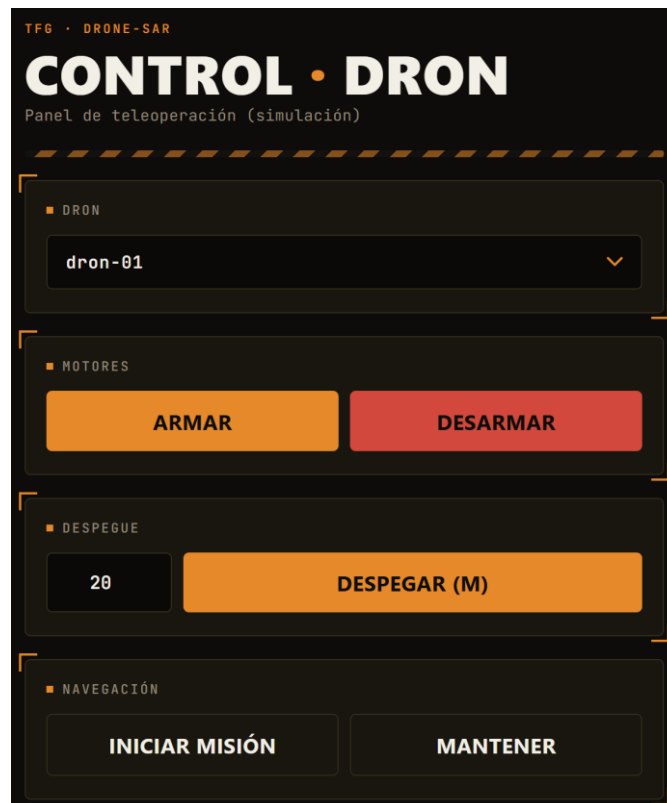


Figura 7.1. Interfaz del panel web de control remoto para el envío de comandos de vuelo

## 7.5. La cadena de mando completa

Una vez caracterizados cada uno de los elementos de la arquitectura, conviene describir el recorrido completo que sigue una orden de control desde que la emite el operador hasta su ejecución física por el autopiloto. Esta cadena atraviesa los tres dominios físicos que componen el sistema: el puesto del operador (cliente), la infraestructura en la nube (servidor EC2) y el vehículo aéreo (nodo *edge*).

Cuando el operador interactúa con un control en el panel web, el flujo de señales se desarrolla a través de la siguiente secuencia:

1. **Petición web segura (HTTPS):** El navegador web envía una solicitud cifrada mediante TLS al subdominio correspondiente (`api.gorostiditfg.com`) en el puerto 443.
2. **Terminación de TLS y proxy inverso (nginx):** El servidor web nginx recibe la petición externa, valida el certificado digital, descifra la información y la redirige internamente a la interfaz local (`localhost`), donde escucha la API REST.
3. **Procesamiento e ingesta en la API (Flask):** El servicio desarrollado en Flask valida la sintaxis y los parámetros del comando recibido y genera una publicación MQTT dirigida al canal de órdenes (`dronsar/{dron_id}/comandos`).
4. **Distribución por mensajería (Mosquitto):** El bróker MQTT entrega el mensaje de forma inmediata a los suscriptores activos a través del canal virtual privado.

5. **Recepción y traducción a bordo (Raspberry Pi 5):** En el nodo *edge*, el script receptor en segundo plano (suscrito al *topic*) procesa el *payload* JSON y lo traduce a la instrucción MAVLink v2.0 correspondiente mediante la librería pymavlink.
6. **Ejecución en el autopiloto (Pixhawk):** La controladora de vuelo recibe el mensaje por el puerto serie UART, ejecuta la maniobra solicitada (como armar motores, iniciar el despegue o conmutar al modo RTL) y ajusta los actuadores.

En cambio, el sistema no ofrece una respuesta directa ni síncrona. La confirmación de que la maniobra fue exitosa se verifica de forma asíncrona mediante el flujo de telemetría que el dominio de vuelo envía de regreso a la nube, lo que cierra el ciclo de supervisión. La Figura 7.2 muestra la arquitectura de esta cadena de mando de extremo a extremo.

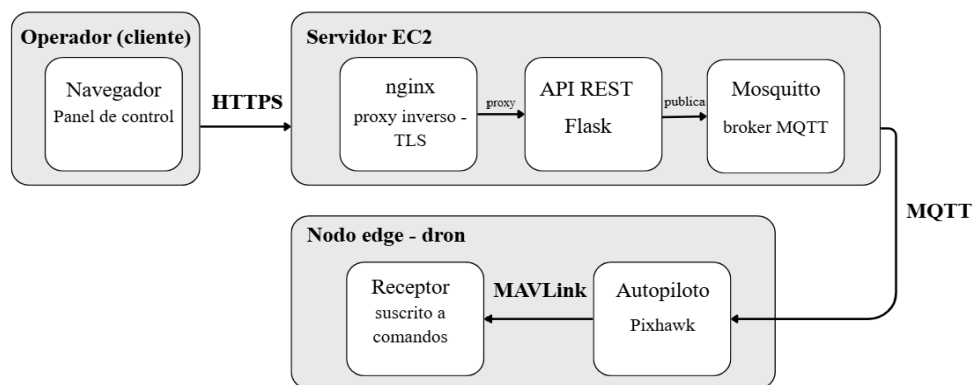


Figura 7.2. Cadena de mando completa, desde el panel del operador hasta el autopiloto.

El valor de este diseño radica en el desacoplamiento estricto entre capas. Ningún eslabón necesita conocer la lógica interna de los componentes no adyacentes: el navegador desconoce la sintaxis del protocolo MAVLink, la API no gestiona el medio físico de enlace celular del dron, y el script receptor procesa órdenes sin requerir información sobre la ubicación de red del operador. Cada tramo dialoga exclusivamente con el siguiente mediante un protocolo adaptado a su contexto (HTTPS en la capa de presentación, MQTT en la capa de transporte y MAVLink en la capa de control de vuelo). Esta independencia permite que el nodo *edge* cambie de red celular o sufra cambios en su dirección IP durante el vuelo sin interrumpir la operatividad del canal de control.

Asimismo, esta arquitectura conforma un sistema de mando dual coordinado con el control en campo descrito en el Capítulo 5:

- **Canal primario de campo (Línea de vista - VLOS):** El piloto dispone de radiocontrol directo (2,4 GHz) y telemetría punto a punto (433/915 MHz) hacia la estación de control en tierra (GCS), lo que garantiza la seguridad en las maniobras críticas de despegue y aterrizaje, sin depender de redes públicas ni de internet.
- **Canal extendido en la nube (Más allá de la línea de vista - BVLOS):** El operador remoto supervisa e interviene en la misión desde cualquier ubicación a

través del panel web y de la infraestructura celular cifrada, sentando las bases tecnológicas para la gestión avanzada de misiones de búsqueda y rescate a larga distancia.

## 7.6. Acceso remoto y red privada (VPN)

En los apartados anteriores se ha explicado cómo el usuario interactúa con la plataforma a través de la web, del panel de control, de la API REST y de la consola de InfluxDB. Sin embargo, el sistema también necesita una vía de comunicación directa que permita acceder a la propia Raspberry Pi 5 para realizar tareas de mantenimiento, depuración del software o comprobación de fallos en pleno vuelo, sin depender de la interfaz gráfica.

Para resolver el acceso a la conexión móvil 4G/LTE, se ha creado una red privada virtual (VPN) que conecta de forma segura a los tres integrantes del sistema: el dron como nodo *edge*, el ordenador del operador en tierra y el servidor desplegado en AWS EC2. Gracias a esta red, los tres equipos se comunican entre sí exactamente igual que si estuviesen enchufados al mismo router en una red local de casa, sin que importe qué dirección IP pública les haya asignado la compañía telefónica en cada momento.

La solución elegida para montar esta red ha sido Tailscale, una herramienta basada en el protocolo WireGuard que crea una red en malla entre todos los dispositivos. Su principal ventaja es que resuelve automáticamente los problemas habituales de las conexiones móviles en las que las operadoras no proporcionan una IP pública accesible. Con este esquema no hace falta contratar servidores intermedios complejos ni abrir puertos en el dron: cada nodo se conecta a la red con su propia identidad digital y el tráfico viaja siempre cifrado de punto a punto.

La Figura 7.3 muestra el esquema de esta red privada y cómo se distribuyen las conexiones seguras entre los tres extremos.

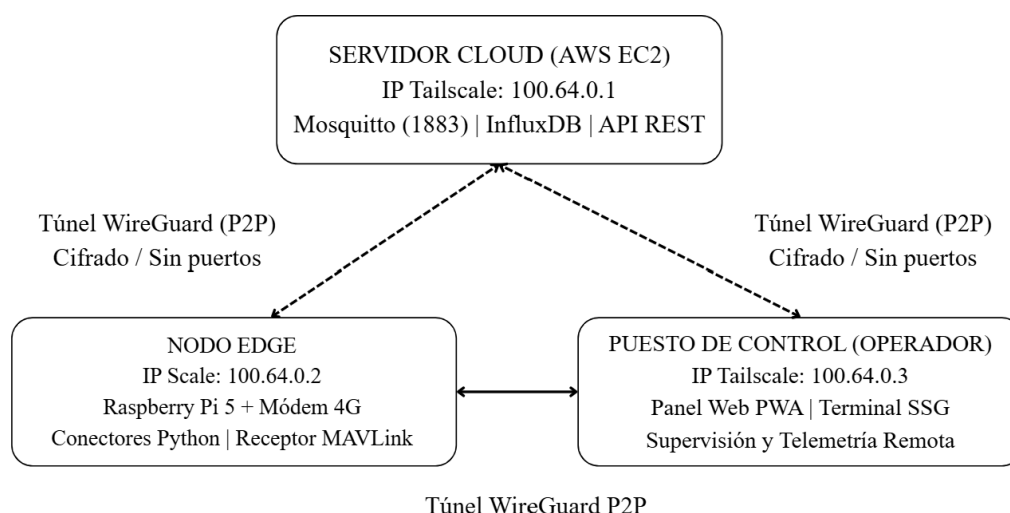


Figura 7.3. Topología de la red privada virtual mallada (Tailscale).

En el día a día del proyecto, la gran ventaja práctica de esta red es que permite abrir un terminal seguro por SSH a la Raspberry Pi 5 desde cualquier lugar con cobertura móvil. Esto permite revisar los registros del sistema, comprobar el consumo de CPU o actualizar los scripts de Python directamente sobre el terreno, sin tener que conectar ningún cable a la aeronave ni exponer el dron a los riesgos de dejar puertos abiertos en internet.

Además, contar con esta red privada deja el proyecto preparado para dos mejoras importantes:

- **Operación con varios drones a la vez:** Si en el futuro se despliega una flota de aeronaves cooperativas, añadir un dron más a la misión es tan sencillo como unirlo a la misma red de Tailscale y compartir el canal seguro, sin tener que tocar la configuración del servidor.
- **Conexión directa con servicios de AWS:** Permite que el propio dron envíe información confidencial directamente a la nube, por ejemplo, para guardar los vídeos y fotografías de una víctima en un almacenamiento seguro de S3, transmitiendo todo a través de un canal cifrado y privado.

### 7.7. Transmisión de vídeo en directo

En los apartados anteriores se ha detallado el mecanismo mediante el cual el operador envía órdenes a la aeronave desde el panel de control web. Sin embargo, gobernar un dron apoyándose únicamente en la telemetría numérica equivale a volar a ciegas. En un escenario de búsqueda y rescate, el retorno visual resulta indispensable: el operador necesita observar el terreno en tiempo real tanto para mantener el control sobre la aeronave más allá del alcance visual (BVLOS) como para contrastar directamente las zonas donde el sistema de visión artificial detecta indicios de personas. Por este motivo, la plataforma añade un canal de vídeo en directo que conecta el vehículo con el panel de teleoperación.

El flujo de vídeo se origina en el propio computador a bordo. En lugar de abrir una segunda captura del sensor de la cámara (lo que saturaría el bus y duplicaría el trabajo del procesador), el sistema reutiliza el fotograma que el modelo YOLO11 ya ha procesado y anotado con las cajas delimitadoras de detección. De esta forma, el operador visualiza exactamente la misma imagen que el detector analiza en el borde.

A partir de ese fotograma procesado, la Raspberry Pi 5 realiza dos tareas en paralelo:

- **Grabación local de alta fidelidad:** Almacena el vídeo en la memoria interna a su resolución nativa y sin pérdida de calidad, preservando el registro íntegro de la misión como evidencia técnica y forense.
- **Retransmisión ligera a la nube:** Codifica una copia reducida y la publica mediante el protocolo RTSP al servidor de AWS. Esta emisión en directo se ajusta deliberadamente a una resolución de  $640 \times 360$  píxeles, 12 fotogramas por segundo y una tasa binaria aproximada de 400 kbps, garantizando que el flujo se transmita con holgura a través del enlace celular 4G/LTE.

Se logra así una separación funcional clara: la grabación local preserva la máxima calidad para el análisis posterior, mientras que la emisión remota prioriza la fluidez y el bajo consumo de ancho de banda para la supervisión en tiempo real.

No obstante, los navegadores web no reproducen flujos RTSP directamente, ni resulta conveniente que los clientes remotos se conecten directamente a la aeronave, cuyo enlace ascendente móvil es limitado. Además, una conexión directa obligaría al computador de a bordo a codificar tantas copias de vídeo como usuarios estuvieran viendo, saturando la CPU y comprometiendo el proceso crítico de detección de víctimas. Para evitarlo, en la máquina virtual AWS EC2 se ha desplegado un servidor de medios (MediaMTX) que actúa como intermediario: el dron emite un único flujo hacia él a través de la red privada virtual cifrada (Tailscale), y es el servidor en la nube quien lo redistribuye a los clientes web que lo soliciten.

Para la entrega del vídeo al operador se evaluó inicialmente el protocolo HLS, pero se descartó al introducir retardos de varios segundos, inaceptables para la teleoperación de una aeronave. En su lugar, se ha adoptado WebRTC, que ofrece una latencia inferior a 1 segundo. El flujo se sirve cifrado mediante TLS y se integra en el panel de control web mediante un marco embebido (Figura 7.4), lo que permite al operador visualizar el vídeo y la telemetría en la misma pantalla desde la que emite los comandos de vuelo.

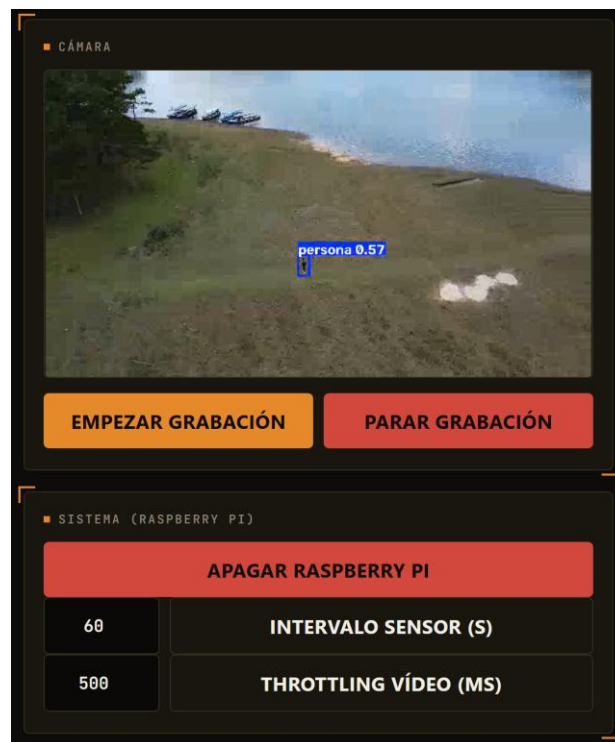


Figura 7.4. Interfaz web de supervisión visual remota

Por último, la transmisión de vídeo se ha diseñado con una política de degradación no bloqueante. Si la cobertura móvil se debilita, el enlace se interrumpe o el servidor de medios cae, el proceso de detección en tiempo real y la grabación local continúan

funcionando con absoluta normalidad. La retransmisión se concibe como un servicio auxiliar que, en ningún caso, interrumpe la función prioritaria de la aeronave: detectar personas y registrar la misión.

La Figura 7.5 sintetiza el recorrido completo del flujo visual, desde la captura física en la cámara hasta su visualización interactiva en el panel del operador.

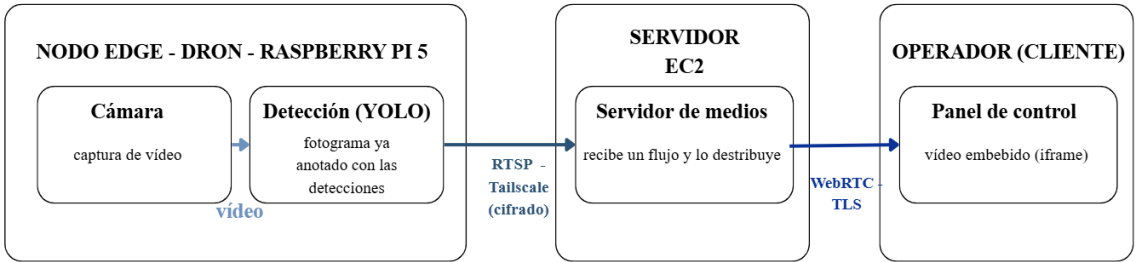


Figura 7.5. Flujo de vídeo en directo desde la cámara del dron al panel del operador.

## 8. DETECCIÓN DE PERSONAS: MACHINE LEARNING CON YOLO11

### 8.1. Introducción a la detección de objetos con YOLO

En una misión de búsqueda y rescate, la función central del sistema es localizar de forma autónoma a las personas. Esta tarea se enmarca en la detección de objetos, un problema de visión por computador que va más allá de la simple clasificación: no basta con determinar si en una imagen aparece una persona, sino que hay que situarla dentro del fotograma mediante un recuadro delimitador (*bounding box*) y asignarle un grado de confianza. Esta diferencia es esencial para el sistema. La posición de detección en la imagen, combinada con la telemetría de vuelo, permite estimar la ubicación geográfica de la persona detectada.

Entre las distintas familias de detectores disponibles, en este proyecto se ha optado por YOLO (*You Only Look Once*), propuesta por Redmon et al. en 2016 [32]. La diferencia principal frente a los métodos clásicos en dos etapas (que primero proponen regiones candidatas y después las clasifican) es que YOLO plantea la detección como un único problema de regresión, resuelto mediante un único procesamiento de la imagen completa. Esta arquitectura de procesamiento único reduce notablemente el coste computacional por inferencia, lo que ha convertido a YOLO en una de las opciones de referencia para aplicaciones que exigen un procesamiento de baja latencia.

Ese bajo coste computacional es, de hecho, la razón principal de su elección en este trabajo. La detección no se ejecuta en un servidor, sino a bordo del propio dron, en hardware embebido de recursos limitados, y debe procesar el vídeo con suficiente rapidez como para resultar útil durante el vuelo. Un detector en dos etapas, más preciso pero considerablemente más pesado, sería inviable en este escenario; la arquitectura de pasada única de YOLO, en cambio, sí permite mantener una detección continua sobre el flujo de imágenes, empleando el acelerador Hailo-8L, cuyo funcionamiento y proceso de integración se detallan en la sección 8.6.

Desde su versión original, YOLO ha evolucionado a lo largo de iteraciones sucesivas que han mejorado tanto su precisión como su eficiencia. La versión empleada en este trabajo pertenece a la línea mantenida por Ultralytics [33], que, junto con el modelo, aporta un ecosistema de entrenamiento y exportación que simplifica de forma notable el flujo de trabajo descrito en las secciones siguientes. Por último, conviene señalar que la calidad de un detector de este tipo no se valora cualitativamente, sino mediante métricas objetivas: *precision*, *recall* y *mean Average Precision* (mAP), cuyo significado y cuyos valores obtenidos se desarrollan en la sección 8.5.

### 8.2. Metodología – Pipeline

El sistema de detección no consiste en un único programa, sino en una serie de etapas que abarcan desde el procesamiento del *dataset* hasta la ejecución del modelo a bordo del dron. La estructura de este pipeline tiene dos fases claramente diferenciadas, ya que no ocurren ni en el mismo momento ni en el mismo lugar.



La primera es una fase preparatoria que incluye la construcción del conjunto de datos, el entrenamiento del modelo, su evaluación y su compilación al formato que ejecuta el acelerador embarcado; todo se lleva a cabo antes del vuelo, el entrenamiento en la nube y la compilación en el equipo local, y se realiza una única vez por cada versión del modelo. La segunda es la fase operativa, la inferencia, que se ejecuta a bordo del dron y durante el vuelo, aplicando el modelo ya entrenado al vídeo de la cámara en tiempo real.

Esta separación es una decisión de diseño deliberada: el cómputo pesado se concentra en tierra y una sola vez, mientras que durante el vuelo el dron ejecuta el modelo ya optimizado de forma autónoma y sin depender de la conexión a la red, un requisito determinante en misiones de búsqueda y rescate, donde la zona de operación apenas dispone de cobertura fiable.

En la fase preparatoria, el proceso se divide en cuatro etapas sucesivas. En primer lugar, la construcción del *dataset* consiste en recopilar y etiquetar imágenes aéreas de personas que sirvan como ejemplos de aprendizaje. A partir de este conjunto, se aplica *transfer learning* a una red YOLO ligera ya preentrenada para adaptarla a la detección de personas; este entrenamiento se realiza en la nube debido al coste de cómputo que implica, como se detalla en la validación preliminar descrita al final de esta sección. El modelo resultante se evalúa con métricas objetivas sobre imágenes que no fueron vistas durante el entrenamiento, para verificar su validez antes del despliegue. Por último, la exportación y la compilación convierten el modelo entrenado en un formato intermedio estándar (ONNX) y luego en el formato nativo del acelerador Hailo-8L (HEF), el único que su procesador neuronal puede ejecutar de forma acelerada.

En la fase operativa, el modelo compilado se ejecuta a bordo en un bucle continuo sobre el vídeo en tiempo real. En cada iteración se captura el fotograma, se adapta al formato de entrada y el acelerador realiza la inferencia, filtrando las detecciones finales junto con su posición y su nivel de confianza. A continuación, los datos de vuelo permiten convertir la posición del recuadro en la imagen en coordenadas geográficas reales; de este modo, el sistema no muestra una simple caja en la pantalla, sino una alerta con la posición geográfica real para el equipo de rescate. Por último, esta información se publica en un *topic* MQTT propio dentro de la arquitectura IoT del sistema (capítulo 6), integrándose con el resto de los datos de vuelo.

La figura 8.1 resume el pipeline completo, distinguiendo visualmente la fase preparatoria (en la nube y en tierra) de la fase operativa (a bordo). Como resultado de esta estructura, la dependencia de la nube se elimina por completo en vuelo: Roboflow solo se utiliza en la fase de entrenamiento, lo que permite que el dron ejecute la detección a bordo de forma autónoma y sin conexión a la red.

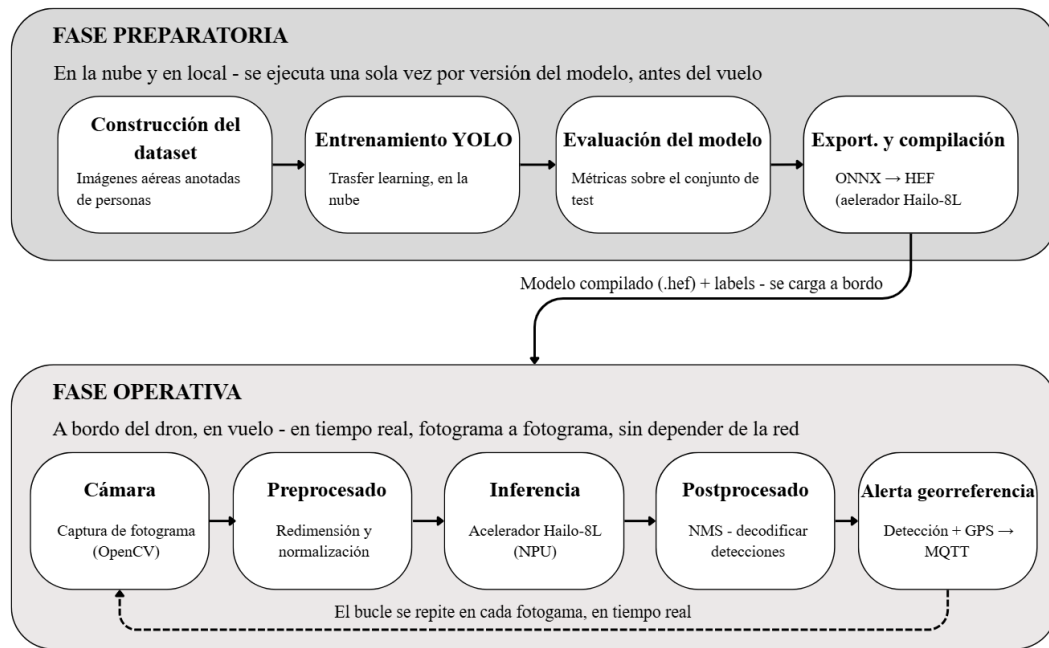


Figura 8.1. Pipeline completo de detección de personas, desde la construcción del *dataset* hasta la inferencia embarcada.

### Validación preliminar del flujo de trabajo

Antes de trabajar con el conjunto de datos real de personas, se realizó una prueba preliminar del flujo de trabajo completo en un caso sencillo. Se usó un detector de doce tipos de fruta. El objetivo no era obtener un modelo muy preciso, sino comprobar todo el pipeline de principio a fin. Esto incluía la creación del conjunto de datos en Roboflow, el entrenamiento mediante *transfer learning* a partir de los pesos de YOLO11-nano y la inferencia sobre vídeo. También se midieron los tiempos de cómputo reales antes de usar el *dataset* final.

De esta prueba se extrajeron dos conclusiones que condicionan el resto del trabajo. La primera es el alto coste de cómputo. Aunque se trata de un modelo ligero y de un conjunto de apenas trescientas imágenes, el entrenamiento local en CPU requirió 2 horas y 40 minutos. Este tiempo no se debe a un exceso de épocas. La parada temprana detuvo el proceso cuando las métricas se estabilizaron. El problema está en el propio hardware. Así, entrenar el modelo real, que es más grande y complejo, no es posible en local. Hay que usar infraestructura en la nube con aceleración por GPU. La segunda es una limitación esperada. Con tan pocos datos, el detector fue poco fiable. Confundió clases poco representadas y generó detecciones falsas en el fondo. Esto evidencia la necesidad de ampliar el conjunto de datos. Es especialmente importante en la búsqueda y el rescate, donde los falsos negativos y falsos positivos afectan la utilidad del sistema. Una vez validado el flujo completo, se comenzó a construir el *dataset* definitivo que se describe a continuación.

### 8.3. Dataset: obtención y construcción

#### 8.3.1. Recopilación de imágenes propias

El punto de partida del sistema de detección fue la construcción de un conjunto de datos propio, adaptado a las condiciones concretas de la aplicación: personas vistas desde un dron en entornos naturales de montaña, bosque y campo abierto, que constituyen los escenarios típicos de una misión de búsqueda y rescate.

Como fuente principal de imágenes, se usaron vídeos aéreos grabados con dron de distintos orígenes. Por un lado, se usaron grabaciones propias. Por otro lado, como apoyo, se añadieron vídeos que algunos colaboradores me enviaron a través de la web del proyecto ([dron-sar.gorostiditfg.com](http://dron-sar.gorostiditfg.com)). Por último, en su mayoría, se eligieron vídeos libres de la plataforma Pexels, un repositorio de material audiovisual con licencia de uso libre. De todo el material disponible, se seleccionaron las grabaciones que mejor se ajustaban al caso de uso. Se dio prioridad a la variedad de escenarios, alturas de vuelo y condiciones de iluminación.

A partir de estos vídeos, la construcción del conjunto de imágenes siguió un proceso completamente manual y supervisado. En primer lugar, cada vídeo se descompuso en fotogramas mediante la herramienta ffmpeg, extrayendo un fotograma cada dos segundos en lugar de todos los disponibles. Esta decisión es deliberada: los fotogramas consecutivos apenas se diferencian entre sí, de modo que extraerlos todos generaría un gran número de imágenes casi idénticas que no aportarían variedad al aprendizaje y multiplicarían innecesariamente el trabajo de etiquetado. Tras la extracción, se realizó una revisión manual para descartar los fotogramas redundantes o de baja calidad, conservando únicamente aquellos que aportaban información distinta, como cambios de escenario, de la posición de las personas en el encuadre o de escala.

Las imágenes seleccionadas se subieron a la plataforma Roboflow, utilizada como herramienta de anotación y gestión del conjunto de datos. Sobre ellas se realizó el etiquetado manual de cada persona visible mediante recuadros delimitadores (*bounding boxes*), bajo una única clase, *persona*, coherente con la definición del problema. El criterio de etiquetado aplicado y la organización posterior del conjunto se detallan en la sección 8.3.3.

#### 8.3.2. Ampliación con fuentes de datos existentes

Los primeros entrenamientos realizados con el propio conjunto permitieron identificar una limitación importante. El material recopilado, en su mayoría procedente de vídeos aéreos, mostraba a las personas, sobre todo, de pie o caminando. Apenas había ejemplos de otras posturas, también importantes en un contexto de búsqueda y rescate, como personas tumbadas o sentadas en el suelo. Estas posturas corresponden al caso crítico de la aplicación: una persona accidentada. Por eso, su escasa presencia en el conjunto de entrenamiento era una carencia que debía corregirse.

Para cubrir este hueco, se recurrió a un conjunto de datos público, el SAR Computer Vision Dataset [34], alojado en Roboflow y distribuido bajo la licencia CC BY 4.0. Se trata de un *dataset* orientado específicamente a tareas de búsqueda y rescate, compuesto

por unas 1980 imágenes aéreas de personas en entornos naturales captadas desde un dron, y de especial interés porque incluye personas en distintas posturas (de pie, caminando, corriendo, sentadas y tumbadas), justo las situaciones poco representadas en el conjunto propio.

La incorporación de este material no se realizó de forma directa, sino tras un proceso de selección y adaptación coherente con el resto del *dataset*. En primer lugar, de las cerca de 1980 imágenes disponibles se seleccionaron únicamente 125, descartando las redundantes o poco representativas y priorizando aquellas que aportaban los casos ausentes en el conjunto propio, en particular personas tumbadas y sentadas. En segundo lugar, el *dataset* original estaba anotado con seis clases distintas según la postura de la persona (*laying\_down*, *not\_defined*, *Running*, *seated*, *stands* y *Walking*), mientras que el sistema desarrollado plantea la detección como un problema de una sola clase. Por ello, todas las anotaciones seleccionadas se reetiquetaron bajo la clase única *persona*, unificando las seis categorías originales en un concepto de interés para la misión: la presencia de una persona, independientemente de su postura.

De este modo, el conjunto público no se empleó como base del sistema, sino como una fuente complementaria destinada a enriquecer el *dataset* propio en aquellos casos en que este resultaba insuficiente. El resultado fue un conjunto de datos combinado que mantiene como núcleo el material propio y lo refuerza con ejemplos externos seleccionados, preservando en todo momento la coherencia con la definición del problema abordado.

### **8.3.3. Etiquetado y particionado del dataset**

La calidad de un detector supervisado depende en gran medida de la calidad y la coherencia de las anotaciones con las que se entrena. Por ello, más allá del volumen de imágenes, se prestó especial atención al criterio de etiquetado y a la organización del conjunto de datos, dos aspectos que resultaron determinantes para el rendimiento final del modelo.

#### **Criterio de etiquetado**

Como se muestra en la Figura 8.2, el etiquetado se realizó manualmente en la plataforma Roboflow, marcando a cada persona con un recuadro delimitador bajo la clase única "persona". Durante este proceso se adoptó un criterio coherente para decidir qué personas debían anotarse: se etiquetaron únicamente aquellas identificables a simple vista en la resolución de trabajo, descartando las que solo resultaban distinguibles al ampliar la imagen. La razón es que el modelo procesa las imágenes a una resolución fija y no permite ampliarlas; una figura que, a esa resolución, resulta indistinguible del fondo no aporta información que pueda aprenderse y, etiquetada como persona, introduce ruido que confunde al modelo, ya que en otras imágenes manchas similares corresponden a rocas o vegetación. Este criterio se aplicó, además, de forma independiente en cada fotograma: una misma persona podía aparecer etiquetada en un fotograma en el que se distinguía con claridad, y no en otro posterior en el que, por la distancia, dejaba de ser identificable. Mantener este criterio de manera coherente en todo el conjunto resultó clave para evitar que el modelo aprendiera ejemplos contradictorios.

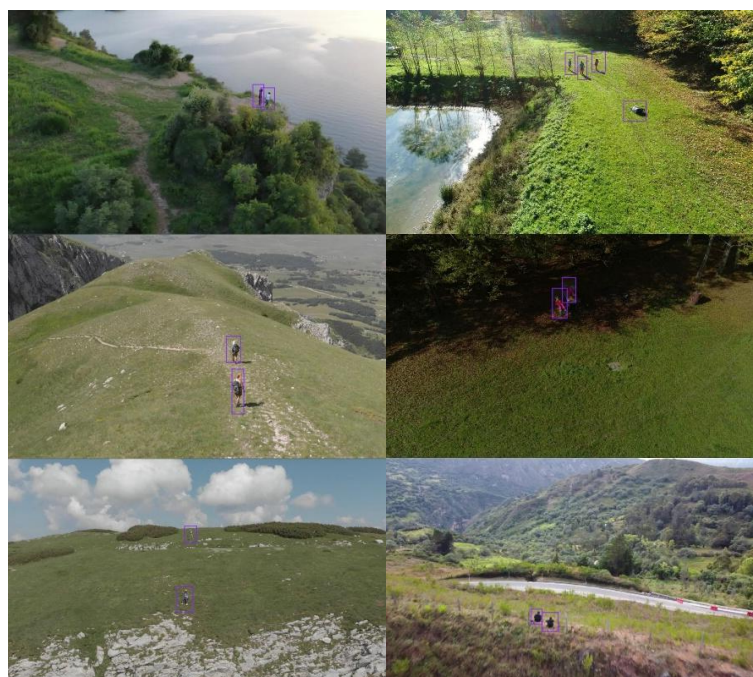


Figura 8.2. Ejemplos de anotación manual en la plataforma Roboflow para el entrenamiento del modelo.

### Formato de las anotaciones

Durante el etiquetado inicial se emplearon indistintamente las herramientas de anotación disponibles en Roboflow, incluido el trazado automático de contornos. Sin embargo, al iniciar los primeros entrenamientos se detectó que una parte de las imágenes era descartada por el framework de entrenamiento, que las señalaba como anotaciones inconsistentes. El análisis del problema reveló su origen: algunas anotaciones se habían guardado como polígonos de segmentación, en lugar de como recuadros de detección, lo que mezclaba ambos formatos en un mismo conjunto. Dado que el sistema plantea un problema de detección —no de segmentación—, todas las anotaciones debían estar en formato de recuadro. El problema se resolvió convirtiendo la totalidad de los polígonos en sus recuadros delimitadores equivalentes, tras lo cual el conjunto quedó anotado de forma homogénea y dejaron de descartarse imágenes. Esta incidencia, si bien menor, pone de manifiesto la importancia de verificar la coherencia del formato de las anotaciones antes de entrenar.

### Particionado del conjunto

La organización del conjunto en subconjuntos de entrenamiento, validación y prueba (*train*, *validation* y *test*) requirió una precaución específica derivada de la naturaleza del material. Al procesar imágenes de vídeo, los fotogramas próximos en el tiempo resultan muy similares entre sí. Si estos fotogramas casi idénticos se distribuyeran aleatoriamente entre los distintos subconjuntos, el modelo podría evaluarse con imágenes prácticamente iguales a las empleadas para entrenar, lo que produciría métricas artificialmente altas que no reflejarían su capacidad real de generalización. Este fenómeno, conocido como fuga

de datos (*data leakage*), es un riesgo habitual en conjuntos de datos contruidos a partir de vídeo.

Para evitarlo, el particionado no se realizó mediante imágenes individuales, sino mediante vídeos completos: todos los fotogramas procedentes de un mismo vídeo se asignaron íntegramente a un único subconjunto, garantizando así que ninguna imagen del conjunto de prueba tuviera una versión casi idéntica en el conjunto de entrenamiento. De este modo, la evaluación se realiza siempre sobre material que el modelo no ha visto durante el entrenamiento, y las métricas obtenidas son representativas de su comportamiento real.

Finalmente, una vez definida esta partición, los subconjuntos de validación y de prueba quedaron definidos de forma definitiva y no se modificaron en las iteraciones posteriores. Esta decisión responde a una necesidad metodológica: para comparar de forma justa distintas versiones del modelo (diferentes configuraciones de entrenamiento o ampliaciones del conjunto de datos), es imprescindible que todas se evalúen sobre el mismo conjunto de prueba. Cualquier modificación del conjunto de evaluación alteraría la referencia y haría que las métricas de distintos experimentos dejaran de ser comparables entre sí. Al mantener fijos los subconjuntos de validación y de prueba, todas las mejoras introducidas a partir de ese punto pudieron medirse sobre una base estable, aislando el efecto real de cada decisión de diseño.

#### 8.3.4. Estadísticas del conjunto de datos

El conjunto de datos de partida, resultado de combinar el material propio con la ampliación procedente del *dataset* público, quedó constituido por un total de 659 imágenes anotadas bajo la clase única “persona”. Estas imágenes se repartieron entre los tres subconjuntos habituales, siguiendo el criterio de particionado por vídeos descrito en la sección anterior, con la distribución que se muestra en la Tabla 8.1.

TABLA 8.1. DISTRIBUCIÓN INICIAL DEL CONJUNTO DE DATOS

| Subconjunto             | Imágenes | Porcentaje |
|-------------------------|----------|------------|
| Entrenamiento (train)   | 482      | 73 %       |
| Validación (validation) | 88       | 13 %       |
| Prueba (test)           | 89       | 14 %       |
| Total                   | 659      | 100 %      |

Tras la ampliación dirigida descrita en la sección 8.5.3, el conjunto alcanzó un total de 717 imágenes (540 de entrenamiento, más los mismos 88 de validación y 89 de prueba, que se mantuvieron fijos).

La proporción resultante, próxima a un reparto 73/13/14, se ajusta a las prácticas habituales en el entrenamiento de modelos de aprendizaje supervisado, reservando la mayor parte de los datos para el entrenamiento y manteniendo subconjuntos de validación y de prueba de tamaño suficiente para obtener métricas fiables.

Cabe subrayar que esta composición constituye el punto de partida del proceso. Como se detalla en la sección 8.5, el conjunto de entrenamiento se amplió posteriormente de forma

dirigida, a partir del análisis de los resultados, mientras que los subconjuntos de validación y prueba se mantuvieron fijos durante todo el proceso. Esta decisión permite comparar de forma justa las distintas versiones del modelo, ya que todas se evalúan con el mismo conjunto de prueba.

Más allá del número de imágenes, una característica relevante del conjunto es su variedad, buscada deliberadamente durante la recopilación. El *dataset* reúne personas vistas desde distintas alturas de vuelo y, por tanto, a escalas muy diferentes, sobre fondos heterogéneos, hierba, tierra, roca, monte y zonas boscosas, bajo distintas condiciones de iluminación y en diferentes posturas, incluyendo las posturas de rescate incorporadas mediante la ampliación externa. Esta diversidad es especialmente importante en un problema de detección desde un dron, ya que permite al modelo generalizarse a las condiciones variables de una misión real.

#### **8.4. Entrenamiento del modelo YOLO11**

El entrenamiento del modelo se apoya en una cadena de herramientas claramente diferenciadas, cada una de las cuales está encargada de una fase concreta del proceso. Esta separación no es accidental, sino que responde a un flujo de trabajo diseñado para aprovechar las ventajas de cada plataforma.

La primera herramienta es Roboflow, empleada exclusivamente para la gestión del conjunto de datos: la subida de imágenes, su etiquetado manual, la organización en subconjuntos de entrenamiento, validación y prueba, y la exportación en el formato requerido por el modelo. Es importante subrayar que en Roboflow no se realiza ningún entrenamiento; su función se limita a preparar y servir el conjunto de datos ya anotado.

El entrenamiento propiamente dicho se llevó a cabo en Google Colab, un entorno de ejecución en la nube que ofrece acceso a una unidad de procesamiento gráfico (GPU). Esta decisión responde a una necesidad de cómputo: el entrenamiento de un detector de objetos es un proceso intensivo que, en una CPU convencional, resultaría prohibitivamente lento, como se comprobó empíricamente durante la validación preliminar del flujo de trabajo. El uso de una GPU en la nube, concretamente una NVIDIA Tesla T4, reduce el tiempo de entrenamiento de horas a minutos y permite iterar con agilidad sobre distintas configuraciones. Conviene remarcar, además, que este entrenamiento se realiza en la nube y no a bordo de la Raspberry Pi: el hardware embarcado no está pensado para entrenar modelos, sino únicamente para ejecutar el modelo ya entrenado durante el vuelo.

El flujo de trabajo completo, de principio a fin, es el siguiente. En primer lugar, el conjunto de datos preparado en Roboflow se exporta e importa directamente en el entorno de Colab mediante la API de la plataforma. A continuación, se utiliza la librería Ultralytics, que implementa la familia de modelos YOLO, para iniciar el entrenamiento a partir de un modelo preentrenado. El punto de partida no es una red inicializada de forma aleatoria, sino los pesos preentrenados de YOLO11 sobre el conjunto COCO, un *dataset* de referencia con millones de imágenes y numerosas categorías, entre ellas la de persona. Esta técnica, conocida como *transfer learning*, permite reaprovechar el

conocimiento general que la red ya posee sobre formas y objetos, reajustándolo al problema concreto de detectar personas desde un dron; gracias a ella, es posible obtener un modelo funcional con un conjunto de datos relativamente reducido, algo que sería inviable entrenar desde cero.

Los parámetros de entrenamiento se ajustaron para lograr un equilibrio entre la calidad visual y los límites de la GPU. Se fijó una resolución de 1024 píxeles para asegurar que las personas situadas a mayor distancia conservaran suficiente detalle como para ser detectadas. Para que la tarjeta gráfica (NVIDIA Tesla T4 en Google Colab) pudiera mover ese volumen de datos sin quedarse sin memoria, las imágenes se procesaron en grupos de 16 (*batch size*). El proceso se extendió a lo largo de 70 épocas, un margen suficiente para alcanzar el punto óptimo de aprendizaje en la época 45 y almacenar automáticamente el modelo definitivo en `best.pt`.

Como resultado del entrenamiento, la librería genera automáticamente los pesos del modelo, conservando dos versiones: el modelo correspondiente a la última época (*last*) y, sobre todo, el que obtuvo el mejor rendimiento en el conjunto de validación a lo largo de todo el proceso (*best*). Es este último, el archivo `best.pt`, el que se toma como modelo entrenado definitivo y que posteriormente se prepara para su despliegue a bordo, tal y como se describe en la sección 8.6.

## 8.5. Evaluación del modelo: métricas de referencia

### 8.5.1. Métricas de evaluación

El rendimiento de un detector de objetos no se evalúa cualitativamente, sino mediante métricas objetivas que cuantifican distintos aspectos de su comportamiento. Antes de presentar los resultados, conviene definir estas métricas y, sobre todo, identificar cuáles son las más relevantes para una aplicación de búsqueda y rescate.

Toda detección del modelo se clasifica, al compararla con la realidad, en una de tres categorías: verdadero positivo (el modelo detecta a una persona que efectivamente está presente), falso positivo (el modelo señala a una persona donde no la hay) y falso negativo (el modelo no detecta a una persona que sí está presente). A partir de estas categorías se construyen las métricas fundamentales.

La precisión (*precision*) mide qué proporción de las detecciones realizadas por el modelo son correctas. Responde a la pregunta: de todo lo que el modelo ha marcado como persona, ¿cuánto lo era realmente? Una alta precisión indica que el modelo se equivoca poco al afirmar haber encontrado a alguien; es decir, genera pocas falsas alarmas.

La exhaustividad (*recall*) mide qué proporción de las personas realmente presentes ha detectado el modelo. Responde a la pregunta: de todas las personas que había, ¿a cuántas ha encontrado? Un *recall* alto indica que al modelo se le escapan pocas personas.

Ambas métricas mantienen una relación de compromiso: un modelo muy conservador, que solo señala detecciones de las que está muy seguro, tiende a una precisión alta pero un *recall* bajo, mientras que un modelo que señala cualquier posible detección logra un



*recall* alto a costa de más falsos positivos. El equilibrio adecuado depende de qué error resulte más costoso en cada aplicación.

Para valorar el rendimiento global se utiliza el *mean Average Precision* (mAP), que combina precisión y *recall* en distintos umbrales. Se reportan dos variantes. El mAP@50 considera correcta una detección cuando su recuadro se solapa con el real en al menos un 50 %, y es la métrica de referencia más habitual. El mAP@50-95 es más exigente, ya que promedia el rendimiento sobre umbrales de solapamiento crecientes (del 50 % al 95 %), penalizando las detecciones cuyo recuadro no se ajusta con precisión al objeto; sus valores son sistemáticamente más bajos, especialmente en objetos pequeños, donde ajustar el recuadro al píxel resulta difícil.

En el contexto de este proyecto, no todas las métricas tienen la misma importancia. En una misión de búsqueda y rescate, el error más grave es no detectar a una persona que necesita ayuda; una falsa alarma, en cambio, es fácilmente descartable por el equipo de rescate y tiene un coste mucho menor. Por ello, el *recall* se considera la métrica prioritaria del sistema: es preferible un modelo que detecte a casi todas las personas, aunque genere algunas falsas alarmas, antes que uno muy preciso que deje a personas sin localizar. Este criterio guía tanto la evaluación de los modelos como las decisiones de diseño de las secciones siguientes.

### **8.5.2. Modelo inicial**

El primer modelo se entrenó sobre el conjunto de datos ya estabilizado, empleando la variante *small* de YOLO11 (yolo11s) a partir de sus pesos preentrenados sobre COCO. Se utilizó una resolución de entrada de 1024 píxeles, elegida para favorecer la detección de personas de pequeño tamaño, sin aplicar aumento de datos. Este modelo constituye el punto de partida desde el cual se valoran las mejoras posteriores.

El entrenamiento del modelo se realizó durante 70 épocas en Google Colab. Como se muestra en la Figura 8.3, los errores del modelo (curvas de pérdida o *loss*) disminuyeron de manera constante a medida que avanzaba el aprendizaje. En las pruebas de validación, estos errores se estabilizaron rápidamente en una meseta sin mostrar signos tempranos de sobreajuste (*overfitting*). Del mismo modo, la precisión y el *recall* fueron mejorando progresivamente hasta alcanzar su punto óptimo alrededor de la época 45. A partir de ese punto, las métricas alcanzaron su máximo y las pérdidas de validación comenzaron a oscilar ligeramente, momento en el que el sistema guardó automáticamente los mejores pesos en el archivo best.pt para la evaluación definitiva.

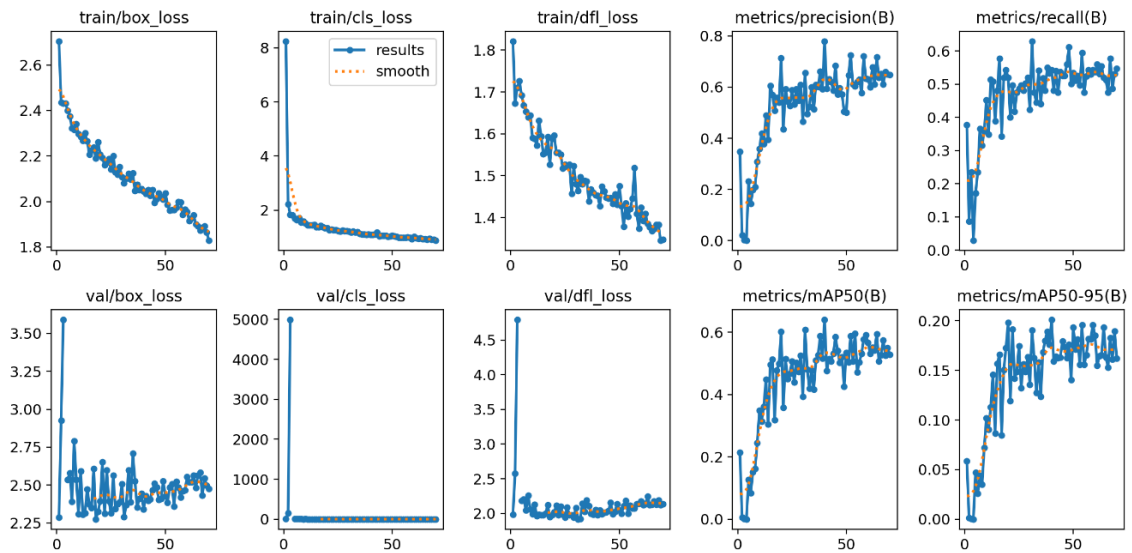


Figura 8.3. Evolución de las funciones de pérdida y de las métricas de evaluación a lo largo de las 70 épocas de entrenamiento.

Su rendimiento se evaluó sobre el conjunto de prueba, formado, como se detalló en la sección 8.3.3, por imágenes que el modelo no había visto durante el entrenamiento, que se mantuvo fijo para comparar de forma justa las distintas versiones. Los resultados obtenidos se recogen en la Tabla 8.2.

TABLA 8.2. MÉTRICAS DEL MODELO INICIAL SOBRE EL CONJUNTO DE PRUEBA

| Métrica   | Modelo inicial |
|-----------|----------------|
| Precisión | 0,693          |
| Recall    | 0,542          |
| mAP@50    | 0,586          |
| mAP@50-95 | 0,184          |

El modelo alcanza una precisión notablemente superior a su *recall*: acierta en la mayoría de las detecciones que realiza, pero localiza a algo más de la mitad de las personas presentes. Dado que el *recall* es la métrica prioritaria en este sistema (no dejar a ninguna persona sin localizar), ese es el aspecto que las mejoras posteriores buscan reforzar.

### Análisis de errores

Las métricas indican cuánto falla el modelo, pero no por qué. Para averiguarlo, se recurrió a la matriz de confusión (Figura 8.4) y a la inspección visual de las detecciones en el conjunto de prueba.

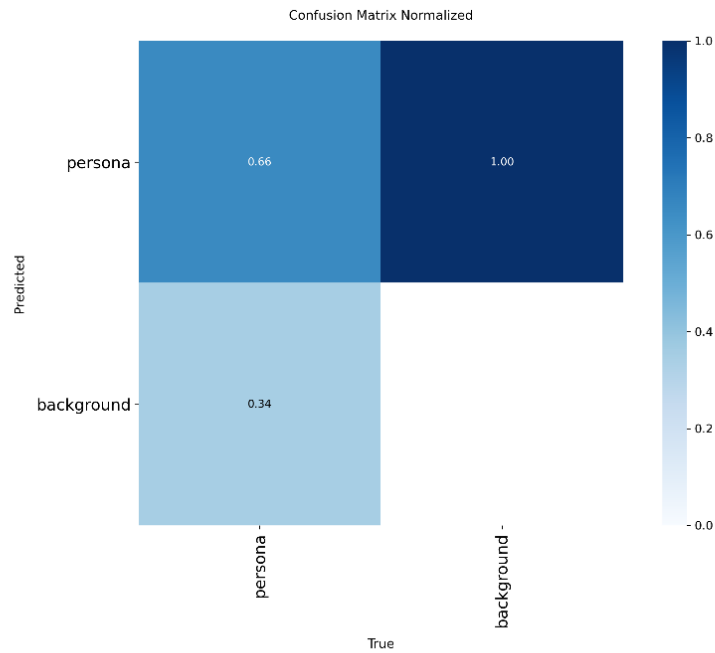


Figura 8.4. Matriz de confusión del modelo inicial.

La matriz de confusión muestra que el modelo detecta a dos de cada tres personas presentes, mientras que el tercio restante no se detecta y se clasifica como fondo. En cambio, apenas comete el error contrario: rara vez marca a alguien como persona que no lo es. El modelo, por tanto, no falla por generar falsas alarmas, sino por no detectar a una parte de las personas presentes, lo que, en una aplicación de rescate, constituye el error más costoso.

La inspección visual de las detecciones matiza este resultado. El modelo, en realidad, sí localiza a la mayoría de las personas, incluso en escenarios variados y en distintas posturas (de pie, tumbadas o parcialmente ocultas por la vegetación, Figura 8.5). Su principal dificultad radica en las personas situadas a mayor distancia, que aparecen con muy pocos píxeles y, a menudo, no se detectan (Figura 8.6).

Este análisis orienta directamente la vía de mejora que se explora en la siguiente sección: reforzar el conjunto de entrenamiento con ejemplos de personas a mayor distancia, que son los casos en los que el modelo muestra mayor dificultad.



Figura 8.5. Detecciones del modelo inicial en imágenes de **escenas variadas**.

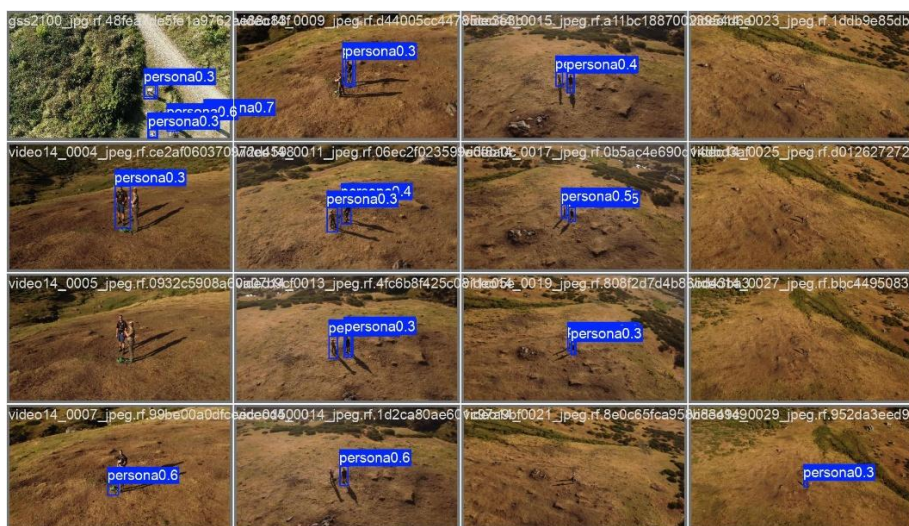


Figura 8.6. Detecciones del modelo inicial en imágenes de prueba de **personas a distancia**.

### 8.5.3. Mejora del modelo mediante ampliación dirigida del conjunto de entrenamiento

El análisis de errores del modelo inicial señaló que su principal debilidad era la detección de personas situadas a mayor distancia, que en la imagen aparecen con muy pocos píxeles. A partir de este diagnóstico, se decidió reforzar el conjunto de entrenamiento con ejemplos precisamente de estos casos, manteniendo fijos los conjuntos de validación y de prueba para medir el efecto real de la ampliación.

La incorporación de nuevas imágenes se realizó de forma incremental. En una primera fase se añadió un número reducido de imágenes de personas a mayor distancia, comprobándose que, pese a su escasa cantidad, ya produjeron una mejora apreciable en el *recall*. Confirmada la utilidad de esta dirección, se amplió el conjunto con más ejemplos del mismo tipo, hasta alcanzar un total de 540 imágenes de entrenamiento, incorporando únicamente casos representativos de las situaciones en las que el modelo



mostraba mayor dificultad. En la Figura 8.7 se pueden ver algunos ejemplos de estas nuevas imágenes añadidas y su etiquetado en Roboflow.



Figura 8.7. Ejemplos de imágenes añadidas en Roboflow durante la ampliación del conjunto de entrenamiento, con personas a mayor distancia y menor resolución en píxeles.

El modelo resultante se evaluó sobre el mismo conjunto de prueba que el modelo inicial. La comparación entre ambos se presenta en la Tabla 8.3.

TABLA 8.3. COMPARACIÓN ENTRE EL MODELO INICIAL Y EL MODELO TRAS LA AMPLIACIÓN DEL CONJUNTO DE ENTRENAMIENTO

| Métrica   | Modelo inicial | Modelo ampliado |
|-----------|----------------|-----------------|
| Precisión | 0,693          | 0,684           |
| Recall    | 0,542          | 0,659           |
| mAP@50    | 0,586          | 0,640           |
| mAP@50-95 | 0,184          | 0,205           |

La mejora más significativa se observa en el *recall*, que pasa de 0,542 a 0,659. Este es precisamente el resultado buscado: al reforzar el entrenamiento con los casos que el modelo no detectaba, este pasa a localizar a una proporción notablemente mayor de las personas presentes. La mejora se refleja también en el mAP@50, que sube de 0,586 a 0,640, mientras que la precisión se mantiene prácticamente estable, con un ligero descenso esperable, ya que, al detectar más personas, el modelo presenta un pequeño aumento de falsas alarmas. En una aplicación de búsqueda y rescate, donde el *recall* es la métrica prioritaria, este intercambio resulta claramente favorable.

Esta mejora se aprecia con especial claridad en la matriz de confusión (Figura 8.8). Frente al modelo inicial, que detectaba a dos de cada tres personas, el modelo ampliado detecta ya a tres de cada cuatro: la proporción de personas correctamente localizadas asciende del 66 % al 75 %, mientras que la fracción de personas no detectadas se reduce del 34 % al 25 %. El modelo mantiene, además, su bajo índice de falsas alarmas.

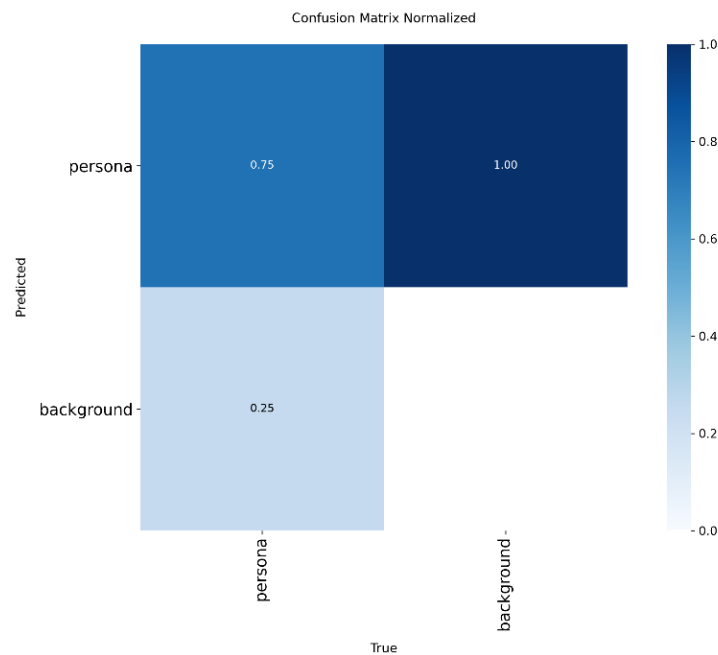


Figura 8.8. Matriz de confusión del modelo tras la ampliación.

La inspección visual de las detecciones confirma este comportamiento. El modelo ahora detecta con mayor fiabilidad a las personas que están a mayor distancia (Figura 8.9). Antes, este era el principal punto débil del modelo inicial. También mantiene su capacidad de localizar personas en muchos escenarios y posturas, como de pie, tumbadas o parcialmente ocultas por la vegetación (Figura 8.10). La mejora, que se ve en todas las métricas, también se nota en el resto de las situaciones porque se aumentó el número de ejemplos de entrenamiento. Sin embargo, la detección de personas a mayor distancia es donde el caso reforzado muestra la mejora más clara.



Figura 8.9. Detecciones del modelo ampliado sobre imágenes de prueba **de personas a distancia**.



Figura 8.10. Detecciones del modelo ampliado sobre imágenes de **escenas variadas**

Este resultado confirma que el enfoque utilizado es válido. El análisis de errores permitió identificar con precisión las carencias del modelo. Una ampliación dirigida del conjunto de entrenamiento, centrada en esas carencias concretas y no en aumentar el volumen de datos de forma indiscriminada, se tradujo en una mejora medible y coherente con las necesidades de la aplicación.

Cabe señalar, asimismo, que el modelo tiende a asignar niveles de confianza moderados a sus detecciones. La elección del umbral de confianza, a partir del cual una detección se considera válida, es un parámetro de configuración relevante en el despliegue, especialmente en una aplicación de rescate, y se aborda en la sección 8.5.



Este modelo se adopta como versión final del sistema de detección. No obstante, el análisis de los resultados evidencia que el rendimiento puede incrementarse mediante un conjunto de entrenamiento más extenso, especialmente en situaciones complejas: personas a gran distancia, siluetas con oclusiones parciales o escenarios con baja visibilidad. La principal limitación actual radica en la escasez de bancos públicos de imágenes aéreas orientados específicamente a emergencias, lo que hace necesaria la captura continua de material propio en terreno.

Conviene señalar este resultado con claridad: un *recall* de 0,659 indica que el modelo no detecta automáticamente a cerca de una de cada tres personas en las imágenes de prueba. Aunque este registro es inferior al habitual en la literatura SAR, donde trabajos sobre conjuntos de referencia como HERIDAL [6] reportan mAP@50 entre 0,73 y 0,81, resulta plenamente coherente con una prueba de concepto entrenada con un *dataset* propio reducido de 717 imágenes y de clase única. Por esta razón, el sistema se plantea como una herramienta de apoyo al operador (*Human-in-the-Loop*): marca en pantalla a los posibles candidatos para su verificación visual y, en ningún caso, sustituye el criterio humano. Esto basta para validar cómo funciona todo el *pipeline* a bordo y sienta las bases para lograr un mejor rendimiento al ampliar el *dataset* en el trabajo futuro (sección 12.2).

#### 8.5.4. Comparativa de variantes (*nano* vs *small*)

Todo el proceso de desarrollo y mejora descrito hasta ahora se realizó utilizando la variante *small* de YOLO11. La elección de esta variante durante la experimentación se debe a que, al disponer de mayor capacidad, refleja con mayor fidelidad el efecto de cada decisión, lo que permite evaluar las mejoras sobre una base más estable.

No obstante, el modelo definitivo debe ejecutarse en una Raspberry Pi, un hardware de recursos limitados en el que la velocidad de inferencia y la ligereza son factores críticos. Por ello, una vez optimizado el modelo, se compararon las dos variantes candidatas, *small* y *nano*, sobre el mismo conjunto de prueba, siendo su única diferencia el tamaño del modelo. Los resultados se presentan en la Tabla 8.4.

TABLA 8.4. COMPARACIÓN ENTRE LAS VARIANTES SMALL Y NANO DE YOLO11

| Métrica              | Small (yolo11s) | Nano (yolo11n) |
|----------------------|-----------------|----------------|
| Precisión            | 0,684           | 0,706          |
| Recall               | 0,659           | 0,601          |
| mAP@50               | 0,640           | 0,591          |
| mAP@50-95            | 0,205           | 0,193          |
| Velocidad inferencia | 6,6 ms          | 4,7 ms         |
| Parámetros           | 9,4 M           | 2,6 M          |

Los resultados reflejan un compromiso claro: el modelo *small* es algo más preciso (mAP@50 de 0,640 frente a 0,591), mientras que el *nano* es sensiblemente más eficiente, con una inferencia un 30 % más rápida y un tamaño casi cuatro veces menor. Dado que el sistema debe procesar vídeo en tiempo real a bordo del dron, donde la eficiencia es determinante, la pérdida moderada de precisión del *nano* se ve ampliamente compensada



por su rapidez y ligereza. Por ello, se selecciona la variante *nano* como modelo final para el despliegue, cuya integración en la Raspberry Pi se describe en la sección 8.6.

## 8.6. Comparativa de motores de inferencia en CPU y despliegue final con NPU Hailo-8L

Después de entrenar y validar el modelo de visión artificial, el siguiente paso fue preparar su implementación operativa en el dron. Antes de incorporar la NPU Hailo-8L HAT+ al hardware de vuelo, se decidió comprobar si la CPU de la Raspberry Pi 5 podía procesar vídeo en tiempo real o si era necesario incorporar un acelerador específico. Para ello, se evaluó el rendimiento del modelo en un mismo vídeo de referencia y bajo las mismas condiciones, utilizando cuatro formatos de inferencia distintos.

La prueba consistió en procesar el vídeo de referencia completo (944 fotogramas, 31,5 s a 29,97 FPS nativos), analizando todos los fotogramas capturados por la cámara (sin submuestreo de vídeo), utilizando el modelo exportado a cada uno de los formatos evaluados. Al mantener constantes el vídeo, la resolución de entrada (640×640) y el hardware, cualquier variación en los resultados se debe únicamente a la eficiencia del propio motor de inferencia. Los cinco formatos analizados fueron: el modelo nativo en PyTorch (.pt), tomado como punto de partida; su exportación a ONNX en precisión simple (FP32); esa misma versión optimizada mediante cuantización a enteros de 8 bits (ONNX-INT8); NCNN, para optimizar la inferencia en arquitecturas móviles y procesadores ARM embebidos; y el formato HEF, compilado para ejecutarse directamente en el coprocesador neuronal Hailo-8L. Los resultados obtenidos se resumen en la Tabla 8.5.

TABLA 8.5. COMPARATIVA DEL RENDIMIENTO, LA LATENCIA Y EL CONSUMO DE RECURSOS DE LOS MOTORES DE INFERENCIA EN LA RASPBERRY PI 5 (CPU Y NPU).

| Parámetro                         | PyTorch (PT)          | ONNX (FP32)           | ONNX (INT8)           | NCNN                  | HEF (Hailo-8L)        |
|-----------------------------------|-----------------------|-----------------------|-----------------------|-----------------------|-----------------------|
| Frames procesados                 | 944                   | 944                   | 944                   | 944                   | 944                   |
| Tiempo total                      | 4 min 14,80 s         | 3 min 58,44 s         | 3 min 57,73 s         | 1 min 54,89 s         | 32,19 s               |
| Tasa de procesamiento media (FPS) | 3,72                  | 3,98                  | 4,00                  | 8,35                  | 29,49                 |
| Latencia media por frame          | 268,59 ms             | 251,22 ms             | 249,93 ms             | 119,77 ms             | 33,91 ms              |
| Latencia p95 por frame            | 309,33 ms             | 288,94 ms             | 278,92 ms             | 140,41 ms             | 35,67 ms              |
| Uso medio de CPU (%)              | 205,3 % (2,1 núcleos) | 394,1 % (3,9 núcleos) | 393,2 % (3,9 núcleos) | 348,2 % (3,5 núcleos) | 175,4 % (1,8 núcleos) |
| Memoria RAM pico                  | 526,5 MB              | 570,6 MB              | 572,2 MB              | 526,1 MB              | 185,5 MB              |
| Detecciones totales               | 852                   | 861                   | 848                   | 854                   | 894                   |
| Frames con detección              | 813                   | 817                   | 799                   | 814                   | 835                   |
| Confianza media                   | 0,59 (59 %)           | 0,59 (59 %)           | 0,59 (59 %)           | 0,59 (59 %)           | 0,58 (58 %)           |

El HEF ofrece los mejores resultados entre los cinco motores evaluados. Reporta el mayor número de detecciones (894, frente a 848-861 del resto) y la confianza media más baja (0,58, frente a 0,59). Esta pequeña diferencia se explica por el postproceso propio del *runtime* del acelerador: tras la inferencia, cada motor aplica un filtro llamado NMS (*Non-*

*Maximum Suppression*), que descarta las cajas redundantes que un mismo objeto genera al solaparse entre sí, quedándose solo con la más fiable. Como el HEF usa una implementación propia de este filtro en lugar de la de Ultralytics que comparten los otros cuatro motores, es esperable que el umbral de descarte no sea idéntico byte a byte. Por eso hay un margen de unas pocas decenas de detecciones y una décima de confianza. Esta diferencia es pequeña y no cambia la conclusión general del capítulo. Además, es el único motor capaz de alcanzar FPS acordes con la velocidad de captura de la cámara, por lo que puede procesar prácticamente todos los fotogramas en tiempo real, sin acumular retraso.

Este dato lo observamos con mayor detalle en la siguiente tabla comparativa. La Tabla 8.6 traduce la latencia media de cada motor al submuestreo mínimo que este necesitaría para no acumular retraso respecto al ritmo de captura de la cámara (29,97 FPS, un presupuesto de 33,37 ms por fotograma). Solo el HEF es capaz de analizar el vídeo completo sin descartar fotogramas: los motores de CPU necesitarían saltarse entre 3 de cada 4 (NCNN) y 8 de cada 9 (PyTorch/ONNX) fotogramas capturados para mantener el ritmo del vídeo.

TABLA 8.6. SUBMUESTREO MÍNIMO NECESARIO POR MOTOR PARA NO ACUMULAR RETRASO FRENTE AL RITMO DE CAPTURA DE LA CÁMARA (29,97 FPS)

| Runtime        | Latencia media/frame | ¿Cabe en 33,37 ms?        | vid_stride mínimo    |
|----------------|----------------------|---------------------------|----------------------|
| PyTorch (PT)   | 268,59 ms            | No (8,05× el presupuesto) | 9                    |
| ONNX (FP32)    | 251,22 ms            | No (7,53×)                | 8                    |
| ONNX (INT8)    | 249,93 ms            | No (7,49×)                | 8                    |
| NCNN           | 119,77 ms            | No (3,59×)                | 4                    |
| HEF (Hailo-8L) | 33,91 ms             | Casi (1,02×)              | 2 (margen del 1,6 %) |

De estos resultados se extraen conclusiones bastante claras sobre las limitaciones del procesamiento en CPU:

- **PyTorch nativo como referencia.** Con una latencia de 268,59 ms por fotograma (3,72 FPS), procesar el vídeo de referencia tarda 4 min 14,8 s, más de 8 veces la duración real del vídeo (31,5 s). Para no acumular retraso frente a una cámara a 30 FPS, este motor necesitaría descartar 8 de cada 9 fotogramas capturados ( $\text{'vid\_stride'} \geq 9$ ), lo que, en la práctica, implica perder de vista a una persona durante fracciones de segundo completas entre cada fotograma analizado.
- **Optimización con ONNX y efecto de la cuantización.** La exportación a ONNX FP32 mejora el rendimiento a 3,98 FPS (251,22 ms), y la cuantización a INT8 lo deja prácticamente igual (4,00 FPS, 249,93 ms). Para este modelo y hardware, la cuantización en CPU no aporta una ganancia de velocidad significativa. Ambas variantes necesitarían un  $\text{'vid\_stride'}$  de al menos 8 para evitar la acumulación de retraso.

- **Ventaja de NCNN frente a la arquitectura ARM.** NCNN es, con diferencia, el más eficiente en CPU: 8,35 FPS (119,77 ms), más del doble que ONNX, con un ‘vid\_stride’ mínimo de 4 para mantener el ritmo. Resulta significativo que lo consiga usando menos núcleos que ONNX (3,5 frente a 3,9; ver más abajo por qué un mayor uso de CPU no se traduce en mayor velocidad).

La confianza media de las detecciones se sitúa entre el 58 % y el 59 % en los cinco motores. Esto confirma que el formato de inferencia no afecta de manera relevante la calidad de las predicciones del modelo. Solo varía la velocidad a la que se obtienen. Por eso, elegir el motor es una decisión de eficiencia y no de precisión.

**Uso de la CPU y su relación con la velocidad.** El uso de la CPU no predice la velocidad obtenida. ONNX *runtime* (FP32 e INT8) satura casi los 4 núcleos de la Raspberry Pi 5 (394,1 % y 393,2 % respectivamente) y, aun así, rinde menos de la mitad que NCNN, que logra 8,35 FPS usando solo 3,5 núcleos. Esto se debe a que ONNX *runtime* paraleliza agresivamente la inferencia entre todos los hilos disponibles por defecto, pero ese paralelismo adicional no se traduce en un *throughput* proporcional: parte del grafo de cómputo tiene dependencias secuenciales que no se benefician de más hilos, y el coste de sincronización entre núcleos empieza a pesar más que la ganancia. NCNN, compilado específicamente para NEON/ARM con un planificador de hilos más ajustado a esta arquitectura, consigue más trabajo útil por núcleo ocupado. PyTorch usa la menor cantidad de CPU de los cuatro motores de software (205,3 %, apenas 2 núcleos) pese a ser el más lento, un indicio de que su *build* por defecto en ARM no explota el paralelismo disponible.

El 175,4 % de CPU del HEF no corresponde a la inferencia en sí, ese cómputo se ha desplazado por completo a la NPU, sino al trabajo que queda en el host: leer el fotograma, convertirlo al formato de entrada, transferirlo al dispositivo Hailo por PCIe y decodificar la salida de la red neuronal. Es, en la práctica, todo el trabajo de apoyo alrededor del modelo, no el modelo en sí.

**Memoria RAM y el efecto de la cuantización.** La versión cuantizada a INT8 no reduce la RAM pico frente a la FP32 (572,2 MB frente a 570,6 MB), de hecho, es marginalmente superior. La explicación está en qué compone esos 570 MB: los pesos del modelo, del orden de unos pocos megabytes para una red nano, son una fracción mínima del total. El grueso corresponde al intérprete de Python, las bibliotecas cargadas y los búferes de activaciones intermedias durante la inferencia, que en un grafo cuantizado no siempre son enteros de 8 bits de principio a fin.

El contraste que sí es contundente es el del HEF: 185,5 MB, menos de un tercio que cualquiera de los motores de CPU. Los pesos del modelo y la mayor parte de los búferes de cómputo residen en la memoria propia del acelerador Hailo-8L, no en la RAM del host, que solo necesita mantener los fotogramas de entrada/salida y el propio intérprete.

Los resultados del acelerador Hailo-8L confirman la ventaja de la aceleración por hardware: 29,49 FPS, con una latencia de 33,91 ms por fotograma. Procesar el vídeo completo de 31,5 s tarda 32,19 s; prácticamente en tiempo real, con un sobre coste de

apenas el 2,2 % sobre la duración del propio vídeo. Es, de los cinco motores evaluados, el único capaz de analizar todos los fotogramas capturados por la cámara sin necesidad de submuestreo (Tabla 8.6): con ‘vid\_stride’ = 1, el resto de los motores acumularía retraso frente al vídeo en directo a un ritmo de entre 3,6 y 8,1 veces la duración real, mientras que el HEF lo sigue casi al mismo ritmo al que se captura.

En sentido estricto, con una latencia de 33,91 ms frente a un presupuesto de 33,37 ms por fotograma, el HEF sigue siendo 0,54 ms/fotograma más lento que el tiempo real, un margen del 1,6 % que en una sesión de vuelo suficientemente larga terminaría acumulando un pequeño retraso. En la práctica, ese margen se asume con un submuestreo mínimo (‘vid\_stride’ = 2) sin pérdida perceptible de cobertura, muy lejos de la degradación que exigiría cualquiera de los motores de CPU.

Como se ha visto, delegar la inferencia en la NPU deja además margen de CPU de sobra para el control de vuelo, la telemetría MAVLink y las comunicaciones simultáneamente.

Por último, conviene aclarar que el mal resultado de la cuantización INT8 en la CPU no anticipa cómo funcionará el modelo en la Hailo. Se trata de métodos completamente distintos. En la CPU, la aritmética de baja precisión se emula con la penalización ya mencionada. En cambio, la Hailo cuenta con hardware específico para ese tipo de operaciones. Así, la cuantización deja de ser un problema y pasa a ser justo lo que el acelerador está hecho para aprovechar.

**Despliegue final con acelerador de hardware (HEF y NPU Hailo-8L).** Como paso definitivo en la optimización del sistema embarcado, el modelo se compiló en el formato específico .hef para ejecutarse directamente en el coprocesador neuronal Hailo-8L integrado en el HAT de la Raspberry Pi 5.

Los resultados de esta configuración, incorporados en la Tabla 8.5, confirman las claras ventajas del uso de aceleración de hardware dedicada: el rendimiento alcanza los 29,49 FPS, con una latencia reducida a 33,91 ms, y el procesamiento se completa en 32,19 segundos. Asimismo, se observa una descarga muy importante en los núcleos de la CPU (cuyo uso baja al 175,4 %, 1,8 núcleos) y un menor consumo de memoria RAM pico (185,5 MB), lo que valida esta arquitectura como la opción óptima para el despliegue operativo en vuelo real.

A diferencia de la CPU, cuyos núcleos generales deben dividirse entre el sistema operativo, las comunicaciones, la telemetría y el cálculo secuencial de la red neuronal, el coprocesador Hailo-8L está diseñado para procesar modelos de inteligencia artificial de forma nativa.

Sin embargo, hay que tener en cuenta que estas métricas se han obtenido a partir de un vídeo de referencia pregrabado. En un vuelo real, con la cámara capturando en directo, el sistema tiene además que gestionar los búferes de vídeo y la entrada/salida de datos del sensor. Aunque esos pequeños factores propios del directo exigen un esfuerzo extra, contar con la NPU dedicada es justo lo que garantiza que el dron disponga siempre del margen de cómputo necesario para absorber esa carga y volar con total estabilidad en una misión SAR.

## 9. VALIDACIÓN EXPERIMENTAL

En este capítulo se detalla la fase final de la integración y la validación experimental del sistema Drone-SAR (*Guardian Eye*). En contraste con los capítulos anteriores, centrados en el diseño individual del hardware, las comunicaciones, la plataforma IoT y la visión artificial, aquí se evalúa el funcionamiento del sistema en su conjunto. Se detalla detenidamente qué pruebas se han realizado en simulación (SITL), cuáles se han validado en tierra con la electrónica real montada (fase Semi-HITL) y qué parámetros se han comprobado durante los ensayos de vuelo real, cerrando el capítulo con la matriz de cumplimiento de objetivos del proyecto.

### 9.1. Metodología y entorno de ensayos

Con el objetivo de garantizar una integración ordenada y segura tanto para los equipos como para el entorno, el proceso de validación se ha estructurado en cuatro fases progresivas mediante una metodología ascendente (*bottom-up*):

- **Fase 1. Simulación SITL (*Software In The Loop*):** En el equipo de desarrollo se ha ejecutado el binario nativo de ArduPilot enlazado con Mission Planner, fijando la posición de origen en el campo de vuelo mediante el parámetro `--home 40.5979097,-3.9988503,880,0`. (Coordenadas exactas del Club Alas de Galapagar, entorno donde mas tarde haríamos las pruebas de vuelo real). Este entorno de simulación ha permitido poner a punto la máquina de estados de navegación, depurar los scripts de control en Python con *pymavlink* y verificar la multiplexación de sockets MAVLink antes de transferir el software a las placas físicas.
- **Fase 2. Banco de Integración Hardware en tierra (Semi-HITL):** Como paso intermedio antes del vuelo y para verificar la arquitectura de cómputo real, se conectó la Raspberry Pi 5 a la controladora Pixhawk 6X del dron mediante el enlace serie UART (puerto TELEM3). En esta prueba, se retiraron las hélices siguiendo los protocolos de seguridad y se verificó la ejecución de las órdenes de armado, el cambio de modos y la lectura de la telemetría directamente en la electrónica del vehículo.
- **Fase 3. Campaña de Ensayos y Preparación Operativa:** Las pruebas físicas de sustentación y navegación se realizaron en las instalaciones autorizadas del Club de Aeromodelismo Alas de Galapagar (Madrid), en un espacio aéreo oficialmente clasificado como zona de aeromodelismo (identificador AIP: LECMUAV090). Se operó bajo la subcategoría A3 de la categoría abierta (AES/AESA) y con las habilitaciones de operador y piloto vigentes (Anexos B y C). La campaña tuvo dos partes. En la primera se revisaron la sustentación, la estabilidad y el peso de la plataforma de vuelo, registrándose un MTOW de 1.817 g (Tabla 4.1).
- **Fase 4. Integración de Emergencia y Validación de Inferencia en Vuelo:** En la segunda parte se realizó un vuelo con el sistema operativo completo a bordo. Como no fue posible integrar a tiempo el regulador UBEC que alimenta el nodo

*edge* desde la batería principal (deuda técnica, apartado 12.3), se optó por una solución alternativa: alimentar la Raspberry Pi 5 y el acelerador mediante una batería externa portátil, lo que sumó unos 210 gramos adicionales y redujo la autonomía y el margen de empuje disponible. Además, y aunque inicialmente estaba previsto hacer el vuelo con una cámara de reducido tamaño, específica de Raspberry, y con un bajo consumo eléctrico (gracias a una conexión por cable CSI), no conseguimos que la cámara fuera reconocida adecuadamente por la Raspberry PI, así que se planteó también una solución alternativa temporal a través de una cámara USB que teníamos accesible (a costa también de otros 200 gramos aproximados adicionales de peso). Aceptando este compromiso para validar el sistema integrado dentro del plazo del trabajo, la Raspberry Pi 5 realizó inferencias de YOLO en tiempo real sobre el vídeo grabado y procesado a bordo, cuyos hallazgos se detallan en los apartados 9.3 y 9.4.

## 9.2. Validación de la cadena de mando y teleoperación

La integración de la cadena de mando remota se validó con éxito en el banco de hardware al conectar la interfaz de usuario con la electrónica física del dron:

1. **Interacción desde el Panel Web (PWA):** Las instrucciones enviadas desde la interfaz gráfica (botones de *Armar*, *Desarmar* y cambio de modos) se transmiten por HTTPS a la API REST en Flask desplegada en AWS EC2.
2. **Pasarela MQTT y traducción a bordo:** La orden se publica en el broker Mosquitto y viaja por el túnel seguro de Tailscale (WireGuard) hasta el script receptor en la Raspberry Pi 5, que la convierte en comandos binarios MAVLink v2.0 para la Pixhawk 6X.
3. **Respuesta de la electrónica física:** En el banco se confirmó la respuesta inmediata de la controladora: el encendido de los variadores (ESC), el cambio de estados y la activación de los avisos acústicos y luminosos de seguridad del autopiloto.

## 9.3. Validación funcional de las comunicaciones y la plataforma IoT

Se evaluó la infraestructura de telecomunicaciones e IoT para comprobar que los canales de enlace funcionan en conjunto, que la retransmisión de vídeo es estable y que los datos se mantienen de forma fiable.

### A. Verificación de los Canales de Comunicación

- **Mando Manual RC (2,4 GHz):** Enlace primario directo para el control manual de seguridad por parte del piloto (FlySky FS-i6X con receptor FS-iA10B).
- **Telemetría Local (433 MHz):** Supervisión punto a punto al Mission Planner en el PC de campo, vía radio SiK, sin requerir conexión a internet.
- **Conectividad Celular 4G/LTE y VPN:** El nodo *edge* se conecta a la nube mediante un módem USB 4G y se enruta de forma segura y privada a través de Tailscale (WireGuard) a una instancia de EC2 de AWS.

- **Retransmisión de Vídeo en Directo:** El flujo de vídeo anotado se publica mediante RTSP en el servidor de medios (MediaMTX) en la nube y puede verse de forma remota en el navegador mediante WebRTC.
- **Red Wi-Fi Local:** En los ensayos de banco se comprobó el alcance del Wi-Fi convencional de la Raspberry Pi, sin antenas ni equipos específicos era de apenas 20 m. Por eso se descartó como enlace principal. Se estableció la red celular 4G/LTE como canal permanente para la nube. Se reservó el Wi-Fi para tareas de corto alcance (mantenimiento, acceso SSH, depuración y descarga de registros), para evitar gastar los datos del plan de prepago de la tarjeta SIM.

## **B. Monitorización en InfluxDB y Resiliencia del Store-and-forward**

Se verificó que InfluxDB recibe datos de forma simultánea y que los cuadros de mando de Grafana muestran información de los cuatro dominios definidos (Ambiental BME680, Sistema RPi 5, Vuelo Pixhawk y Detección).

También se probó el mecanismo *store-and-forward* durante una interrupción forzada de la conexión. Cuando se cortó la conexión 4G, los procesos continuaron guardando datos en bases de datos SQLite locales (con enviado = 0). Al recuperar la conexión, el sistema envió los datos pendientes por MQTT con acuse de recibo (QoS 1) y actualizó el estado a enviado = 1. Se comprobó que las series temporales en InfluxDB conservaron la marca de tiempo original sin generar picos artificiales al reconectar.

## **C. Comportamiento de los enlaces y de la persistencia en vuelo real**

Durante el vuelo del sistema integrado (apartado 9.1), se observó el funcionamiento del enlace celular y del mecanismo de *store-and-forward* en condiciones reales. El enlace de datos 4G/LTE se mantuvo activo con el dron en tierra y a baja altura. Sin embargo, la transmisión de vídeo en directo (RTSP al servidor de medios en la nube) se cortó cuando la aeronave alcanzó unos 10 metros de altura. Como se trataba de un flujo en tiempo real sin almacenamiento intermedio, no hubo imagen remota durante ese periodo.

En cambio, la grabación local en la Raspberry Pi 5 se realizó sin problemas. Los datos de los cuatro dominios, incluidos los eventos de detección generados durante la pérdida de enlace, se guardaron de forma segura en el búfer local de SQLite. Cuando se restableció la conexión tras el descenso, la información se sincronizó por completo con InfluxDB y conservó sus marcas de tiempo originales. Este comportamiento confirma, en vuelo real, la arquitectura de *store-and-forward*. Hasta ahora, solo se había probado su funcionamiento mediante un corte de red forzado en el banco de pruebas (apartado B).

La causa exacta de la caída del enlace de vídeo en altura no ha sido identificada. Podría estar relacionada con interferencias electromagnéticas por la cercanía del módem a otro componente, una orientación desfavorable de la antena en el fuselaje, la reelección de celda o un ancho de banda de subida insuficiente. Por este motivo, se remite a las pruebas posteriores (apartado 12.4). Este fenómeno, en principio, coincide con la literatura previa sobre vehículos aéreos conectados a redes celulares, que señala la estabilidad del enlace a distintas alturas como un desafío técnico aún sin resolver (apartado 2.2 [7]).

Sin embargo, es importante ser cautos al hacer esa comparación. Los estudios citados en la sección 2.2 analizan la degradación del enlace a alturas de unos 170 m [9], mucho mayores que los apenas 10 m en los que se produjo el corte en este ensayo. A esa altura, el dron todavía estaba dentro del lóbulo principal de las antenas de las estaciones base cercanas, que es el mecanismo concreto descrito en esa literatura. Por eso, también es posible que la causa fuera un problema puntual de la línea de datos del módem USB, como una desconexión de la sesión de datos o un fallo del propio adaptador, y no necesariamente un problema de cobertura celular en altura. Se considera necesario repetir el ensayo, incluso con otro operador de telefonía móvil, para confirmar si el comportamiento observado depende de la red utilizada o si se trata de un problema puntual del módem.

#### 9.4. Evaluación de la Detección Edge-AI sobre Vídeo y Resultados del Modelo

La validación del sistema de visión artificial demostró que puede procesar vídeo aéreo y localizar personas en tiempo real mediante el modelo entrenado y la aceleración de hardware.

- **Rendimiento del entrenamiento (YOLO11):** Al ampliar el *dataset* con ejemplos más complejos (personas tumbadas, sentadas y a mayor distancia), el modelo logró un *recall* de 0,659 (antes 0,542) y un mAP@50 de 0,640 en el conjunto de *test* independiente. Este *recall*, en nano, de 0,601 corresponde al modelo de prototipo; como se detalla en la sección 8.5.3, implica que una de cada tres personas no llega a ser detectada, por lo que un uso operativo real requeriría un modelo con un *recall* claramente superior.
- **Aceleración con NPU Hailo-8L:** Al usar el modelo compilado en formato HEF en el acelerador neuronal, el sistema procesó el vídeo a 29,49 FPS, con una latencia media de 33,91 ms por fotograma. Al delegar la inferencia en la NPU, el uso de CPU se redujo a 175,4 % (1,8 núcleos), frente al 348,2 % (3,5 núcleos) de NCNN, lo que libera capacidad de cómputo para el control de vuelo y las comunicaciones y reduce la carga de trabajo sostenida sobre el procesador principal de la Raspberry Pi.
- **Procesamiento de secuencias y visualización de detecciones:** En las pruebas con vídeo aéreo, se confirmó la detección continua de personas marcadas con sus *bounding boxes*. El sistema generó automáticamente eventos de alerta georreferenciados a partir del archivo compartido *posicion\_actual.json* y envió la información en tiempo real al panel del operador.
- **Detección en vuelo real:** En el vuelo del sistema integrado (apartado 9.1), la Raspberry Pi 5 procesó el vídeo de la cámara a bordo y realizó inferencias en tiempo real sobre el acelerador. El modelo detectó a las dos personas que entraron en el campo de visión de la cámara y generó los eventos de alerta correspondientes. Sin embargo, la imagen presentó oscilaciones debido a las vibraciones y a los cambios de actitud de la aeronave en vuelo. Aunque estas perturbaciones no impidieron la detección en esta prueba, degradan la estabilidad



del fotograma y confirman, con hechos, la conveniencia de instalar una estabilización mecánica de la cámara, como un *gimbal* de dos ejes.

Esta opción se descartó en el prototipo actual por falta de presupuesto. Sin embargo, se incluyó como línea de trabajo futura (apartado 12.4). A pesar de eso, la prueba válida de extremo a extremo la cadena de visión, que incluye la captura, la inferencia acelerada, la generación de alerta y la grabación en condiciones de vuelo. Además, complementa con éxito la evaluación cuantitativa del modelo en el conjunto de prueba (apartado 8.5).

El video del vuelo real, y los resultados de la inferencia, pueden consultarse en [https://www.youtube.com/watch?v=1aA\\_bSsyBIM](https://www.youtube.com/watch?v=1aA_bSsyBIM). En la Figura 9.1 puede verse una muestra del vídeo y el código QR para acceso desde el móvil.



Figura 9.1. Demostración en vuelo del sistema integrado (vídeo), con enlace directo al mismo.

Asimismo, al inicio de la prueba se registró un falso positivo, un resultado esperable en un prototipo de prueba de concepto. Este comportamiento es coherente con la precisión moderada del modelo y con el tamaño reducido del *dataset* propio, y no invalida el resultado: en un contexto SAR se prioriza el *recall*, por lo que un exceso puntual de falsas alarmas resulta preferible a una detección omitida, y el operador humano descarta visualmente esos avisos sin coste para la misión (modelo *Human-in-the-Loop*). Lejos de ser una limitación del enfoque, esto evidencia el amplio margen de mejora del sistema: la reducción de estos falsos positivos mediante la ampliación del conjunto de datos se plantea como línea de trabajo futura (apartado 12.2).

## 9.5. Matriz de Cumplimiento de Objetivos

Como conclusión del trabajo, en la Tabla 9.1 se resume el nivel de cumplimiento de los objetivos específicos establecidos en la Sección 1.2, relacionándolo directamente con los resultados experimentales y las evidencias técnicas detalladas en la memoria.

TABLA 9.1. MATRIZ DE CUMPLIMIENTO DE OBJETIVOS DEL TFG.

| Objetivo                                              | Estado y evidencia                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|-------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1. Análisis normativo y regulatorio SAR               | Cumplido. Estudio completo y tramitación efectiva de las habilitaciones (Cap. 3 y 10; Anexos B y C).                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| 2. Selección y montaje de la plataforma hardware      | Cumplido. Selección justificada de la plataforma frente a alternativas (piezas sueltas, reutilización de un dron cedido) y montaje e integración del kit Holybro X500 V2 con la Pixhawk 6X y el nodo <i>edge</i> ; balance de masas y de potencia verificados, con MTOW real de 1.817 g medido en báscula (Cap. 4; Tablas 4.1 y 4.2).                                                                                                                                                                                                                             |
| 3. Diseño del sistema de comunicaciones y telecontrol | Cumplido. RC y telemetría verificados en vuelo real; enlace 4G/Tailscale/AWS verificado en banco (Cap. 5 y 9).                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| 4. Arquitectura IoT en la nube (AWS EC2)              | Cumplido y validado en el banco. Ingesta MQTT→InfluxDB, cuadros de Grafana y resiliencia <i>store-and-forward</i> probada con corte forzado de 4G (Cap. 6, 7 y 9.3).                                                                                                                                                                                                                                                                                                                                                                                              |
| 5. Construcción y curación de <i>dataset</i> propio   | Cumplido. Conjunto de 717 imágenes, clase única, particionado en videos (Cap. 8).                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| 6. Entrenamiento y optimización YOLO en <i>Edge</i>   | Cumplido a nivel de prototipo. Se entrenó y optimizó el modelo; el mejor <i>recall</i> y mAP se obtuvieron con la variante small ( <i>recall</i> 0,659; mAP@50 0,640). Para el despliegue a bordo se seleccionó la variante nano ( <i>recall</i> 0,601; mAP@50 0,591), más rápida y ligera, ejecutada a 29,49 FPS en la NPU Hailo-8L con vídeo aéreo pregrabado (Cap. 8 y 9.4). El <i>recall</i> alcanzado es propio de una prueba de concepto (8.5.3 y 8.5.4).                                                                                                   |
| 7. Validación integral del sistema completo           | Cumplida. Validación en fases (SITL, banco Semi-HITL y vuelo real). Cadena de mando y de visión validadas de extremo a extremo en el banco. Plataforma validada en vuelo real. Demostración en vuelo del sistema integrado, con el nodo <i>edge</i> infiriendo y grabando a bordo, con enlace 4G y <i>store-and-forward</i> , con alimentación provisional del nodo <i>edge</i> mediante batería externa (Cap. 9). La integración definitiva de esa alimentación desde la batería principal, usando regulador UBEC, permanece como deuda técnica (11.1.4 y 12.3). |

En la Tabla 9.2 se resume qué subsistemas y funciones se evalúan a lo largo de las tres fases de validación. Al revisar la tabla por filas, se comprueba que todos los bloques quedan verificados. El vuelo conjunto de la plataforma y el nodo de procesamiento en el borde se puso en marcha al final, con una alimentación provisional del nodo *edge* mediante una batería externa (apartado 9.1). Lo único pendiente es integrar de forma definitiva esa alimentación desde la batería principal mediante el regulador UBEC, que sigue como tarea pendiente en los apartados 11.1.4 y 12.3

TABLA 9.2. COBERTURA DE SUBSISTEMAS SEGÚN LA FASE DE VALIDACIÓN EXPERIMENTAL.

| Subsistema / función                                                     | Fase SITL                            | Fase Semi-HITL (banco)        | Fase Vuelo real                                                                               |
|--------------------------------------------------------------------------|--------------------------------------|-------------------------------|-----------------------------------------------------------------------------------------------|
| Lógica de guiado (pymavlink, máquina de estados de misión)               | ✓                                    | ✓                             | ✓                                                                                             |
| Autopiloto y sensores inerciales (IMU, GNSS)                             | Simulado                             | ✓ (Pixhawk 6X)                | ✓ (Pixhawk 6X)                                                                                |
| Enlaces de radio directos (RC 2,4 GHz y telemetría SiK 433 MHz)          | —                                    | ✓                             | ✓                                                                                             |
| Enlace 4G/LTE, VPN Tailscale e infraestructura AWS                       | ✓ (telemetría simulada)              | ✓                             | ✓ parcial - vídeo en directo interrumpido > 10 m; datos íntegros por <i>store-and-forward</i> |
| Inferencia YOLO y cadena de visión (detección → alerta georreferenciada) | ✓ (vídeo pregrabado + posición SITL) | ✓ (tiempo real, NPU Hailo-8L) | ✓                                                                                             |
| Cadena de mando remota (panel PWA → API → MQTT → receptor → Pixhawk)     | ✓ (sin nodo <i>edge</i> )            | ✓ (extremo a extremo)         | ✓                                                                                             |
| Armado/desarmado y arranque de motores (respuesta de los ESC)            | ✓ (SITL)                             | ✓ (banco, sin hélices)        | ✓                                                                                             |
| Persistencia y resiliencia (store-and-forward, InfluxDB)                 | ✓                                    | ✓ (con corte de red forzado)  | ✓ (con recuperación de pérdida real de enlace 4G en vuelo)                                    |
| Sustentación, estabilidad y peso en vuelo                                | Simulado                             | —                             | ✓ (MTOW 1.817 g real)                                                                         |

Nota. La campaña de vuelo real se realizó en dos partes: una primera con la plataforma por sí sola (sustentación, estabilidad y peso) y una segunda con el sistema integrado. Se alimentó el nodo *edge* con una batería externa portátil porque no hubo tiempo para integrar el regulador UBEC (apartado 12.3). En esta segunda parte, se probaron en vuelo la inferencia embarcada, la grabación a bordo, el enlace 4G/LTE y el mecanismo de *store-and-forward*. Se presentaron las incidencias descritas en los apartados 9.3 y 9.4. El vídeo en directo se interrumpió por encima de ~10 m. Al reconectar, se recuperaron todos los datos. Solo queda pendiente integrar de forma definitiva la alimentación del nodo *edge* a partir de la batería principal.

**Alcance de la validación:** conviene aclarar el significado real de los resultados obtenidos. Primero, se ha comprobado por completo que cada subsistema funciona correctamente por separado. También se ha verificado cómo se integran al trabajar juntos en el banco. Además, se ha observado el comportamiento del sistema completo durante un vuelo real. Durante el vuelo, el sistema realiza la inferencia y la grabación a bordo. Al mismo tiempo, los datos se sincronizan con la nube. Para un prototipo cuyo objetivo es demostrar que la arquitectura propuesta es técnicamente viable, y no que el sistema ya esté listo para su uso, esta validación cubre todo lo definido en los objetivos del trabajo (apartado 1.2).

Sin embargo, quedan claramente pendientes tres temas que siguen de forma natural. El primero es la integración final de la alimentación del nodo *edge* desde la batería principal mediante el regulador UBEC, tal como se detalla en el apartado 12.3. El segundo es medir

el efecto de la oscilación de la cámara sobre la calidad de la detección y calcular el tamaño de un *gimbal* de dos ejes, que no se incluyó en el prototipo por falta de presupuesto desde el inicio, según el apartado 12.4. El tercero es analizar por qué se pierde el enlace de vídeo en directo a más de 10 metros de altura, también en el apartado 12.4. Ninguno de estos puntos afecta la validez de la arquitectura ni de los algoritmos utilizados, que ya se han demostrado por completo.

## 10. MARCO REGULADOR

### 10.1. Cumplimiento operativo de la normativa de la AESA o de la EASA.

El desarrollo y las pruebas en campo del sistema *Drone-SAR (Guardian Eye)* se rigen estrictamente por el marco legal vigente para aeronaves no tripuladas en España y en la Unión Europea. Este marco está regulado por el Reglamento de Ejecución (UE) 2019/947 [11] y el Real Decreto 517/2024 [14]. Como es un prototipo experimental montado a partir de componentes separados, con una masa total al despegue (MTOW) de 1.817 g, la aeronave no tiene marcado de clase CE (de C0 a C6). Por eso, se considera legalmente una aeronave de construcción privada.

Con esta condición, la operación en la **Categoría Abierta** se limita a la **subcategoría A3**. Esta clasificación obliga a mantener una distancia horizontal mínima de 150 metros respecto de zonas residenciales, comerciales, industriales o recreativas. También prohíbe estrictamente volar sobre personas que no participan. Para cumplir con todos los requisitos legales establecidos por la Agencia Estatal de Seguridad Aérea (AESA), durante el proyecto se realizaron las siguientes habilitaciones y registros.

- **Registro Oficial de Operador UAS.** Se tramitó ante la sede electrónica de AESA, con el número de registro oficial “ESPkpq4pb7xmj65p”, adjunto en el Anexo B. Se fijó la placa identificativa reglamentaria en el chasis de la aeronave.
- **Certificado de Competencia de Piloto a Distancia.** Superación de las pruebas teóricas oficiales emitidas por la AESA para las subcategorías A1 y A3 (Anexo C). Esto acredita la formación necesaria para manejar el vector aéreo de forma segura.
- **Identificación Remota Directa (Direct Remote-ID):** Se conecta el módulo transmisor autónomo Holybro DRI al puerto de telemetría de la Pixhawk 6X. Este módulo transmite de forma abierta y continua la posición GNSS, la altitud, la velocidad, el rumbo y el código del operador mediante Bluetooth o Wi-Fi, según el estándar europeo.

### 10.2. Procedimientos Operativos en el Espacio Aéreo (LECMUAV090)

La elección del lugar para las pruebas de vuelo en condiciones reales se basó en un análisis previo realizado con la herramienta oficial de *ENAIRE Drones*. Se descartaron las instalaciones del campus universitario de Leganés porque se encuentran dentro del espacio aéreo controlado de la Base Aérea de Getafe. Por eso, los ensayos se realizaron en el campo de vuelo del Club de Aeromodelismo Alas de Galapagar, en Madrid. Esta zona figura oficialmente en la Publicación de Información Aeronáutica (AIP España, sección ENR 5.5) [19] con el identificador LECMUAV090.

Realizar los ensayos en estas instalaciones simplifica la operativa, ya que no es necesario coordinarse con el control aéreo para cada vuelo. Además, el campo opera dentro de los 120 metros de altura sobre el terreno permitidos por la normativa, manteniendo una distancia segura respecto de las zonas urbanas y de las personas ajenas a las pruebas.

### 10.3. Cumplimiento del Reglamento de Inteligencia Artificial (AI Act) y Protección de Datos

Se revisó el uso del modelo YOLO11 en la NPU Hailo-8L conforme al Reglamento (UE) 2024/1689 (AI Act). El sistema se clasifica en la categoría de riesgo mínimo. No hace identificación biométrica remota. No reconoce identidades ni extrae patrones faciales. Solo detecta la clase general «persona» y crea una caja que la rodea. Funciona siempre bajo supervisión humana (Human-in-the-Loop). Su único objetivo es salvar vidas en emergencias.

En cuanto a la privacidad (RGPD y LOPD-GDD), se siguió estrictamente el principio de minimización de datos. Las grabaciones y capturas de fotogramas durante los ensayos de campo solo se realizaron con colaboradores informados que dieron su consentimiento expreso. Se eliminaron todos los archivos residuales después del entrenamiento y de la validación de las redes neuronales.

### 10.4. Vía regulatoria para una operación real de búsqueda y rescate

El sistema desarrollado en este trabajo se ha concebido y probado como un prototipo experimental. Las pruebas de vuelo descritas en el Capítulo 9 se han realizado dentro de los límites de la subcategoría A3 de la categoría abierta. Durante todas las pruebas se mantuvo el contacto visual directo (VLOS) y se operó en un campo de vuelo federado y registrado en la AIP de ENAIRE. Sin embargo, el entorno de una misión real de búsqueda y rescate es muy distinto. Requiere volar más allá del alcance visual del piloto (BVLOS), trabajar cerca de personas ajenas y operar en un espacio aéreo no separado. Estas condiciones no están permitidas en la categoría abierta normal. Por eso, es necesario definir el marco legal que permitiría a una plataforma como esta formar parte de un operativo de emergencias real.

Las actividades de vuelo relacionadas con los servicios de rescate y de protección civil quedan fuera del Reglamento (UE) 2019/947. Estas se conocen operativamente como «actividades no EASA». En España, este régimen se rige por los artículos 12 y 44 del Real Decreto 517/2024 [14] y cuenta con el respaldo de la Ley 17/2015 del Sistema Nacional de Protección Civil [17]. El artículo 44, en concreto, obliga a mantener una coordinación continua y en tiempo real con los proveedores de servicios de tránsito aéreo, siempre que la intervención afecte al espacio aéreo controlado, a las zonas de información de vuelo (FIZ) o a las áreas cercanas a aeródromos y helipuertos.

Para dar respuesta a estas situaciones críticas, el artículo 12 del Real Decreto 517/2024 contempla dos vías de actuación:

- **Modalidad directa.** El propio organismo público responsable de la intervención asume y ejecuta directamente la operación. Esto incluye los servicios de bomberos, las Fuerzas y Cuerpos de Seguridad del Estado o las agrupaciones oficiales de Protección Civil.
- **Modalidad indirecta (artículo 12.2):** La autoridad al mando requiere la intervención de un operador o piloto externo cualificado. En este caso, el operador

debe acreditar formalmente esta relación mediante un convenio de colaboración, un contrato de servicios o una solicitud expresa del mando de la emergencia.

Esta vía indirecta es la forma natural de transferir e incorporar un desarrollo tecnológico independiente, como el que se propone en este proyecto. Permite añadirlo a los dispositivos de auxilio gestionados por las administraciones públicas.

Cuando el espacio aéreo afectado está bajo la gestión de ENAIRE, el trámite operativo se realiza a través de su centro SYSRED H24 siguiendo el procedimiento para vuelos urgentes y en emergencias [35]. Este proceso requiere enviar, tanto por escrito como por teléfono, los datos clave de la misión. Hay que indicar la delimitación de la zona, el tipo de vuelo (VLOS o BVLOS), la masa y el número de aeronaves, así como la velocidad y la altura máxima previstas, que inicialmente se limitan a 120 metros sobre el terreno. También deben detallarse las capacidades de navegación y de detección del equipo a bordo. Como respuesta, ENAIRE comunica los requisitos técnicos y de seguridad necesarios para garantizar la convivencia con el tráfico aéreo tripulado. Antes de presentar esta solicitud, la normativa exige elaborar una Evaluación y Atenuación del Riesgo Operacional (EARO) ajustada a las características del entorno.

Buena parte de las exigencias técnicas que demanda este protocolo y la correspondiente evaluación de riesgos se encuentran ya cubiertas en el diseño del prototipo:

- Identificación remota directa (Remote ID).
- Enlace de control y telemetría en capas separadas.
- Consulta previa del espacio aéreo a través de la plataforma ENAIRE Drones.
- Registro oficial del operador ante la AESA.
- Certificado de piloto a distancia A1/A3.
- Póliza de seguro de responsabilidad civil vigente.

La formalización de los acuerdos institucionales con los cuerpos de auxilio y la redacción del estudio específico de riesgos operativos no están dentro del alcance técnico de este TFG. Se consideran requisitos indispensables antes de cualquier puesta en marcha en terreno, tal como se indica en la sección 12.2

## 11. ENTORNO SOCIO-ECONÓMICO Y OBJETIVOS ODS

### 11.1. Planificación temporal (Gantt, hitos, Kanban).

Siguiendo la metodología de trabajo descrita en la sección 1.3, el desarrollo de Drone-SAR (Guardian Eye) ha durado catorce semanas. Comenzó la primera semana de junio de 2026 y terminó con la entrega de la memoria el 7 de septiembre de 2026. Una sola persona ha realizado todo el proyecto. Además, reúne líneas de trabajo muy distintas entre sí. Por eso, la planificación no se ha organizado en una secuencia de etapas cerradas. Se ha planteado como un conjunto de líneas que han avanzado en paralelo y se han retroalimentado mutuamente. En esta sección se muestran las fases del proyecto, su cronograma en un diagrama de Gantt, los hitos alcanzados y la forma en que se han gestionado las desviaciones respecto de lo previsto.

#### 11.1.1. Fases del proyecto

El trabajo se ha dividido en diez fases técnicas y en dos líneas transversales: documentación y cierre. Las fases no siguen un orden estricto. Varias se han llevado a cabo simultáneamente. La Tabla 11.1 resume su contenido y el momento en que ocurrieron. El diagrama de Gantt de la sección 11.1.2 muestra en detalle los solapamientos.

TABLA 11.1. FASES DEL PROYECTO Y PERIODO APROXIMADO DE EJECUCIÓN.

| Fase                                              | Contenido principal                                                                                                                                                                                                                                                                                                  | Periodo aprox.        |
|---------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------|
| F0. Definición y estudio previo                   | Objetivos, alcance y requisitos; estado del arte; marco normativo; formación y habilitaciones (certificado de piloto A1/A3, registro como operador de UAS, alta en el Club Alas de Galapagar); selección preliminar de la plataforma.                                                                                | Junio (S1-S4)         |
| F1. Infraestructura cloud e IoT                   | Instancia AWS EC2, broker MQTT (Mosquitto), base de datos InfluxDB, dominio y DNS (Cloudflare), servidor web nginx con TLS, primeros repositorios en GitHub.                                                                                                                                                         | Junio-julio (S3-S7)   |
| F2. Control de vuelo y telemetría (SITL)          | Protocolo MAVLink y pymavlink, simulación SITL con ArduPilot y Mission Planner, guiado autónomo (armado → despegue → waypoints), migración del control desde el PC al nodo <i>edge</i> .                                                                                                                             | Junio-agosto (S4-S9)  |
| F3. Servicios web y cadena de mando remota        | API REST en Flask en EC2, puente de comandos por MQTT a la Raspberry Pi, panel de control web, subdominios y proxy inverso con certificado propio.                                                                                                                                                                   | Julio (S6-S10)        |
| F4. Adquisición de datos <i>edge</i> multidominio | Diseño en cuatro dominios independientes (ambiental, sistema, vuelo, detección), colectores, almacenamiento local ( <i>store-and-forward</i> ) en SQLite, ejecución autónoma con systemd, verificación multidrón.                                                                                                    | Julio-agosto (S7-S12) |
| F5. Visión artificial (YOLO11)                    | Validación preliminar del pipeline; construcción y curación del <i>dataset</i> propio; entrenamiento mediante <i>transfer learning</i> en GPU en la nube; evaluación con métricas; ampliación dirigida y reentrenamiento; comparativa de variantes y de motores de inferencia; despliegue en el acelerador Hailo-8L. | Julio-agosto (S7-S13) |
| F6. Construcción física del UAV                   | Análisis de componentes reutilizables de un hexacóptero cedido, decisión de compra y reserva del kit, ensamblaje del chasis y de la Pixhawk 6X, integración de radios y del nodo <i>edge</i> , calibración inicial.                                                                                                  | Julio-agosto (S6-S12) |



|                                            |                                                                                                                                                                                                                                                    |                                          |
|--------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------|
| F7. Comunicaciones celulares y red privada | Enlace 4G/LTE, red privada virtual mallada (Tailscale/WireGuard), enrutado con salida primaria por 4G y arranque autónomo del módem.                                                                                                               | Agosto (S10-S12)                         |
| F8. Transmisión de vídeo en directo        | Servidor MediaMTX en la nube, publicación de RTSP desde la Raspberry Pi a través de la VPN, retransmisión WebRTC de baja latencia, integración en el panel PWA, grabación por sesiones y optimización del bucle de captura.                        | Agosto (S12-S13)                         |
| F9. Validación experimental                | Simulación SITL, banco de integración de hardware en tierra (Semi-HITL, sin hélices), campaña de vuelo real en el espacio autorizado LECMUAV090 y matriz de cumplimiento de objetivos. La metodología de los ensayos se detalla en la sección 9.1. | Finales de agosto - septiembre (S12-S14) |
| Transversal. Documentación y seguimiento   | Diario público en tiempo real, sitios web del proyecto (página de micromecenazgo y documentación técnica), redacción de la memoria capítulo a capítulo, revisiones periódicas con el tutor.                                                        | Todo el proyecto (S1-S14)                |
| Cierre                                     | Conclusiones y trabajo futuro, entorno socio-económico y presupuesto, maqueta, bibliografía en formato IEEE y anexos (glosario, certificados de AESA), revisión global y entrega.                                                                  | Septiembre (S13-S14)                     |

La infraestructura en la nube e IoT (F1) se preparó con anticipación y de forma independiente del vehículo. Así, buena parte de la validación de las comunicaciones, la mensajería MQTT y la persistencia de datos pudo realizarse mediante telemetría simulada mucho antes de contar con el dron real. Al mismo tiempo, el control de vuelo (F2) se desarrolló por completo en el simulador SITL de ArduPilot. La máquina de estados de navegación, los scripts de guiado con pymavlink y la migración del control desde el PC al nodo *edge* se probaron y mejoraron en simulación. Esto permitió pasar a las pruebas con hardware real con el software ya estable.

La creación del *dataset* (parte de F5) fue un trabajo continuo, no una etapa puntual. Las primeras grabaciones aéreas se realizaron tan pronto como se tuvo acceso a una plataforma de vuelo. Los diferentes vídeos han sido extraídos tanto mediante grabaciones propias en el club como a partir de material audiovisual de libre uso. El conjunto se fue etiquetando y ampliando poco a poco en Roboflow durante julio y agosto, al mismo tiempo que el resto del trabajo. Así se llegó a la composición final descrita en el capítulo 8.

La construcción física del UAV (F6) comenzó en etapas tempranas. Primero, se analizó qué piezas se podían reutilizar de un hexacóptero prestado por un miembro del club y se definió la lista de compra. Sin embargo, terminar el montaje dependía de que las piezas llegaran poco a poco y de la disponibilidad del kit de vuelo, que estuvo agotado durante parte del tiempo. Para manejar esta incertidumbre, se avanzó en todo lo que no requería el hardware completo. El montaje detallado quedó en segundo plano y se retomaba cada vez que llegaba algún componente. Así, nunca se convirtió en un obstáculo para el resto del proyecto.

La transmisión de vídeo en directo (F8) se trató de forma intencional al final del proceso. Aunque la base de programación del detector y la infraestructura en la nube estaban listas semanas antes, el streaming se dejó para el final porque dependía de tener listo tanto el

pipeline de detección como la red privada sobre 4G. Además, era la función con mayor margen de recorte si faltaba tiempo. Una vez integrados el servidor MediaMTX, la publicación RTSP desde la Raspberry Pi a través de la VPN y la retransmisión WebRTC de baja latencia, se dedicó un esfuerzo específico a optimizarlos para que se ajustaran al presupuesto de cómputo del nodo *edge*. La comparación de motores de inferencia y la elección de NCNN frente a otras opciones, junto con ajustes en el bucle de captura, liberaron suficiente CPU para mantener a la vez la inferencia, la grabación y el envío de vídeo.

### 11.1.2. Diagrama de Gantt

La Figura 11.1 recoge el cronograma del proyecto en forma de diagrama de Gantt, con las tareas en filas y las catorce semanas de desarrollo en columnas (S1 = primera semana de junio de 2026; S14 = semana de la entrega, el 7 de septiembre). Las barras oscuras marcan tareas con inicio y final definidos. Las barras claras indican actividades continuas o transversales que se mantuvieron durante buena parte del proyecto, como la construcción del *dataset* y la redacción de la memoria. Se puede ver con claridad el solapamiento entre las líneas de trabajo descritas en los apartados anteriores.

| Tarea                                                                         | JUNIO |    |    |    | JULIO |    |    |    |    | AGOSTO |     |     |     | SE<br>P. |
|-------------------------------------------------------------------------------|-------|----|----|----|-------|----|----|----|----|--------|-----|-----|-----|----------|
|                                                                               | S1    | S2 | S3 | S4 | S5    | S6 | S7 | S8 | S9 | S10    | S11 | S12 | S13 | S14      |
| A. Definición de objetivos, alcance y requisitos                              |       |    |    |    |       |    |    |    |    |        |     |     |     |          |
| B. Estado del arte                                                            |       |    |    |    |       |    |    |    |    |        |     |     |     |          |
| C. Marco normativo y regulatorio                                              |       |    |    |    |       |    |    |    |    |        |     |     |     |          |
| D. Formación y habilitaciones (piloto A1/A3, operador, club)                  |       |    |    |    |       |    |    |    |    |        |     |     |     |          |
| E. Selección de plataforma y componentes                                      |       |    |    |    |       |    |    |    |    |        |     |     |     |          |
| F. Infraestructura cloud e IoT base (EC2, MQTT, InfluxDB, TLS)                |       |    |    |    |       |    |    |    |    |        |     |     |     |          |
| G. Control de vuelo y telemetría en SITL (pymavlink)                          |       |    |    |    |       |    |    |    |    |        |     |     |     |          |
| H. Servicios web y cadena de mando remota (API REST, panel)                   |       |    |    |    |       |    |    |    |    |        |     |     |     |          |
| I. Adquisición de datos <i>edge</i> multidominio ( <i>store-and-forward</i> ) |       |    |    |    |       |    |    |    |    |        |     |     |     |          |
| J. Construcción del <i>dataset</i> : campañas de vuelo (fin de semana)        |       |    |    |    |       |    |    |    |    |        |     |     |     |          |
| K. Entrenamiento, evaluación y optimización del modelo YOLO11                 |       |    |    |    |       |    |    |    |    |        |     |     |     |          |
| L. Construcción e integración física del UAV (piezas escalonadas)             |       |    |    |    |       |    |    |    |    |        |     |     |     |          |
| M. Comunicaciones celulares y red privada (4G/LTE, Tailscale)                 |       |    |    |    |       |    |    |    |    |        |     |     |     |          |
| N. Transmisión de vídeo en directo y PWA (MediaMTX, WebRTC)                   |       |    |    |    |       |    |    |    |    |        |     |     |     |          |
| O. Validación experimental (SITL, banco Semi-HITL, vuelo real)                |       |    |    |    |       |    |    |    |    |        |     |     |     |          |
| P. Redacción de la memoria y documentación (diario, webs)                     |       |    |    |    |       |    |    |    |    |        |     |     |     |          |
| Q. Cierre: conclusiones, presupuesto, maquetación y entrega                   |       |    |    |    |       |    |    |    |    |        |     |     |     |          |

Figura 11.1. Diagrama de Gantt del proyecto (junio–septiembre de 2026). Barra oscura: tarea acotada. Barra clara: actividad continua o transversal.

### 11.1.3. Hitos principales

La Tabla 11.2 recoge los hitos más relevantes alcanzados durante el desarrollo, tomados del diario de seguimiento del proyecto.

TABLA 11.2. HITOS RELEVANTES ALCANZADOS DURANTE EL DESARROLLO

| Fecha aproximada                                         | Hito                                                                                                                                                                     |
|----------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1. <sup>a</sup> semana de junio                          | Apertura del diario público de seguimiento y del tablero Kanban del proyecto.                                                                                            |
| Mediados de junio                                        | Certificado de piloto A1/A3 y registro como operador de UAS obtenidos; alta como socio del Club Alas de Galapagar (seguro y acceso al campo LECMUAV090).                 |
| Última semana de junio                                   | Infraestructura operativa en la nube: pipeline de sensores → MQTT → InfluxDB, con almacenamiento de tipo <i>store-and-forward</i> y arranque autónomo mediante systemd.  |
| 2. <sup>a</sup> semana de julio                          | Primera campaña de vuelo de campo y primeras grabaciones aéreas del <i>dataset</i> . Ciclo completo de misión autónoma validado en SITL (armado → despegue → waypoints). |
| 3. <sup>a</sup> semana de julio                          | Cadena de mando remota completa validada de extremo a extremo: navegador → API REST → MQTT → receptor en la Raspberry Pi → MAVLink.                                      |
| 1. <sup>a</sup> semana de agosto                         | Pipeline de visión validado de extremo a extremo en un caso preliminar; confirmación y reserva del kit de vuelo.                                                         |
| 2. <sup>a</sup> semana de agosto                         | Arquitectura de adquisición de datos multidominio (cuatro dominios independientes) implementada y verificada en modo multidron.                                          |
| 3. <sup>a</sup> semana de agosto                         | Enlace 4G/LTE con red privada mallada (Tailscale) operativo. UAV completamente construido y listo para las pruebas de campo.                                             |
| 4. <sup>a</sup> semana de agosto                         | Cierre de la fase de programación; transmisión de vídeo en directo (WebRTC) en funcionamiento e integrada en el panel PWA.                                               |
| Finales de agosto - 1. <sup>a</sup> semana de septiembre | Validación experimental: simulación SITL, banco de integración Semi-HITL en tierra y campaña de vuelo real en LECMUAV090 (véase el capítulo 9).                          |

### 11.1.4. Seguimiento del proyecto y gestión de desviaciones

El seguimiento constante mediante el tablero Kanban y el diario permitió detectar a tiempo las desviaciones respecto de lo planificado. Esto permitió reaccionar sin comprometer la fecha de entrega. La columna «En pruebas / Bloqueada» del tablero fue muy útil para mostrar las dependencias externas y los problemas que detenían una tarea concreta, sin frenar el resto del proyecto (Figura 11.2). Las principales incidencias y decisiones de replanificación fueron:

- Los componentes llegaron poco a poco y la disponibilidad del kit de vuelo fue limitada. El kit Holybro X500 V2 estuvo agotado durante parte del periodo. Esto obligó a buscar una alternativa con componentes sueltos y a reservar el kit con antelación. Por eso, la construcción del vehículo se planeó como una tarea de fondo sin una fecha crítica en medio. Se priorizó el trabajo que no requería el hardware

completo. Un segundo caso de esta misma dependencia fue el regulador UBEC necesario para alimentar el nodo *edge* a partir de la batería principal del dron. Este regulador llegó a pocos días del cierre del trabajo, sin margen para su soldadura ni su integración. Como solución provisional, se alimentó la Raspberry Pi con una batería externa portátil (power bank) independiente. La campaña de vuelo real se realizó sin el computador de a bordo. La cadena Raspberry Pi-Pixhawk ya se había validado en el banco de pruebas en tierra, en el apartado 9.1. La integración definitiva de la alimentación queda recogida como deuda técnica en los apartados 12.2 y 12.3.

- Corrupción de la tarjeta microSD de la Raspberry Pi. Un apagado repentino durante las pruebas dejó ilegible la tarjeta del nodo *edge*. Gracias al uso sistemático del control de versiones, la pérdida se limitó a un único script. Después del problema, se mejoraron los procedimientos de apagado ordenado y de arranque automático mediante servicios de systemd.
- Rediseño de la arquitectura de adquisición de datos. Se descartó la idea inicial de un único proceso con un planificador cooperativo debido al riesgo de concurrencia. En su lugar, se usaron cuatro procesos independientes, uno para cada dominio.
- Coste de cómputo del entrenamiento y de la evolución del modelo. La prueba preliminar mostró que entrenar en local sobre CPU no era posible debido al tiempo que requería. Tardó 2 horas y 40 minutos con un conjunto pequeño. Por eso, el entrenamiento del modelo final se realizó en una infraestructura con GPU en la nube. De la misma forma, la primera evaluación del modelo mostró que faltaban ejemplos en algunas posturas. Esto llevó a ampliar de manera dirigida el *dataset* y a volver a entrenarlo.
- Incidencias de integración menores. Hubo conflictos de puertos en el simulador. Se presentó un bloqueo de contenido mixto entre el panel web (HTTPS) y la API (HTTP). Esto se resolvió creando un subdominio con un proxy inverso y un certificado propio.

La Figura 11.2 muestra un ejemplo del tablero Kanban en una fase intermedia del desarrollo.

| Pendiente                                                                                                                                                                                                                                                          | En curso                                                                                                                                                                         | En pruebas / Bloqueada                                                                                                                                                                                                                                                             | Hecho                                                                                                                                                                                                                                                                                                                                                      |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"> <li>▶ Ampliar el <i>dataset</i> con posturas tumbado/sentado - IA</li> <li>▶ Compilar el modelo a formato HEF (requiere Linux x86-64) - IA</li> <li>▶ Script de conmutación de cobertura 4G ↔ Wi-Fi - Comunicaciones</li> </ul> | <ul style="list-style-type: none"> <li>▶ Integrar la controladora Pixhawk 6X en el chasis - Hardware</li> <li>▶ Redacción del capítulo 6 (arquitectura IoT) - Memoria</li> </ul> | <ul style="list-style-type: none"> <li>▶ Panel web: fallo de armado por contenido mixto HTTPS→HTTP - Web bloqueada hasta crear el subdominio api.</li> <li>▶ Recuperar la tarjeta microSD corrupta del nodo edge - Sistema bloqueada; se abre la tarea de reinstalación</li> </ul> | <ul style="list-style-type: none"> <li>▶ Despliegue de EC2 + broker MQTT + InfluxDB - Sistema y nube</li> <li>▶ Pipeline sensor → MQTT → InfluxDB con <i>store-and-forward</i> - Sistema y nube</li> <li>▶ Guiado autónomo en SITL: armado → despegue → waypoints - Vuelo</li> <li>▶ Detector preliminar (caso fruta) entrenado y validado - IA</li> </ul> |

Figura 11.2. Ejemplo de tablero Kanban para el seguimiento de tareas en una fase intermedia del proyecto. Cada tarjeta lleva una etiqueta de área.

En conjunto, la combinación de una planificación con líneas de trabajo paralelas, un seguimiento diario y una funcionalidad final —la transmisión de vídeo— con margen de ajuste permitió absorber estas desviaciones y cumplir todos los objetivos previstos dentro del plazo. La dedicación total estimada, coherente con las aproximadamente 300 horas de ingeniería contabilizadas en el presupuesto (sección 11.2), se concentró especialmente en julio y agosto.

## 11.2. Presupuesto del TFG

La evaluación económica del proyecto incluye la inversión en componentes de hardware instalados en el sistema, el equipo de apoyo en tierra, los servicios en la nube y las horas de ingeniería dedicadas durante las 300 horas oficiales del Trabajo de Fin de Grado (12 ECTS).

En cuanto a la plataforma UAS, el presupuesto incluye el costo total de reponer todos los componentes necesarios para su montaje e integración. Aunque durante el desarrollo ya se contaba con algunos equipos, como la Raspberry Pi 5 o el dron de apoyo, y otros elementos se consiguieron gracias a donaciones y préstamos de colaboradores (es importante destacar que se consiguió gran parte del material a través de una página web de crowdfunding y micromecenazgo del proyecto: <https://drone-sar.gorostiditfg.com/>), la tabla de presupuesto muestra el valor de mercado total. Así, el balance económico muestra con claridad y honestidad el costo real de reproducir la solución diseñada desde cero.

## A. Costos de hardware e instrumentos físicos

TABLA 11.3. PRESUPUESTO DESGLOSADO DE COMPONENTES HARDWARE E INSTRUMENTAL

| Componente                                                                           | Cantidad | Precio unitario (€) | Importe (€)       |
|--------------------------------------------------------------------------------------|----------|---------------------|-------------------|
| Kit Holybro X500 V2 (chasis fibra de carbono, motores 2216, ESCs, hélices 1045, PDB) | 1,00 €   | 445,00 €            | 445,00 €          |
| Autopiloto Holybro Pixhawk 6X + Power Module                                         | 1,00 €   | 285,00 €            | 285,00 €          |
| Módulo GNSS/Brújula Holybro M10 + Mástil                                             | 1,00 €   | 72,00 €             | 72,00 €           |
| Pareja de radios de telemetría Holybro SiK 433 MHz                                   | 1,00 €   | 58,00 €             | 58,00 €           |
| Emisora FlySky FS-i6X + Receptor FS-iA10B (2,4 GHz)                                  | 1,00 €   | 64,50 €             | 64,50 €           |
| Computadora de a bordo Raspberry Pi 5 (4 GB) + Cooler activo                         | 1,00 €   | 85,00 €             | 85,00 €           |
| Acelerador NPU Raspberry Pi AI HAT+ (Hailo-8L, 13 TOPS)                              | 1,00 €   | 78,00 €             | 78,00 €           |
| Cámara Raspberry Pi Camera Module 3 (CSI) + Sensor BME680 (I2C)                      | 1,00 €   | 46,00 €             | 46,00 €           |
| Módem USB 4G/LTE Huawei + Tarjeta SIM datos M2M                                      | 1,00 €   | 38,00 €             | 38,00 €           |
| Baterías LiPo 4S 14,8 V 5000 mAh (2 unidades) + Cargador balanceador                 | 1,00 €   | 165,00 €            | 165,00 €          |
| Módulo Holybro Direct Remote-ID + Cableado y conectores XT60/JST                     | 1,00 €   | 52,00 €             | 52,00 €           |
| <b>Subtotal Hardware e Instrumental</b>                                              |          |                     | <b>1.388,50 €</b> |

## B. Servicios Cloud, Dominio y Plataformas

TABLA 11.4. PRESUPUESTO DE SERVICIOS CLOUD, GESTIÓN DE DOMINIO Y PLATAFORMAS DE CÓMPUTO.

| Concepto                                                                                                                     | Periodo                 | Coste (€)       |
|------------------------------------------------------------------------------------------------------------------------------|-------------------------|-----------------|
| Instancia AWS EC2 t3.small (Ubuntu Server, región eu-south-2 España)                                                         | 6 meses                 | 84,00 €         |
| Registro de dominio gorostiditfg.com (Cloudflare DNS)                                                                        | 1 año                   | 12,50 €         |
| Roboflow (etiquetado, aumento y exportación del <i>dataset</i> ); cuenta académica gratuita para estudiantes y universidades | Uso durante el proyecto | 0,00 €          |
| Google Colab Pro (cómputo en GPU NVIDIA Tesla T4)                                                                            | 3 meses                 | 33,00 €         |
| Cuota de socio Club Alas de Galapagar (espacio LECMUAV090)                                                                   | Temporada 2026          | 60,00 €         |
| Licencia federativa anual (Federación Aérea de Madrid), que incluye el seguro de responsabilidad civil obligatorio del UAV   | Temporada 2026          | 40,00 €         |
| <b>Subtotal Servicios y Licencias</b>                                                                                        |                         | <b>229,50 €</b> |

El ordenador de desarrollo y las herramientas de taller no se han adquirido para este proyecto. Tienen una vida útil superior a la del proyecto. Por eso, se calculan mediante amortización lineal.

$$\text{coste imputable} = \text{coste de adquisición} \times \left( \frac{\text{meses de uso}}{\text{vida útil en meses}} \right) \times \text{porcentaje de dedicación}$$

TABLA 11.5. AMORTIZACIÓN DEL EQUIPO DE USO GENERAL

| Equipo                                                                   | Coste adq. (€) | Vida útil | Uso   | Dedic. | Imputable (€)  |
|--------------------------------------------------------------------------|----------------|-----------|-------|--------|----------------|
| Ordenador portátil de desarrollo                                         | 1.200,00 €     | 60 m      | 3,5 m | 100 %  | 70,00 €        |
| Estación de soldadura e instrumental de taller (polímetro, herramientas) | 150,00 €       | 60 m      | 3,5 m | 100 %  | 8,75 €         |
| Báscula de precisión y pequeño material de medida                        | 30,00 €        | 60 m      | 3,5 m | 100 %  | 1,75 €         |
| <b>Subtotal amortización</b>                                             |                |           |       |        | <b>80,50 €</b> |

### C. Recursos Humanos y Presupuesto Consolidado

Se contabilizan 300 horas de trabajo como ingeniera graduada en tecnologías de telecomunicaciones, a un coste medio de mercado de **25,00 €/hora**. Esto supone un coste laboral directo de **7.500,00 €**.

TABLA 11.6. PRESUPUESTO CONSOLIDADO Y COSTE TOTAL DE EJECUCIÓN DEL PROYECTO.

| Concepto                                                     | Importe (€)        |
|--------------------------------------------------------------|--------------------|
| Coste de personal (300 h × 25 €/h)                           | 7.500,00 €         |
| Material: componentes del UAV (Tabla 11.3)                   | 1.388,50 €         |
| Material: amortización de equipo de uso general (Tabla 11.5) | 80,50 €            |
| Servicios, conectividad y licencias (Tabla 11.4)             | 229,50 €           |
| <b>Costes directos</b>                                       | <b>9.198,50 €</b>  |
| Costes indirectos (8 %)                                      | 735,88 €           |
| <b>Presupuesto de ejecución material (sin IVA)</b>           | <b>9.934,38 €</b>  |
| IVA (21 %)                                                   | 2.086,22 €         |
| <b>Presupuesto total de ejecución del proyecto</b>           | <b>12.020,60 €</b> |

El presupuesto anterior cubre el desarrollo completo del sistema. Hacer una unidad extra, cuando ya se tiene el diseño, la documentación de montaje, el estudio y el código, cuesta mucho menos. La mayor parte del trabajo original, como el análisis, la arquitectura, el desarrollo de software y la validación, no se repite.

TABLA 11.7. COSTO ESTIMADO PARA REPLICAR UNA UNIDAD ADICIONAL DEL SISTEMA.

| Concepto                                                                                                           | Importe aprox. (€) |
|--------------------------------------------------------------------------------------------------------------------|--------------------|
| Componentes del UAV (reutilizando la emisora de radiocontrol y el cargador de baterías de una unidad ya existente) | 1.300 €            |
| Montaje, cableado, configuración, calibración y prueba en banco (una jornada, ≈ 8 h × 25 €/h)                      | 200 €              |
| Tarjeta SIM M2M adicional                                                                                          | 15 €               |
| Coste de una unidad adicional (material y montaje)                                                                 | ≈ 1.500 €          |

Este coste no incluye el entrenamiento del modelo de visión. Tampoco cubre la infraestructura en la nube, como la instancia EC2, el dominio, el broker MQTT ni la red de Tailscale. No incluye la certificación del operador, el ordenador ni el instrumental de taller. Estos son costes únicos ya pagados o que se comparten entre todas las unidades de una posible flota. Un segundo vehículo se suma a la infraestructura existente sin un coste añadido significativo.

### 11.3. Impacto Socio-Económico

En una emergencia de rescate, el tiempo es el factor determinante para la supervivencia: localizar a una persona en las primeras horas define la probabilidad de encontrarla con vida. El sistema desarrollado aborda esta problemática mediante el rastreo autónomo de áreas extensas y la generación inmediata de avisos ante la detección de posibles víctimas. A continuación, se evalúa el impacto práctico de la puesta en servicio de esta plataforma, considerando sus beneficios operativos directos, así como los costes y los retos de su despliegue.

#### Beneficios esperados

- **Seguridad del personal de emergencias:** La aeronave permite explorar barrancos, laderas inestables o zonas boscosas densas sin exponer a los rescatadores a riesgos innecesarios, reservando la intervención física exclusivamente para las maniobras de atención médica y de evacuación de la víctima.
- **Eficiencia frente a medios aéreos tripulados:** La operación de un helicóptero de rescate supone un coste estimado de entre 2.500 y 3.000 € por hora de vuelo [36]. Frente a este gasto, el coste de operación del UAV resulta prácticamente marginal. El sistema no persigue reemplazar a las aeronaves tripuladas, sino reducir las horas de búsqueda a ciegas y concentrar su intervención en las fases en las que resulta estrictamente indispensable.
- **Monitorización ambiental en el puesto de mando:** La integración a bordo del sensor BME680 permite registrar en tiempo real las magnitudes de temperatura, humedad, presión y concentración de gases. La transmisión continua de estas medidas a la nube proporciona al puesto de mando una lectura directa de las condiciones del escenario antes de ordenar el despliegue del personal, lo cual resulta especialmente relevante ante conatos de incendio forestal o embolsamientos de humo.

#### Costes y retos operativos

- **Privacidad y percepción social:** El uso de cámaras aéreas en operaciones de vuelo genera preocupación entre la ciudadanía por la protección de datos. Para que la sociedad comprenda y acepte su uso, los vuelos deben limitarse únicamente a rescates reales. No deben conservar grabaciones innecesarias y deben operar siempre bajo el control directo de un piloto acreditado (apartado 3.4).
- **Compromiso en la detección:** Mientras que un falso positivo solo ocasiona unos segundos de verificación visual por parte del operador en tierra, un falso negativo supone dejar desatendida a una víctima real. Por este motivo, el diseño del sistema prioriza el parámetro de exhaustividad (*recall*), asumiendo un filtrado humano complementario de las alertas generadas.
- **Mantenimiento y capacitación técnica:** No se puede descuidar el funcionamiento del sistema. Se necesita personal con titulación de piloto a



distancia A1/A3. Es necesario seguir un protocolo estricto para cargar y conservar las baterías de polímero de litio (LiPo). También deben estar disponibles repuestos. El sistema debe verse como una herramienta de apoyo que forme parte de los protocolos habituales de bomberos o de Protección Civil. No debe considerarse un recurso aislado.

- **Gestión de residuos electrónicos:** Si bien a lo largo de su vida útil se evitan emisiones procedentes de combustibles de aviación, la plataforma incorpora acumuladores de litio y circuitos impresos que exigen una gestión y un reciclaje regulados al concluir su ciclo operativo.

### **Plan de explotación y balance**

- **Difusión y código abierto:** La totalidad de la arquitectura, los esquemas eléctricos y el código desarrollado se publican en abierto (Anexo E) para facilitar la auditoría técnica y la reproducibilidad del prototipo por parte de terceros.
- **Transferencia a servicios de emergencia:** Con un coste de materiales cercano a 1.500 € por unidad adicional, la plataforma es una opción viable para agrupaciones locales de Protección Civil o brigadas comarcales. Estas pueden aprovechar la infraestructura cloud y el modelo neuronal sin pagar licencias comerciales propietarias.
- **Extensión a otros ámbitos:** La arquitectura del sistema, basada en la inferencia en el borde mediante la Raspberry Pi 5 y en la transmisión ligera de eventos a través de la red celular, no se limita exclusivamente al entorno SAR. Este mismo esquema puede utilizarse directamente en escenarios como la monitorización de explotaciones agrícolas, la vigilancia perimetral de instalaciones industriales o el apoyo a los servicios de salvamento en zonas costeras.

En conclusión, comprar el sistema solo para intervenciones puntuales no compensa los costes de integración ni el tiempo de formación del personal. Por el contrario, cuando el UAV se utiliza de manera regular en la operación de un servicio de emergencias, el ahorro en horas de vuelo de helicóptero y la disminución del riesgo para los rescatadores justifican la inversión desde las primeras fases de su uso.

### **11.4. Objetivos ODS**

El proyecto contribuye a la Agenda 2030 de las Naciones Unidas como parte central de su misión operativa, no como algo adicional. Un sistema pensado para encontrar personas en peligro de forma más rápida, segura y fácil impacta directamente en varios Objetivos de Desarrollo Sostenible (ODS). El análisis muestra tres aportes principales y dos secundarios.

#### **Contribuciones principales**

- **ODS 3: Salud y bienestar.** Metas 3.d (reforzar la capacidad de alerta temprana y de respuesta rápida ante emergencias) y 3.6 (reducir las muertes por accidentes). En una operación SAR, la probabilidad de encontrar con vida a una persona desaparecida es máxima en las primeras horas: cuanto antes se localice, mayores

son sus probabilidades de supervivencia. El sistema aborda ese factor y detecta a las personas a bordo mediante YOLO11 en el acelerador Hailo-8L. Alcanza los 29,49 FPS, prácticamente en tiempo real respecto al ritmo de captura de la cámara. Esto evita descargar vídeo de forma continua y no requiere cobertura de red externa en el área de vuelo. En estas zonas se produce la mayoría de los rescates en montaña. El sistema envía alertas georreferenciadas directamente al puesto de mando. Un ejemplo del nivel de mejora es el rescate del Monviso (Alpes italianos, julio de 2025) [5]. Un software de inteligencia artificial procesó en una sola tarde 2.600 imágenes que un equipo humano habría tardado semanas en revisar. Este proyecto lleva a cabo ese análisis a bordo, en tiempo real durante el propio vuelo.

- **ODS 9: Industria, innovación e infraestructura.** Metas 9.1 (desarrollo de infraestructuras fiables y resilientes) y 9.5 (fomento de la investigación y la capacidad tecnológica). El trabajo reúne tres áreas de la ingeniería: robótica aérea con ArduPilot, aceleración de la inteligencia artificial en el borde con Raspberry Pi 5 y comunicaciones celulares. Todo esto se organiza en una arquitectura abierta, documentada y fácil de reproducir, en la que el código y los esquemas de montaje se publican en abierto (Anexo E). Con un coste de materiales cercano a 1.500 € por unidad adicional (apartado 11.1), se ofrece una capacidad avanzada a agrupaciones locales de Protección Civil o aeroclubes, sin depender de plataformas comerciales cerradas. Además, el diseño de comunicaciones redundantes (control por RF a 2,4 GHz, telemetría directa y canal 4G/LTE con VPN) ofrece una infraestructura resiliente pensada para operar en zonas con cobertura degradada.
- **ODS 11: Ciudades y comunidades sostenibles.** Metas 11.5 (reducir muertes y pérdidas económicas causadas por desastres) y 11.b (gestión integral del riesgo de desastres según el Marco de Sendai). El sistema ofrece una herramienta fácil de usar para apoyar la respuesta ante incendios forestales, inundaciones o derrumbes. La aeronave puede vigilar por sí sola las zonas afectadas. El sensor ambiental BME680 detecta con rapidez gases y cambios de temperatura asociados al inicio de un incendio (apartado 6.3.1). Además, el sistema de almacenamiento y reenvío en SQLite almacena los datos de vuelo y las mediciones, aunque las redes móviles fallen en el área afectada. La jerarquía de *topics* de MQTT permite escalar el despliegue a flotas coordinadas en cuadrículas grandes.

### Contribuciones secundarias

- **ODS 4: Educación de calidad.** Meta 4.4, incrementar las competencias técnicas y profesionales. La documentación abierta del proyecto, los esquemas de integración y los repositorios son recursos educativos de acceso libre para aprender sobre sistemas aéreos no tripulados, el procesamiento en el borde y los protocolos de IoT.
- **ODS 13: Acción por el clima.** Meta 13.1, reforzar la resiliencia frente a los riesgos climáticos. Detectar a tiempo los focos de incendio forestal mediante

visión artificial y monitoreo ambiental mejora la respuesta ante los efectos del cambio climático.

## 12. CONCLUSIONES Y TRABAJO FUTURO

### 12.1. Conclusiones

En este Trabajo de Fin de Grado se ha diseñado, ensamblado, programado y probado en experimentos un sistema UAV autónomo completo, pensado para misiones de Búsqueda y Rescate, lo que demuestra que es posible integrar percepción visual acelerada por hardware, comunicaciones celulares de largo alcance y arquitecturas IoT distribuidas en una plataforma física multirrotor.

Las principales conclusiones técnicas alcanzadas se resumen en:

1. **Viabilidad de la Inferencia Edge-AI Embarcada:** Se supera así la limitación clásica de los sistemas SAR basados en el volcado posterior de imágenes. La integración del acelerador Hailo-8L en la Raspberry Pi 5 permitió ejecutar el modelo YOLO11n a 29,49 FPS, con una latencia media de 33,91 ms por fotograma y un uso de CPU del 175,4 % (1,8 núcleos), frente a los 8,35 FPS y el mayor consumo de CPU (348,2 %, 3,5 núcleos) registrados mediante procesamiento exclusivo en CPU con NCNN. El detector alcanzado es una prueba de concepto (*recall* 0,601): valida el pipeline completo (*dataset*, entrenamiento, compilación e inferencia acelerada), pero un despliegue operativo exigiría ampliar el conjunto de datos hasta lograr un *recall* muy superior. En todo caso, la detección se plantea como asistencia al operador humano, no como sustituto.
2. **Robustez de la Arquitectura Distribuida e IoT:** El diseño modular en cuatro dominios desacoplados (Ambiental, Sistema, Vuelo y Detección), con persistencia local (*store-and-forward* en SQLite), demuestra su capacidad de operar sin pérdida de datos ante caídas y reconexiones de la red 4G/LTE, preservando la integridad de las marcas de tiempo en InfluxDB. Este comportamiento se ha confirmado en vuelo real ante una pérdida no forzada del enlace 4G: la grabación local y los datos de los cuatro dominios se conservaron y se sincronizaron íntegros al recuperar la conexión (apartado 9.3).
3. **Jerarquía de Comunicaciones y Mando Dual:** La coexistencia de cuatro canales independientes (RC 2,4 GHz directo para emergencias VLOS, radio SiK 433 MHz local, VPN WireGuard/Tailscale sobre 4G celular y streaming WebRTC de baja latencia) garantiza tanto el cumplimiento de las exigencias de seguridad de AESA como la operatividad remota de la aeronave a través de una interfaz web PWA.
4. **Metodología de Validación Gradual y Cumplimiento:** La estrategia escalonada (simulación en SITL con pymavlink, integración del hardware en banco en tierra y vuelo real de sustentación en el espacio autorizado LECMUAV090 de Galapagar). La cadena de visión completa (captura, inferencia acelerada, alerta y grabación) se ha ejercitado, además, en un vuelo real, con detección efectiva de las personas presentes en el campo de visión y con las limitaciones de estabilidad de imagen propias de una cámara sin estabilización mecánica (apartado 9.4).

## 12.2. Lecciones Aprendidas

Desarrollar un sistema que abarca desde el chasis físico hasta la nube y la inteligencia artificial en un plazo de 300 horas (los 12 ECTS del TFG, desglosados en el apartado 11.1) exigió priorizar de forma continua y resolver imprevistos sobre la marcha. Enfrentarse a los problemas reales de la integración de hardware y software aportó un aprendizaje práctico que no se adquiere en simulaciones puramente teóricas. A continuación, se sintetizan las lecciones aprendidas más determinantes para el desarrollo de proyectos con una arquitectura similar:

- **Apagado ordenado del ordenador de a bordo.** Un corte de alimentación no controlado de la Raspberry Pi durante las pruebas corrompió su tarjeta de almacenamiento y obligó a reinstalar el sistema (véase la sección 11.1.4). De esta incidencia se derivaron dos prácticas que se incorporaron al proyecto: reducir la frecuencia de escritura en el almacenamiento persistente (se implementó un parámetro para cambiar la frecuencia de recogida de datos y su posterior escritura en disco), y no interrumpir nunca la alimentación sin un apagado controlado previo. Ambas medidas figuran también como deuda técnica pendiente de cierre (sección 12.3).
- **Limitaciones reales del enlace Wi-Fi del nodo *edge*.** Durante las pruebas de campo se comprobó que el enlace Wi-Fi convencional de la Raspberry Pi, sin antenas ni equipos específicos, perdía la conexión a unos 20 m de distancia entre el dron y el punto de acceso. Esta limitación, mayor de lo previsto, llevó a fijar la red celular 4G/LTE como canal principal permanente de enlace con la nube y a reservar el Wi-Fi para tareas de corto alcance (mantenimiento y banco de pruebas), lo que, además, evita el consumo de datos del plan de prepago de la tarjeta SIM. El estudio de un enlace Wi-Fi propietario de largo alcance, planteado en el apartado 12.4, se deriva de esta observación.
- **Uso sistemático del banco de pruebas para ensayos en interiores.** Durante la validación de la cadena de mando se realizó una prueba de armado y desarmado de motores desde el panel de control, con la aeronave apoyada en el suelo y en un recinto cerrado. Un error en la estimación del empuje mínimo sobre el mando de control hizo que el dron despegara de forma imprevista, alcanzara el techo y cayera, con rotura de una hélice y del tren de aterrizaje, y con riesgo de daños mayores. La conclusión, adoptada como norma para el resto del proyecto, es que toda prueba en interior o sin espacio de seguridad debe realizarse en banco, con la aeronave firmemente sujeta y sin hélices, simulando el vuelo por telemetría; los ensayos con hélices quedan reservados en exclusiva para el campo de vuelo autorizado. Esta es la práctica descrita en la Fase 2 de la sección 9.1.
- **El control de versiones como copia de seguridad operativa.** El uso, desde el inicio, de un repositorio con commits frecuentes limitó la pérdida provocada por la incidencia de la tarjeta a un único fichero aún no versionado. La lección es tratar el control de versiones no como un trámite, sino como un respaldo de trabajo: un commit tras cada avance funcional y antes de cualquier prueba en hardware.

- **Aprovisionamiento anticipado de material y disponibilidad de repuestos.** La falta de existencias de algún componente, en particular del kit de vuelo, condicionó el calendario y obligó a reservar con antelación y a estudiar alternativas. Un segundo ejemplo fue el regulador UBEC necesario para alimentar el nodo *edge* desde la batería principal del dron, que llegó pocos días antes del cierre del trabajo, sin margen para su soldadura e integración, que, además, requiere añadir conectores específicos; por lo que hubo que recurrir a una solución alternativa de urgencia. Como solución provisional, se alimentó la Raspberry Pi con una batería externa portátil independiente, y la integración definitiva queda recogida como deuda técnica (apartado 12.3). Por último, la rotura de elementos en la incidencia del banco de pruebas puso de manifiesto la necesidad de contar siempre con un juego de repuestos de las piezas más frágiles y económicas (hélices, tren de aterrizaje, cableado).
- **Criterio para la elección del equipo de radiocontrol.** Se optó por una emisora de gama de entrada (FlySky FS-i6X) por su bajo coste. En retrospectiva, con un desembolso ligeramente superior, se podría haber elegido un modelo de gama media, como los de la familia RadioMaster, que, además del control de vuelo, permite recibir telemetría MAVLink en la propia emisora. Esto habría permitido mostrar en la pantalla del mando la información de vuelo y, sobre todo, los avisos de localización de personas generados por el sistema de detección, sin depender del puesto de mando. La sustitución del equipo de radio se contempla como una línea de trabajo futura (sección 12.4).
- **Disponer de una alternativa de sensor probada.** La cámara CSI prevista falló en la conexión justo antes de la campaña de validación. Tener ya una cámara USB y un soporte orientable permitió realizar el vuelo de validación a tiempo. La lección es probar primero una alternativa para los componentes de los que depende una prueba clave, y no solo contar con repuestos de las piezas frágiles.

### 12.3. Deuda Técnica Asumida

A lo largo del desarrollo surgieron de forma continua nuevas ideas, mejoras y funcionalidades candidatas. Ante la limitación de tiempo del trabajo, se priorizaron las que aportaban mayor valor a los objetivos del TFG (la integración del sistema completo y su validación experimental) y se asumió, de forma consciente, cierta deuda técnica en aquellos puntos en los que una solución más elaborada no resultaba imprescindible para demostrar la viabilidad del sistema.

La Tabla 12.1 resume esa deuda, indicando en cada caso el estado actual, la justificación de la decisión y la mejora prevista para su cierre, la cual se enmarca en el trabajo futuro (sección 12.4).

TABLA 12.1. DEUDA TÉCNICA ASUMIDA DURANTE EL DESARROLLO DEL TFG Y PREVISIÓN DE CIERRE.

| Elemento                                            | Estado actual (deuda asumida)                                                                                                                                                                                                                                                      | Justificación de la priorización                                                                                                                                                                                                                                                                    | Cierre previsto                                                                                                                                                                                                                    |
|-----------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Seguridad del canal MQTT (TLS)                      | El broker Mosquitto opera sin cifrado TLS ni certificados de cliente.                                                                                                                                                                                                              | Todo el tráfico MQTT circula exclusivamente por la red privada mallada de Tailscale (WireGuard), que ya ofrece cifrado extremo a extremo y autenticación de nodos; el broker no está expuesto a Internet.                                                                                           | Habilitar TLS mutuo (mTLS) con una autoridad de certificación propia cuando el sistema deje de operar en red privada o se abra a integradores externos.                                                                            |
| Autenticación y autorización de usuarios            | El panel de control y la API REST no requieren credenciales de usuario.                                                                                                                                                                                                            | Mismo aislamiento mediante VPN: el acceso está restringido a los nodos de la red privada y las pruebas se realizaron con un único operador.                                                                                                                                                         | Autenticación con tokens (JWT u OAuth2/OIDC) y control de acceso por roles (RBAC), tal como se detalla en la sección 12.4.                                                                                                         |
| Escritura en el almacenamiento del nodo <i>edge</i> | Los procesos escriben en la tarjeta microSD (o en el SSD) de la Raspberry Pi a una frecuencia elevada.                                                                                                                                                                             | Simplicidad de implementación durante el desarrollo; el volumen de datos por vuelo es reducido.                                                                                                                                                                                                     | Trasladar los ficheros de estado y los búferes a memoria (tmpfs), reducir la cadencia de sincronización con el disco y emplear la escritura atómica. Relacionado con la lección aprendida de la sección 12.2.                      |
| Gestión de energía del ordenador de a bordo         | Se ha incorporado un pulsador externo de apagado seguro para la Raspberry Pi, que permite ordenar el apagado del sistema sin necesidad de acceso remoto. El nodo <i>edge</i> todavía no reacciona automáticamente ante una batería baja en el vehículo.                            | En las pruebas la alimentación de la Raspberry Pi se gestionó de forma manual y supervisada.                                                                                                                                                                                                        | Implementar en el nodo <i>edge</i> un apagado ordenado al detectar, en la telemetría MAVLink, el umbral de batería baja notificado por la Pixhawk, antes de la desconexión.                                                        |
| Alimentación integrada del nodo <i>edge</i>         | La Raspberry Pi 5 y el acelerador se alimentan mediante una batería externa portátil (power bank) independiente, no de la batería principal del dron. Con esta solución se realizaron tanto los ensayos de banco como el vuelo de validación del sistema integrado (apartado 9.1). | El regulador UBEC necesario para la integración llegó pocos días antes del cierre del trabajo, sin margen para su soldadura y montaje. La solución provisional permitió realizar todos los ensayos previstos, incluido el vuelo, a cambio de un peso adicional de unos 400 g y una autonomía menor. | Soldar e integrar el regulador UBEC con sus conectores para alimentar el nodo <i>edge</i> desde la batería principal y disponer de una autonomía única para todo el sistema durante la campaña de vuelo con computador de a bordo. |
| Descarga de multimedia de alta resolución           | Los vídeos completos de misión y las fotos testigo se recuperan copiándolos manualmente mediante SSH/SCP desde la Raspberry Pi.                                                                                                                                                    | No es una función crítica para la validación del sistema.                                                                                                                                                                                                                                           | Explorador y descarga remota en alta resolución desde el propio panel de control (sección 12.4).                                                                                                                                   |
| Cuadros de mando de Grafana                         | Se dispone de paneles básicos, uno para cada dominio de datos.                                                                                                                                                                                                                     | Son suficientes para verificar la ingesta simultánea y la resiliencia del mecanismo de <i>store-and-forward</i> .                                                                                                                                                                                   | Paneles más ricos y variados, con las miniaturas de cada detección correlacionadas con la línea temporal de la telemetría del dron y con los índices ambientales (sección 12.4).                                                   |
| Funcionalidad del panel de control                  | El panel cubre las operaciones básicas: armar/desarmar, cambio de modo, inicio de misión y grabación.                                                                                                                                                                              | Estas operaciones son las que intervienen en los casos de uso validados experimentalmente.                                                                                                                                                                                                          | Botones directos de despegue (Takeoff) y de retorno a base (RTL), forzado de armado en tierra, edición de parámetros de misión y controles de guiado avanzados (mantenimiento de posición, órbitas), descritos en la sección 12.4  |

|                                               |                                                                                                                                                                                                              |                                                                                                                                                                                                 |                                                                                                                                                                                                         |
|-----------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Cámara de a bordo.                            | El módulo CSI previsto presentó problemas de conexión, y la validación en vuelo se realizó con una cámara USB montada sobre un soporte orientable más pesado.                                                | La cámara USB, ya disponible, permitió mantener el calendario de pruebas; su ángulo regulable coincide con el del conjunto de datos.                                                            | Resolver la integración de la cámara CSI o adoptar formalmente la cámara USB, ajustando el balance de masas.                                                                                            |
| Protección física de la electrónica embarcada | Salvo la carcasa de la Raspberry Pi, el resto de la electrónica de a bordo (Pixhawk, cableado, conectores, distribución de energía) queda expuesta, sin cubierta que la proteja de polvo, humedad o impactos | El montaje se centró en validar la integración funcional del sistema; diseñar y fabricar una cubierta a medida no era imprescindible para las pruebas en banco y en vuelo controlado realizadas | Diseñar una cubierta específica en una herramienta CAD (SolidWorks o Solid Edge) y fabricarla mediante impresión 3D, ajustada a la geometría del chasis y los componentes ya integrados (sección 12.4). |

En términos generales, la deuda técnica que queda está bien definida y no afecta el funcionamiento principal del sistema. La inferencia visual a bordo, la captura continua de métricas y los enlaces de comunicación redundantes ya se han probado en experimentos. Los puntos que faltan se centran en mejorar la seguridad de los servicios y en optimizar la integración física del nodo *edge*. Estos temas son normales al pasar de un prototipo a un uso real y constituyen la prioridad inmediata para el trabajo futuro.

#### 12.4. Trabajo futuro y líneas de mejora

El desarrollo de este proyecto ha puesto sobre la mesa un volumen muy amplio de mejoras posibles, funcionalidades adicionales y nuevos ámbitos de aplicación, superior a lo que podía abordarse en el marco temporal de un TFG. Por ese motivo, más que una lista cerrada de tareas, este apartado busca establecer un criterio de priorización que distinga dos tipos de trabajo futuro con prioridades distintas.

**a) Cierre de la deuda técnica** (sección 12.3). Consiste en solucionar los atajos tomados deliberadamente durante el desarrollo por falta de tiempo. Es la primera prioridad y se considera una condición previa. Antes de agregar cualquier función nueva, se debe resolver esta deuda, ya que afecta la seguridad, la solidez y el funcionamiento del sistema en un despliegue real. Las tareas concretas se presentan en la Tabla 12.1. Incluyen el cifrado TLS mutuo del canal MQTT, la autenticación y control de acceso de los usuarios del panel, la escritura segura en el almacenamiento del nodo *edge*, la gestión de energía con aviso de batería baja y apagado ordenado, la integración de la alimentación del nodo *edge* mediante el regulador UBEC, la descarga remota de multimedia en alta resolución y el enriquecimiento de los cuadros de mando de Grafana. Algunas de estas tareas se explican con más detalle en los siguientes puntos.

**b) Evolución del producto y nuevos ámbitos de aplicación.** Consiste en ampliar las capacidades del sistema y aplicarlo a usos que van más allá de la búsqueda y el rescate en entornos montañosos. Entre estos usos se encuentran la vigilancia y el control perimetral de grandes extensiones de terreno, como explotaciones agrícolas y ganaderas, polígonos industriales e infraestructuras lineales. También incluye el apoyo al salvamento acuático, mediante la carga útil liberable para transportar un flotador o un kit de emergencia hasta una persona en el agua. Estas aplicaciones usan sin cambios el núcleo del sistema, que incluye detección embarcada, arquitectura IoT y comunicaciones redundantes. Solo deberían abordarse una vez que la base descrita en el punto anterior esté consolidada.



Por muy desarrollado y probado que esté el dron a nivel técnico, intervenir en un rescate real no depende solo de la ingeniería. Para poder actuar en una emergencia real, es obligatorio cumplir primero con los trámites legales descritos en la sección 10.4. Solo se puede operar si lo solicita un cuerpo público como Bomberos o Protección Civil, según el artículo 12 del Real Decreto 517/2024 [14]. También se debe contar con la evaluación de riesgos de la operación. Este procedimiento administrativo es una condición previa necesaria que no forma parte del desarrollo técnico de este trabajo.

Las líneas de evolución del producto identificadas se enumeran a continuación.

- **Potenciar la página web:** Elevar el proyecto (actualmente hay una primera versión en <https://guardian-eye.gorostiditfg.com/> ) a la categoría OpenSource, distribuyendo los detalles del TFG, código usado, material empleado, y todas las características, en la actual página web del proyecto, evolucionándola hacia un espacio actualizado, colaborativo y de conocimiento.
- **Evolución y Maduración del Panel de Control Remoto (PWA / API REST):**
  - **Planificación y Envío Dinámico de Misiones:** Crear una interfaz interactiva con cartografía digital en el panel web. Esta debe permitir definir polígonos de búsqueda SAR y cargar de inmediato listas de waypoints, incluidas rutas en cuadrícula, altitudes y velocidades de barrido. Los datos se enviarán directamente a la Pixhawk 6X mediante los endpoints dedicados de la API.
  - **Ampliación de Controles de Guiado:** Añadir botones y controles nuevos de teleoperación en tiempo real, como mantenimiento de posición asistido (*PosHold*), creación de órbitas automáticas sobre puntos de interés georreferenciados y retorno o aterrizaje con umbrales que se pueden ajustar
- **Integración de controles físicos y avisos en el equipo de radiocontrol:** Por un lado, la limitación en la asignación de canales de la emisora FlySky actual puede subsanarse mediante la instalación de firmware abierto, lo que habilita los interruptores superiores para controlar el armado de motores y el inicio de la inferencia desde el propio mando. No obstante, al no admitir el protocolo de enlace requerido para la recepción de telemetría de retorno, se plantea sustituir el equipo por una emisora abierta (tipo RadioMaster con ExpressLRS). Esto permitirá integrar en la pantalla del piloto la monitorización del vuelo y las alertas de personas georreferenciadas sin depender de la pantalla del puesto de control.
- **Integración de Enlace Wi-Fi Propietario de Largo Alcance (Wi-Fi Broadcast):**
  - Estudio de un enlace Wi-Fi propietario de largo alcance. A raíz de las limitaciones de alcance del Wi-Fi convencional detectadas en las pruebas (sección 12.2), se plantea analizar un enlace digital propietario de baja latencia, en la línea de lo que emplean algunos fabricantes comerciales (por ejemplo, DJI). Una vía es Wi-Fi Broadcast: adaptadores en modo monitor con

inyección de paquetes en la banda de 5,8 GHz, sin la pila TCP/IP convencional.

- Este tipo de enlace permitiría enviar simultáneamente telemetría MAVLink y vídeo digital de alta definición a distancias de varios kilómetros. No tendría las desconexiones ni los tiempos de reasociación que suelen presentar las redes Wi-Fi habituales. Serviría como canal principal de alta capacidad en zonas rurales sin cobertura 4G/LTE.
- **Supervisión Visual Avanzada en Grafana con Miniaturas (*Thumbnails*):**
  - Mejorar los cuadros de mando de Grafana para permitir el acceso a los datos transmitidos por IoT.
  - Incluir en esta mejora la posibilidad de ver, de forma interactiva y en orden cronológico, las miniaturas de cada detección generada por YOLO, con sus cajas delimitadoras superpuestas. Estas miniaturas estarán correlacionadas en la misma línea temporal con las métricas cinemáticas del dron como rumbo, actitud, altitud barométrica y batería, junto con los índices ambientales del sensor BME680
- **Integración de Carga Útil Termográfica (Sensor Dual RGB/Térmico):** Incluir una microcámara infrarroja, como la FLIR Lepton, junto a la cámara RGB. Así se pueden entrenar modelos multispectrales que ayudan a detectar víctimas por la noche o bajo follaje denso mediante el contraste térmico.
- **Estabilización activa de la carga óptica mediante un gimbal de 2 ejes.** En el vuelo de validación se observó que las vibraciones y los cambios de actitud de la aeronave provocan oscilaciones en la imagen. Aunque esto no impidió la detección, sí afecta la estabilidad del fotograma (apartado 9.4). Si se monta la cámara sobre un estabilizador motorizado de dos ejes, el detector recibirá un flujo de vídeo más estable, sobre todo a velocidad de crucero. Además, esto permite medir el efecto real de la vibración sobre las métricas de detección. Esta mejora puede utilizarse junto con la integración de la carga útil termográfica mencionada en el punto anterior.
- **Ampliación continua del conjunto de datos SAR y del aprendizaje activo.** Automatizar el etiquetado y el reentrenamiento del modelo YOLO11 mediante flujos de aprendizaje activo que seleccionen fotogramas de baja confianza en vuelos reales. Así, se puede volver a entrenar la NPU para enfrentar posturas atípicas y oclusiones complejas.
- **Envío de Material Auxiliar a través del Dron:** El dron debe tener una cesta que se libere a través de un servo, y en la cual poder llevar una carga útil, como un kit de emergencia, un flotador (para rescate en el mar), una manta térmica o incluso un walkie-talkie de larga distancia.
- **Despliegue Multi-UAV en Enjambre Cooperativo.** A través del espacio de nombres escalable de MQTT (`dronsar/{dron_id}/#`) y la topología mallada de

Tailscale, se podrían orquestar flotas coordinadas de varios drones. Estos dividen y cubren grandes cuadrículas de búsqueda de forma colaborativa

- **Análisis del enlace de datos celular.** En el vuelo de validación, la transmisión de vídeo en directo por 4G/LTE se interrumpió al superar la aeronave unos 10 m de altura. El resto de los datos se recuperó íntegramente mediante el método *store-and-forward* al descender (apartado 9.3). Es necesario estudiar este comportamiento en campañas específicas. Se deben variar la altura, la orientación y la ubicación de la antena del módem en el fuselaje, y medir la potencia de la señal y el ancho de banda de subida disponible. Así se podrá saber si el vídeo en directo es posible de forma continua sobre la red celular o si conviene apoyarlo en el enlace Wi-Fi propio de largo alcance que se plantea en este mismo apartado.

En conjunto, la cantidad de mejoras y de nuevos ámbitos de aplicación que este proyecto hace posibles es muy superior a lo que cabía en un Trabajo Fin de Grado. Por ello, conviene insistir en el criterio anterior: primero, cerrar la deuda técnica asumida de forma consciente (apartado a), lo que convierte el prototipo validado en un sistema desplegable con garantías; y solo después, crecer en funcionalidad y en ámbitos de uso (apartado b). Mantener esa distinción evita que se acumulen nuevas capacidades sobre una base todavía incompleta.

## BIBLIOGRAFÍA

- [1] X. Zhang, Y. Feng, N. Wang, G. Lu y S. Mei, "Aerial Person Detection for Search and Rescue: Survey and Benchmarks," *J. Remote Sens.*, vol. 5, art. 0474, ene. 2025, doi: 10.34133/remotesensing.0474.
- [2] R. Marín, "Un dron con tecnología infrarroja ayudó en la búsqueda y rescate de un hombre con demencia perdido en Malibú," *Infobae*, 27 de diciembre de 2024. Accedido: 8 de agosto de 2026. [En línea]. Disponible en: <https://www.infobae.com/estados-unidos/2024/12/27/un-dron-con-tecnologia-infrarroja-ayudo-en-las-busqueda-y-rescate-de-un-hombre-con-demencia-perdido-en-malibu/>
- [3] General Drones, "Drones de salvamento y emergencia," *General Drones*. Accedido: 6 de agosto de 2026. [En línea]. Disponible en: <https://www.generaldrones.es/>
- [4] Eagle Eyes Search, "Eagle Eyes Search: AI-Powered Search and Rescue Solution," *Eagle Eyes Search*. Accedido: 6 de agosto de 2026. [En línea]. Disponible en: <https://www.eagleeyesearch.com/>
- [5] K. Annapurna, "AI solves the mystery of missing Italian climber," *Explorersweb*, 13 de julio de 2025. Accedido: 6 de agosto de 2026. [En línea]. Disponible en: <https://explorersweb.com/ai-solves-the-mystery-of-missing-italian-climber/>
- [6] Roboflow, "HERIDAL Dataset for Search and Rescue," *Roboflow Universe*, 2026. Accedido: 6 de agosto de 2026. [En línea]. Disponible en: <https://universe.roboflow.com/licenta-ynwvo/heridal-lrbkc>
- [7] M. M. Azari, F. Rosas y S. Pollin, "Cellular Connectivity for UAVs: Network Modeling, Performance Analysis and Design Guidelines," *IEEE Trans. Wireless Commun.*, vol. 18, n.º 7, pp. 3366-3381, jul. 2019, doi: 10.1109/TWC.2019.2910112.
- [8] P. Gorczak, C. Bektas, F. Kurtz, T. Lübcke y C. Wietfeld, "Robust Cellular Communications for Unmanned Aerial Vehicles in Maritime Search and Rescue," en *2019 IEEE International Symposium on Safety, Security, and Rescue Robotics (SSRR)*, Würzburg, Alemania, sep. 2019, pp. 229-234, doi: 10.1109/SSRR.2019.8848932.
- [9] M. A. Zulkifley, M. Behjati, R. Nordin y M. S. Zakaria, "Mobile Network Performance and Technical Feasibility of LTE-Powered Unmanned Aerial Vehicle," *Sensors*, vol. 21, n.º 8, art. 2848, abr. 2021, doi: 10.3390/s21082848.
- [10] M. K. Sharma, C.-F. Liu, I. Farhat, N. Sehad, W. Hamidouche y M. Debbah, "UAV Immersive Video Streaming: A Comprehensive Survey, Benchmarking, and Open Challenges," *arXiv:2311.00082*, oct. 2023. [En línea]. Disponible en: <https://arxiv.org/abs/2311.00082>

- [11] "Reglamento de Ejecución (UE) 2019/947 de la Comisión, de 24 de mayo de 2019, relativo a las normas y los procedimientos aplicables a la utilización de aeronaves no tripuladas," *Diario Oficial de la Unión Europea*, L 152, pp. 45-71, 11 jun. 2019. [En línea]. Disponible en: [http://data.europa.eu/eli/reg\\_impl/2019/947/oj](http://data.europa.eu/eli/reg_impl/2019/947/oj)
- [12] Agencia Estatal de Seguridad Aérea (AESA), "Concepto normativo," en *Curso de formación para pilotos en las subcategorías A1/A3*, ref. A-DUAS-FOR-OP01-v6, Ministerio de Transportes, Movilidad y Agenda Urbana, Madrid, España, mayo 2022, diapositiva 16. [En línea]. Disponible en: <https://www.seguridadaaerea.gob.es/sites/default/files/Curso.Formacion.A1.A3.Completo.v6.pdf>
- [13] "Reglamento Delegado (UE) 2019/945 de la Comisión, de 12 de marzo de 2019, sobre los sistemas de aeronaves no tripuladas y los operadores de terceros países de sistemas de aeronaves no tripuladas," *Diario Oficial de la Unión Europea*, L 152, pp. 1-40, 11 jun. 2019. [En línea]. Disponible en: [http://data.europa.eu/eli/reg\\_del/2019/945/oj](http://data.europa.eu/eli/reg_del/2019/945/oj)
- [14] "Real Decreto 517/2024, de 4 de junio, por el que se desarrolla el régimen jurídico para la utilización civil de sistemas de aeronaves no tripuladas (UAS)," *Boletín Oficial del Estado*, núm. 136, pp. 65362-65436, 5 jun. 2024. [En línea]. Disponible en: <https://www.boe.es/eli/es/rd/2024/06/04/517>
- [15] "Sentencia de 19 de junio de 2025, de la Sala Tercera del Tribunal Supremo, que estima parcialmente el recurso contencioso-administrativo interpuesto por la Asociación Nacional de Pilotos de Drones contra el Real Decreto 517/2024, de 4 de junio, declarando la nulidad de la Sección 3.ª del Capítulo VI," *Boletín Oficial del Estado*, núm. 168, 14 jul. 2025. [En línea]. Disponible en: <https://www.boe.es/buscar/doc.php?id=BOE-A-2025-14308>
- [16] "Ley 21/2003, de 7 de julio, de Seguridad Aérea," *Boletín Oficial del Estado*, núm. 162, 8 jul. 2003. [En línea]. Disponible en: <https://www.boe.es/buscar/act.php?id=BOE-A-2003-13616>
- [17] "Ley 17/2015, de 9 de julio, del Sistema Nacional de Protección Civil," *Boletín Oficial del Estado*, núm. 164, pp. 57409-57435, 10 jul. 2015. [En línea]. Disponible en: <https://www.boe.es/eli/es/l/2015/07/09/17>
- [18] ENAIRE, "ENAIRE Drones." Accedido: 10 de agosto de 2026. [En línea]. Disponible en: <https://drones.enaire.es/>
- [19] ENAIRE, "AIP España, ENR 5.5: Aeromodelismo, deportes aéreos y otras actividades." Accedido: 10 de agosto de 2026. [En línea]. Disponible en: [https://aip.enaire.es/AIP/contenido\\_AIP/ENR/LE\\_ENR\\_5\\_5\\_es.html](https://aip.enaire.es/AIP/contenido_AIP/ENR/LE_ENR_5_5_es.html)
- [20] "Ley 11/2022, de 28 de junio, General de Telecomunicaciones," *Boletín Oficial del Estado*, núm. 155, 29 jun. 2022. [En línea]. Disponible en: <https://www.boe.es/buscar/act.php?id=BOE-A-2022-10757>

- [21] "Orden TDF/732/2026, de 10 de julio, por la que se aprueba el Cuadro Nacional de Atribución de Frecuencias (CNAF)," *Boletín Oficial del Estado*, núm. 173, pp. 100980-101328, 17 jul. 2026. [En línea]. Disponible en: <https://www.boe.es/eli/es/o/2026/07/10/tdf732>
- [22] "Directiva 2014/53/UE del Parlamento Europeo y del Consejo, de 16 de abril de 2014, relativa a la armonización de las legislaciones de los Estados miembros sobre la comercialización de equipos radioeléctricos, y por la que se deroga la Directiva 1999/5/CE," *Diario Oficial de la Unión Europea*, L 153, pp. 62-106, 22 may. 2014. [En línea]. Disponible en: <http://data.europa.eu/eli/dir/2014/53/oj>
- [23] "Reglamento (UE) 2016/679 del Parlamento Europeo y del Consejo, de 27 de abril de 2016, relativo a la protección de las personas físicas en lo que respecta al tratamiento de datos personales y a la libre circulación de estos datos y por el que se deroga la Directiva 95/46/CE (Reglamento general de protección de datos)," *Diario Oficial de la Unión Europea*, L 119, pp. 1-88, 4 may. 2016. [En línea]. Disponible en: <http://data.europa.eu/eli/reg/2016/679/oj>
- [24] "Ley Orgánica 3/2018, de 5 de diciembre, de Protección de Datos Personales y garantía de los derechos digitales," *Boletín Oficial del Estado*, núm. 294, 6 dic. 2018. [En línea]. Disponible en: <https://www.boe.es/buscar/act.php?id=BOE-A-2018-16673>
- [25] 3GPP, "Study on Enhanced LTE Support for Aerial Vehicles," Rep. téc. TR 36.777, versión 15.0.0, ene. 2018. Accedido: 5 de septiembre de 2026. [En línea]. Disponible en: [https://www.3gpp.org/ftp/Specs/archive/36\\_series/36.777/](https://www.3gpp.org/ftp/Specs/archive/36_series/36.777/)
- [26] Dronecode Foundation, "MAVLink Developer Guide." Accedido: 5 de septiembre de 2026. [En línea]. Disponible en: <https://mavlink.io/en/>
- [27] A. Banks, E. Briggs, K. Borgendale y R. Gupta, Eds., "MQTT Version 5.0," OASIS Standard, 7 de marzo de 2019. [En línea]. Disponible en: <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>
- [28] H. Schulzrinne, A. Rao y R. Lanphier, "Real Time Streaming Protocol (RTSP)," Internet Engineering Task Force, RFC 2326, abr. 1998, doi: 10.17487/RFC2326.
- [29] IEEE, "IEEE Standard for Information Technology—Telecommunications and Information Exchange between Systems—Local and Metropolitan Area Networks—Specific Requirements—Part 11: Wireless LAN Medium Access Control (MAC) and Physical Layer (PHY) Specifications," IEEE Std 802.11-2020, feb. 2021. [En línea]. Disponible en: <https://standards.ieee.org/ieee/802.11/7028/>
- [30] "Reglamento (UE) 2024/1689 del Parlamento Europeo y del Consejo, de 13 de junio de 2024, por el que se establecen normas armonizadas en materia de inteligencia artificial (Reglamento de Inteligencia Artificial)," *Diario Oficial de la Unión Europea*, L, 12 jul. 2024. [En línea]. Disponible en: <http://data.europa.eu/eli/reg/2024/1689/oj>

- [31] LigPower, "Air Gear 450II Combo Set — Multi-Rotor UAV Power, Technical Data Sheet," tabla "Test Data" (ensayo a 16 V, motor AIR2216II-KV920 con hélice T1045). Accedido: 7 de septiembre de 2026. [En línea]. Disponible en: [https://cdn.robotshop.com/media/T/Tmo/RB-Tmo-266/pdf/ligpower\\_airgear\\_450ii\\_combo\\_set\\_multi\\_rotor\\_uav\\_p\\_datasheet.pdf](https://cdn.robotshop.com/media/T/Tmo/RB-Tmo-266/pdf/ligpower_airgear_450ii_combo_set_multi_rotor_uav_p_datasheet.pdf)
- [32] J. Redmon, S. Divvala, R. Girshick y A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," en *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Las Vegas, NV, EE. UU., jun. 2016, pp. 779-788, doi: 10.1109/CVPR.2016.91.
- [33] J. Terven, D.-M. Córdova-Esparza y J.-A. Romero-González, "A Comprehensive Review of YOLO Architectures in Computer Vision: From YOLOv1 to YOLOv8 and YOLO-NAS," *Mach. Learn. Knowl. Extr.*, vol. 5, n.º 4, pp. 1680-1716, dic. 2023, doi: 10.3390/make5040083.
- [34] data, «SAR Computer Vision Dataset», Roboflow Universe, 2024. Accedido: 5 de septiembre de 2026. [En línea]. Disponible en: <https://universe.roboflow.com/data-1qpze/sar-qfrqe>
- [35] ENAIRE, "Vuelos urgentes y en emergencias." Accedido: 5 de septiembre de 2026. [En línea]. Disponible en: [https://www.enaire.es/vuela\\_tu\\_dron/donde\\_puedes\\_volar\\_tu\\_dron/vuelos\\_urgentes\\_y\\_en\\_emergencias](https://www.enaire.es/vuela_tu_dron/donde_puedes_volar_tu_dron/vuelos_urgentes_y_en_emergencias)
- [36] SoriaNoticias, "Así se paga un rescate en helicóptero," SoriaNoticias, 16 de agosto de 2026. Accedido: 5 de septiembre de 2026. [En línea]. Disponible en: <https://sorianoticias.com/noticia/2026-08-16-asi-se-paga-un-rescate-en-helicoptero-135657>

## ANEXO A. GLOSARIO DE ABREVIATURAS

|                  |                                                                                                    |
|------------------|----------------------------------------------------------------------------------------------------|
| <b>4G / LTE</b>  | <i>Long-Term Evolution</i> — red de telefonía móvil de cuarta generación                           |
| <b>A1 / A3</b>   | Subcategorías de operación de UAS dentro de la categoría «abierta» (AESA/EASA)                     |
| <b>AESA</b>      | Agencia Estatal de Seguridad Aérea                                                                 |
| <b>AIP</b>       | <i>Aeronautical Information Publication</i> — Publicación de Información Aeronáutica               |
| <b>AMI</b>       | <i>Amazon Machine Image</i> — imagen de máquina de Amazon (AWS)                                    |
| <b>API</b>       | <i>Application Programming Interface</i> — interfaz de programación de aplicaciones                |
| <b>ARM</b>       | <i>Advanced RISC Machines</i> — arquitectura de procesador                                         |
| <b>AWS</b>       | <i>Amazon Web Services</i>                                                                         |
| <b>BEC / C</b>   | <i>(Universal) Battery Eliminator Circuit</i> — regulador de tensión para alimentar la electrónica |
| <b>BOE</b>       | Boletín Oficial del Estado                                                                         |
| <b>BVLOS</b>     | <i>Beyond Visual Line of Sight</i> — operación más allá del alcance visual                         |
| <b>C0–C6</b>     | Clases de marcado de UAS según el Reglamento (UE) 2019/945                                         |
| <b>C2PSA</b>     | Bloque de atención espacial de la arquitectura YOLO11                                              |
| <b>CC BY 4.0</b> | <i>Creative Commons Attribution 4.0</i> — licencia de uso abierto                                  |
| <b>CNAF</b>      | Cuadro Nacional de Atribución de Frecuencias                                                       |
| <b>CNN</b>       | <i>Convolutional Neural Network</i> — red neuronal convolucional                                   |
| <b>COCO</b>      | <i>Common Objects in Context</i> — conjunto de datos de referencia en visión                       |
| <b>CPU</b>       | <i>Central Processing Unit</i> — unidad central de procesamiento                                   |
| <b>CRC</b>       | <i>Cyclic Redundancy Check</i> — comprobación de redundancia cíclica                               |
| <b>CS</b>        | <i>Chip Select</i> — pin de selección de dispositivo                                               |
| <b>CSI</b>       | <i>Camera Serial Interface</i> — interfaz serie de cámara (MIPI-CSI)                               |
| <b>CSS</b>       | <i>Cascading Style Sheets</i> — hojas de estilo en cascada                                         |
| <b>CTR</b>       | <i>Control Zone</i> — zona de control de tránsito aéreo                                            |
| <b>DNS</b>       | <i>Domain Name System</i> — sistema de nombres de dominio                                          |
| <b>DoD</b>       | <i>Depth of Discharge</i> — profundidad de descarga (de la batería)                                |



|                     |                                                                                                    |
|---------------------|----------------------------------------------------------------------------------------------------|
| <b>DRI</b>          | <i>Direct Remote Identification</i> — identificación remota directa                                |
| <b>EASA</b>         | <i>European Union Aviation Safety Agency</i> — Agencia de Seguridad Aérea de la UE                 |
| <b>EC2</b>          | <i>(Amazon) Elastic Compute Cloud</i>                                                              |
| <b>ECTS</b>         | <i>European Credit Transfer and Accumulation System</i>                                            |
| <b>Edge-AI</b>      | Inteligencia artificial ejecutada en el borde (inferencia a bordo del dron)                        |
| <b>ENAIRE</b>       | Proveedor de servicios de navegación aérea de España                                               |
| <b>ENR</b>          | <i>En-Route</i> — sección «En ruta» de la AIP                                                      |
| <b>ESC</b>          | <i>Electronic Speed Controller</i> — controlador electrónico de velocidad                          |
| <b>FP32</b>         | <i>Floating Point 32-bit</i> — coma flotante de 32 bits                                            |
| <b>FPS</b>          | <i>Frames Per Second</i> — fotogramas por segundo                                                  |
| <b>FPV</b>          | <i>First Person View</i> — vista en primera persona                                                |
| <b>GCS</b>          | <i>Ground Control Station</i> — estación de control en tierra                                      |
| <b>GEO</b>          | Zonas geográficas de UAS ( <i>UAS geographical zones</i> )                                         |
| <b>GNSS</b>         | <i>Global Navigation Satellite System</i> — sistema global de navegación por satélite              |
| <b>GPIO</b>         | <i>General Purpose Input/Output</i> — entrada/salida de propósito general                          |
| <b>GPS</b>          | <i>Global Positioning System</i> — sistema de posicionamiento global                               |
| <b>GPU</b>          | <i>Graphics Processing Unit</i> — unidad de procesamiento gráfico                                  |
| <b>HAT</b>          | <i>Hardware Attached on Top</i> — placa de expansión de la Raspberry Pi                            |
| <b>HEF</b>          | <i>Hailo Executable Format</i> — formato ejecutable compilado para la NPU Hailo                    |
| <b>HIL / HITL</b>   | <i>Hardware In The Loop</i> — hardware en el bucle de simulación                                   |
| <b>HLS</b>          | <i>HTTP Live Streaming</i> — protocolo de transmisión de vídeo                                     |
| <b>HMAC</b>         | <i>Hash-based Message Authentication Code</i> — código de autenticación de mensajes basado en hash |
| <b>HTML</b>         | <i>HyperText Markup Language</i>                                                                   |
| <b>HTTP / HTTPS</b> | <i>HyperText Transfer Protocol (Secure)</i> — protocolo de transferencia de hipertexto (seguro)    |
| <b>I2C</b>          | <i>Inter-Integrated Circuit</i> — bus serie de dos hilos                                           |

|                  |                                                                                                                |
|------------------|----------------------------------------------------------------------------------------------------------------|
| <b>IA / IAG</b>  | Inteligencia Artificial / Inteligencia Artificial Generativa                                                   |
| <b>ICMP</b>      | <i>Internet Control Message Protocol</i>                                                                       |
| <b>IEEE</b>      | <i>Institute of Electrical and Electronics Engineers</i>                                                       |
| <b>IETF</b>      | <i>Internet Engineering Task Force</i>                                                                         |
| <b>INT8</b>      | <i>Integer 8-bit</i> — entero de 8 bits (cuantización)                                                         |
| <b>IoT</b>       | <i>Internet of Things</i> — Internet de las cosas                                                              |
| <b>IP</b>        | <i>Internet Protocol</i> — protocolo de internet / dirección IP                                                |
| <b>ISM</b>       | <i>Industrial, Scientific and Medical</i> — banda de radiofrecuencia de uso común                              |
| <b>IVA</b>       | Impuesto sobre el Valor Añadido                                                                                |
| <b>JSON</b>      | <i>JavaScript Object Notation</i>                                                                              |
| <b>JWT</b>       | <i>JSON Web Token</i>                                                                                          |
| <b>KV</b>        | Constante de velocidad de un motor (rpm por voltio)                                                            |
| <b>LiPo</b>      | <i>Lithium Polymer</i> — batería de polímero de litio                                                          |
| <b>LOPD-GDD</b>  | Ley Orgánica de Protección de Datos Personales y Garantía de los Derechos Digitales                            |
| <b>MAVLink</b>   | <i>Micro Air Vehicle Link</i> — protocolo de comunicación con el autopiloto                                    |
| <b>MTOW</b>      | <i>Maximum Take-Off Weight</i> — peso máximo al despegue                                                       |
| <b>MQTT</b>      | <i>Message Queuing Telemetry Transport</i> — protocolo de mensajería ligero para IoT                           |
| <b>NCNN</b>      | Biblioteca de inferencia de redes neuronales para dispositivos embebidos (Tencent)                             |
| <b>NPU</b>       | <i>Neural Processing Unit</i> — unidad de procesamiento neuronal                                               |
| <b>OASIS</b>     | <i>Organization for the Advancement of Structured Information Standards</i> — organismo estandarizador de MQTT |
| <b>OAuth 2.0</b> | <i>Open Authorization</i> — marco de autorización                                                              |
| <b>ODS</b>       | Objetivos de Desarrollo Sostenible                                                                             |
| <b>OIDC</b>      | <i>OpenID Connect</i> — capa de identidad sobre OAuth 2.0                                                      |
| <b>ONNX</b>      | <i>Open Neural Network Exchange</i> — formato abierto de intercambio de modelos                                |
| <b>P2P</b>       | <i>Peer to Peer</i> — comunicación entre pares                                                                 |
| <b>PCIe</b>      | <i>Peripheral Component Interconnect Express</i> — bus de expansión                                            |

|                  |                                                                                   |
|------------------|-----------------------------------------------------------------------------------|
| <b>PDB</b>       | <i>Power Distribution Board</i> — placa de distribución de potencia               |
| <b>PPM</b>       | <i>Pulse Position Modulation</i> — modulación por posición de pulso               |
| <b>PWA</b>       | <i>Progressive Web App</i> — aplicación web progresiva                            |
| <b>PWM</b>       | <i>Pulse Width Modulation</i> — modulación por ancho de pulso                     |
| <b>QoS</b>       | <i>Quality of Service</i> — calidad de servicio                                   |
| <b>RAM</b>       | <i>Random Access Memory</i> — memoria de acceso aleatorio                         |
| <b>RBAC</b>      | <i>Role-Based Access Control</i> — control de acceso basado en roles              |
| <b>RC</b>        | <i>Radio Control</i> — radiocontrol                                               |
| <b>REST</b>      | <i>Representational State Transfer</i> — estilo de arquitectura de API            |
| <b>RF</b>        | <i>Radio Frequency</i> — radiofrecuencia                                          |
| <b>RGB</b>       | <i>Red, Green, Blue</i> — rojo, verde y azul                                      |
| <b>RGPD</b>      | Reglamento General de Protección de Datos (Reglamento (UE) 2016/679)              |
| <b>RPi</b>       | Raspberry Pi                                                                      |
| <b>RTL</b>       | <i>Return To Launch</i> — regreso automático al punto de despegue                 |
| <b>RTSP</b>      | <i>Real-Time Streaming Protocol</i> — protocolo de transmisión en tiempo real     |
| <b>RX / TX</b>   | Recepción / Transmisión (líneas serie UART)                                       |
| <b>SAR</b>       | <i>Search and Rescue</i> — búsqueda y rescate                                     |
| <b>SCL / SDA</b> | <i>Serial Clock Line / Serial Data Line</i> — líneas de reloj y datos del bus I2C |
| <b>SCP</b>       | <i>Secure Copy Protocol</i> — copia segura de archivos                            |
| <b>SDO</b>       | <i>Serial Data Out</i> — pin de salida de datos del sensor                        |
| <b>SHA-256</b>   | <i>Secure Hash Algorithm 256-bit</i> — algoritmo de resumen criptográfico         |
| <b>SIM</b>       | <i>Subscriber Identity Module</i> — módulo de identidad del abonado               |
| <b>SiK</b>       | Firmware de código abierto para las radios de telemetría de 433 MHz               |
| <b>SITL</b>      | <i>Software In The Loop</i> — software en el bucle de simulación                  |
| <b>SoC</b>       | <i>System on Chip</i> — sistema en un chip                                        |
| <b>SPI</b>       | <i>Serial Peripheral Interface</i> — interfaz periférica serie                    |
| <b>SQL</b>       | <i>Structured Query Language</i> — lenguaje de consulta estructurado              |
| <b>SSD</b>       | <i>Solid State Drive</i> — unidad de estado sólido                                |

|               |                                                                                              |
|---------------|----------------------------------------------------------------------------------------------|
| <b>SSH</b>    | <i>Secure Shell</i> — protocolo de acceso remoto seguro                                      |
| <b>TCP</b>    | <i>Transmission Control Protocol</i> — protocolo de control de transmisión                   |
| <b>TFG</b>    | Trabajo Fin de Grado                                                                         |
| <b>TLS</b>    | <i>Transport Layer Security</i> — seguridad de la capa de transporte                         |
| <b>TOPS</b>   | <i>Tera Operations Per Second</i> — billones de operaciones por segundo                      |
| <b>TWR</b>    | <i>Thrust-to-Weight Ratio</i> — relación empuje-peso                                         |
| <b>UART</b>   | <i>Universal Asynchronous Receiver-Transmitter</i> — transmisor-receptor asíncrono universal |
| <b>UAS</b>    | <i>Unmanned Aircraft System</i> — sistema de aeronave no tripulada                           |
| <b>UAV</b>    | <i>Unmanned Aerial Vehicle</i> — vehículo aéreo no tripulado                                 |
| <b>UC3M</b>   | Universidad Carlos III de Madrid                                                             |
| <b>UDP</b>    | <i>User Datagram Protocol</i> — protocolo de datagramas de usuario                           |
| <b>UE</b>     | Unión Europea                                                                                |
| <b>USB</b>    | <i>Universal Serial Bus</i> — bus serie universal                                            |
| <b>VLOS</b>   | <i>Visual Line of Sight</i> — operación dentro del alcance visual                            |
| <b>VPN</b>    | <i>Virtual Private Network</i> — red privada virtual                                         |
| <b>VTX</b>    | <i>Video Transmitter</i> — transmisor de vídeo (FPV)                                         |
| <b>WebRTC</b> | <i>Web Real-Time Communication</i> — comunicación web en tiempo real                         |
| <b>Wi-Fi</b>  | Red inalámbrica según el estándar IEEE 802.11                                                |
| <b>YOLO</b>   | <i>You Only Look Once</i> — arquitectura de detección de objetos en una sola pasada          |

## ANEXO B. CERTIFICADO DE REGISTRO COMO OPERADOR UAS ANTE AESA.

|                                                                                                                                                                                                                                                                                                                                   |                                                                                   |                                                                                                                             |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| <br>European Union Aviation Safety Agency                                                                                                                                                                                                        | CID: AESAPIPAUAS0000N4PDKI04SCPQLDA                                               | <br>AGENCIA ESTATAL<br>DE SEGURIDAD AÉREA |
|                                                                                                                                                                                                                                                  |  |                                                                                                                             |
| <b>Registro de Operador UAS</b><br>UAS OPERATOR REGISTRATION                                                                                                                                                                                                                                                                      |                                                                                   |                                                                                                                             |
| <b>Nombre (First name)</b><br>NEREA                                                                                                                                                                                                                                                                                               | <b>Apellidos (Last name)</b><br>GOROSTIDI GARCIA                                  |                                                                                                                             |
| <b>Número de registro (Registration number)</b><br>ESPkpq4pb7xmj65p                                                                                                                                                                                                                                                               | <b>Fecha de caducidad (Expiration date)</b><br>07/07/2029 17:00:43                |                                                                                                                             |
| <br>Código Seguro de Verificación (CSV/CID) : AESAPIPAUAS0000N4PDKI04SCPQLDA<br>Este documento puede ser verificado en / This document may be verified at: <a href="https://sede.seguridadaerea.gob.es">https://sede.seguridadaerea.gob.es</a> |                                                                                   |                                                                                                                             |

## ANEXO C. CERTIFICADO DE PILOTO A DISTANCIA A1/A3.



MINISTERIO  
DE TRANSPORTES  
Y MOVILIDAD SOSTENIBLE



### O F I C I O

NEREA GOROSTIDI GARCIA

S/REF:  
N/REF: 2026/LPAE0/010948  
ASUNTO: RESOLUCIÓN POSITIVA  
  
ORIGEN: E04865601 - Agencia Estatal de Seguridad Aérea  
  
DESTINATARIO: 06642977W - NEREA GOROSTIDI GARCIA

### RESOLUCIÓN POR LA QUE SE OTORGA LA PRUEBA DE SUPERACIÓN DE FORMACIÓN EN LÍNEA PARA REALIZAR OPERACIONES CON UAS EN CATEGORÍA ABIERTA, SUBCATEGORÍAS A1 Y A3

En relación con la solicitud de emisión de la prueba de superación de formación en línea presentada en la Agencia Estatal de Seguridad Aérea con fecha **26 de marzo de 2026** (registro de entrada **2026088302**), por D/D<sup>a</sup> **GOROSTIDI GARCIA, NEREA**, se comunica que, al haber superado la formación y examen exigible de acuerdo con los apartados UAS.OPEN.020 y UAS.OPEN.040 del Reglamento de Ejecución (UE) 2019/947 esta dirección

#### RESUELVE:

Conceder la emisión de la prueba de superación de formación en línea necesaria para realizar operaciones con UAS en categoría abierta, subcategorías A1 y A3, a favor de

NEREA GOROSTIDI GARCIA, con DNI 06642977W

EL JEFE DE LA DIVISIÓN DE SISTEMAS DE AERONAVES NO TRIPULADAS  
(P.D.F. de la Directora de Seguridad de Aeronaves)  
Fdo.: ROBERTO GÁNDARA OSSEL

(firmado digitalmente)

Utilice el siguiente enlace para descargar su certificado de piloto

Tipo de documento (Código CID)

**PRUEBA DE SUPERACIÓN DE FORMACIÓN EN LÍNEA (AESASGEEPGE000M6R8JHE7M51G1DA)**

<https://sede.seguridadaerea.gob.es/CID/servlet/Ficheros?id=AESASGEEPGE000M6R8JHE7M51G1DA&origen=ws&firma=firmaBarra>

Contra la presente resolución, que no pone fin a la vía administrativa, los interesados pueden interponer, en los supuestos indicados en el artículo 112.1 y de conformidad con los artículos 121 y 122 de la Ley 39/2015, de 1 de octubre, del Procedimiento Administrativo Común de las Administraciones Públicas, recurso de alzada ante la Directora de la Agencia Estatal de Seguridad Aérea, en el plazo de un mes, a contar

desde el día siguiente al de su notificación, sin perjuicio de que puedan ejercitar cualquier otro que estimen oportuno.

CORREO ELECTRÓNICO:  
[drones.aesa@seguridadaerea.es](mailto:drones.aesa@seguridadaerea.es)

CLASIFICACIÓN SENSIBLE

La clasificación de este documento indica el nivel de seguridad para su tratamiento interno en AESA.  
Si el documento le ha llegado por los cauces legales, no tiene ningún efecto para usted

PASEO DE LA CASTELLANA 112  
28046 MADRID  
TEL.: +34 91 396 8000

## ANEXO D. CAMPO VUELO CLUB ALAS DE GALAPAGAR

Para realizar las pruebas de vuelo se utilizaron las instalaciones del Club de Aeromodelismo Alas de Galapagar, en Madrid. La captura de pantalla de la aplicación oficial de ENAIRE Drones en la Figura D.1 muestra que la zona de vuelo está fuera del espacio aéreo controlado. Esto permite operar hasta el límite legal de 120 metros sobre el terreno. La Tabla D.1 recoge los datos operativos y de localización del campo de pruebas.

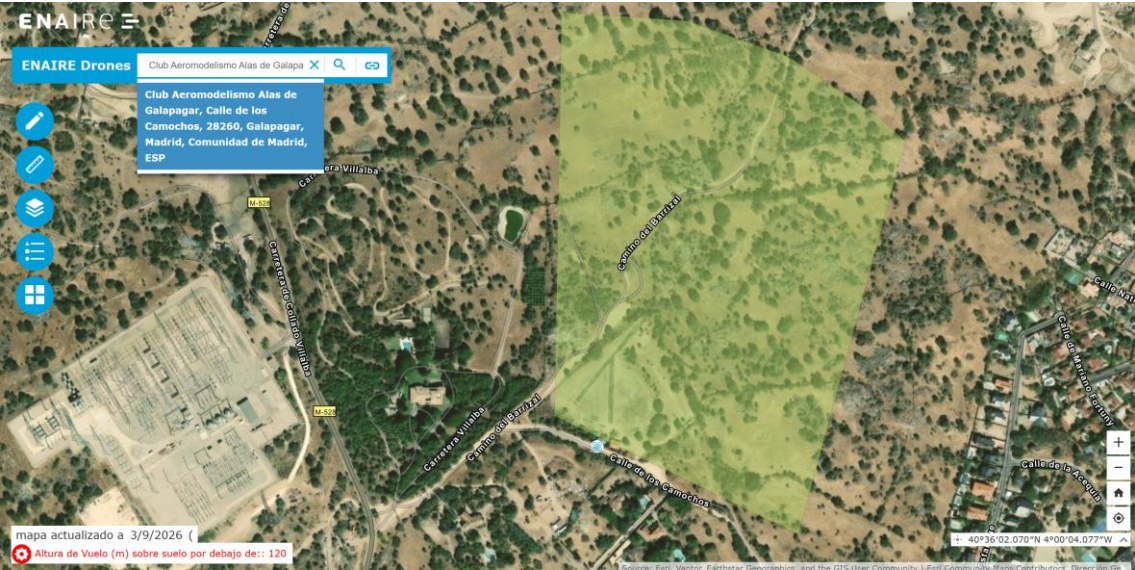


Figura D.1. Ubicación y zona de vuelo del club en ENAIRE drones.

TABLA D.1. DATOS DEL CAMPO DE VUELO.

| Campo                               | Dato                                                                                                                                           |
|-------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| Denominación                        | Club de Aeromodelismo Alas de Galapagar                                                                                                        |
| Ubicación                           | Galapagar (Comunidad de Madrid)                                                                                                                |
| Coordenadas (WGS-84)                | 40.5979097 N, -3.9988503 O                                                                                                                     |
| Elevación aproximada                | ≈ 880 m sobre el nivel del mar                                                                                                                 |
| Identificador AIP (ENAIRE)          | LECMUAV090                                                                                                                                     |
| Publicación oficial                 | AIP España, sección ENR 5.5 (Aeromodelismo, deportes aéreos y otras actividades) [20]                                                          |
| Altura máxima de operación          | 120 m sobre el terreno                                                                                                                         |
| Clasificación del espacio aéreo     | Fuera de espacio aéreo controlado (colindante con el de la Base Aérea de Getafe)                                                               |
| Subcategoría de operación           | A3 – categoría abierta (AESA/EASA)                                                                                                             |
| Habilitaciones de operador y piloto | Anexos B y C                                                                                                                                   |
| Régimen de acceso                   | Alta como socio (temporada 2026) y licencia federativa anual de la Federación Aérea de Madrid, con seguro de responsabilidad civil obligatorio |

Como ejemplo de la integración física y de las jornadas de ensayos en el campo autorizado, la Figura D.2 muestra a la autora junto al prototipo de la aeronave en las instalaciones del club.



Figura D.2. Nerea Gorostidi junto al prototipo desarrollado tras una sesión de ensayos de campo en las instalaciones del Club de Aeromodelismo Alas de Galapagar (LECMUAV090).



## ANEXO E. REPOSITORIOS DE CÓDIGO Y HERRAMIENTAS DEL PROYECTO

Todo el software, la infraestructura de despliegue y los conjuntos de datos desarrollados en este TFG se han publicado en repositorios públicos. Esto busca que otras personas puedan reproducir y reutilizar el sistema. En este anexo se explica dónde se encuentra cada módulo, qué contiene y qué plataformas se utilizaron.

### E.1. Repositorios de código

El código fuente del proyecto se gestiona mediante control de versiones en repositorios públicos alojados en GitHub. Todos están centralizados en el perfil de usuario (<https://github.com/nereagorostidi>). La interfaz se muestra en la Figura E.1. La Tabla E.1 detalla el desglose de los repositorios y sus funciones técnicas en el sistema.

TABLA E.1. REPOSITORIOS PÚBLICOS DEL PROYECTO EN GITHUB.

| Repositorio           | Lenguaje         | Contenido                                                                                                                                                                                                                                                                                                 |
|-----------------------|------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| drone-edge-companion  | Python           | Nodo <i>edge</i> en la Raspberry Pi 5: colectores de los cuatro dominios de datos y publicación por MQTT, receptor de comandos MAVLink y sistema de visión (YOLO) para la detección de personas.                                                                                                          |
| drone-cloud-server    | Python / HTML    | Infraestructura en la nube (AWS EC2): puente MQTT → InfluxDB, API REST para mando y control y panel de control web (PWA).                                                                                                                                                                                 |
| drone-sar-training    | Python / Jupyter | Entrenamiento del modelo de visión: cuaderno de entrenamiento en Google Colab (apartado F.2), configuración del <i>dataset</i> (data.yaml), resultados de la evaluación (métricas, curvas y matrices de confusión, apartado 8.5) y exportación/compilación al formato del acelerador (ONNX → NCNN → HEF). |
| yolo-pipeline-test    | Python           | Prueba preliminar del flujo de trabajo de visión (detección de fruta) con la que se validó el pipeline de extremo a extremo antes de abordar el modelo real (apartado 8.2).                                                                                                                               |
| sardrone-web-proyecto | TypeScript       | Sitio web técnico de documentación del proyecto (guardian-eye.gorostiditfg.com), planteado como una evolución abierta del trabajo, ilustrado en la Figura E.2.                                                                                                                                            |
| sardrone-landing-page | HTML             | Página de presentación y de la campaña de micromecenazgo (drone-sar.gorostiditfg.com), ilustrada en la Figura E.3.                                                                                                                                                                                        |

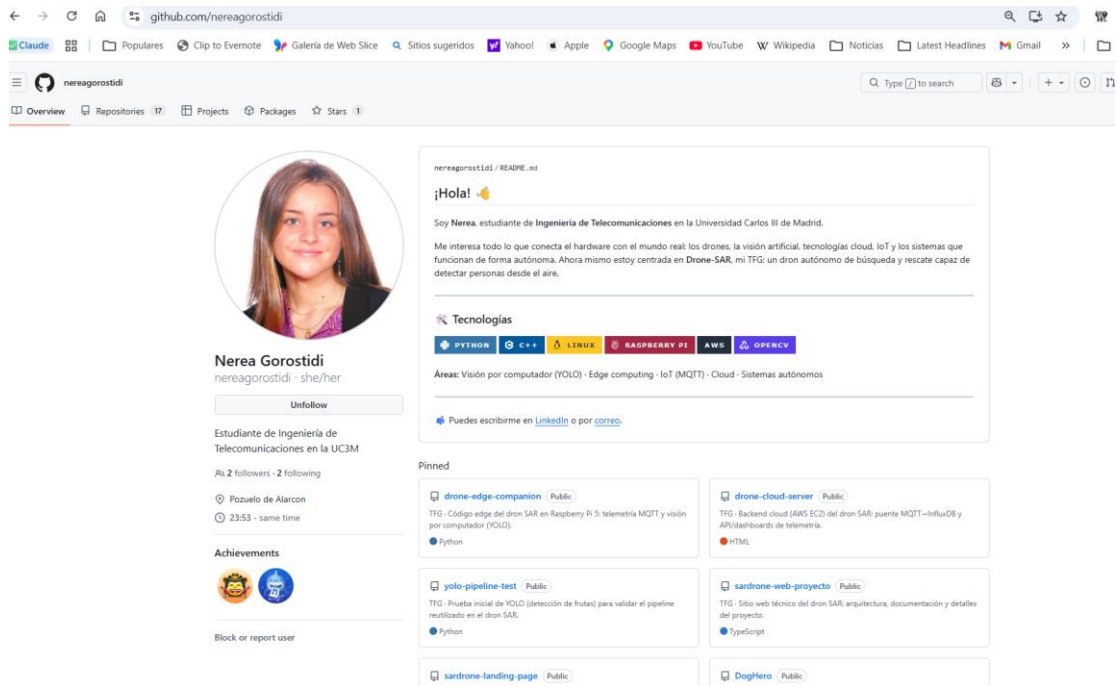


Figura E.1. Perfil público de GitHub del proyecto (github.com/nereagorostidi), con los repositorios destacados.

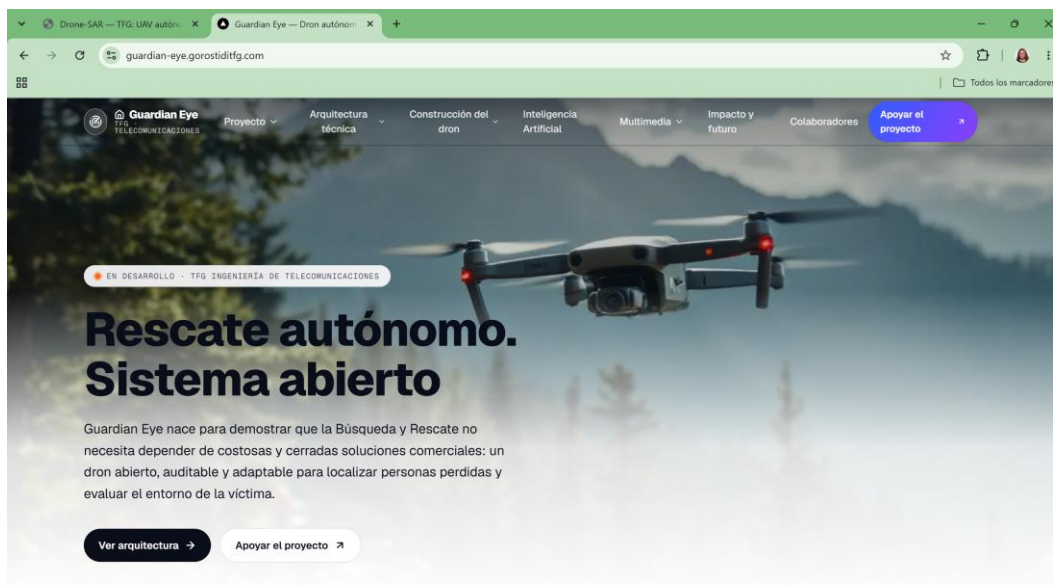


Figura E.2. Portal técnico y documentación en abierto de Guardian Eye (guardian-eye.gorostiditfg.com).

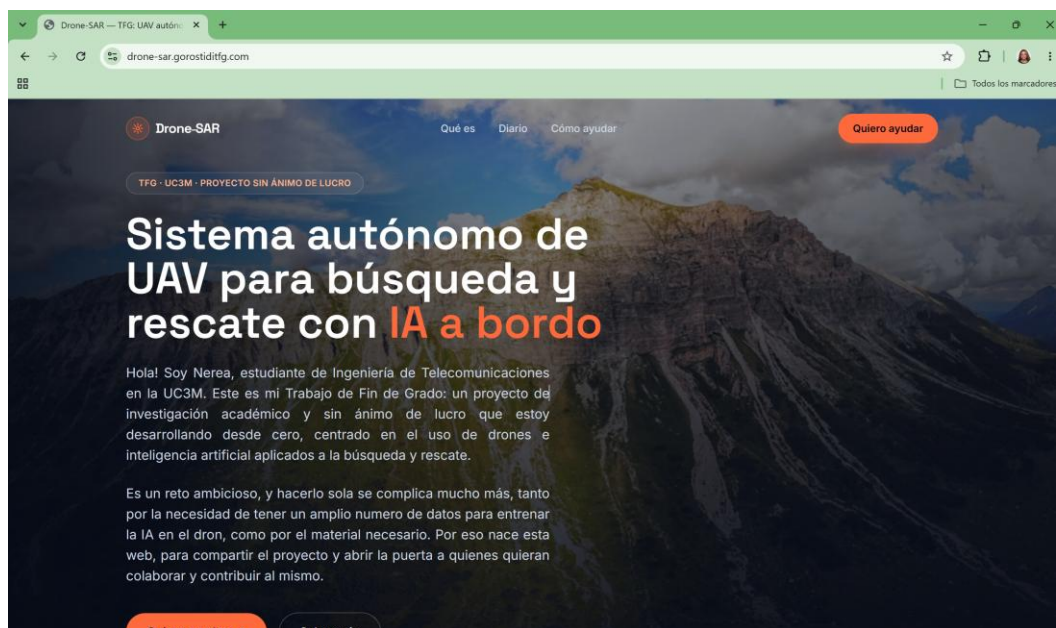


Figura E.3. Página web de difusión pública y presentación del proyecto (drone-sar.gorostiditfg.com)

## E.2. Cuaderno de entrenamiento del modelo (Google Colab)

El entrenamiento del modelo YOLO11 se realizó en Google Colab con aceleración por GPU (NVIDIA T4), cuya cabecera de ejecución se muestra en la Figura E.4. El cuaderno se encuentra versionado en el repositorio público drone-sar-training y es accesible a través de los enlaces detallados en la Tabla E.2.

TABLA E.2. ACCESO AL ENTORNO DE ENTRENAMIENTO DEL MODELO

| Acceso         | Enlace                                                                                                                                                                                                                                                    |
|----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| GitHub         | <a href="https://github.com/nereagorostidi/drone-sar-training">https://github.com/nereagorostidi/drone-sar-training</a>                                                                                                                                   |
| Abrir en Colab | <a href="https://colab.research.google.com/github/nereagorostidi/drone-sar-training/blob/main/notebooks/entrenamiento_yolo11.ipynb">colab.research.google.com/github/nereagorostidi/drone-sar-training/blob/main/notebooks/entrenamiento_yolo11.ipynb</a> |

El cuaderno descarga el conjunto de datos de Roboflow (apartado E.3), ejecuta el entrenamiento mediante *transfer learning* con la librería Ultralytics y genera las métricas y los pesos del modelo. La clave de acceso a la API de Roboflow se solicita de forma interactiva al ejecutar el cuaderno y no se almacena en él.

The screenshot shows the Google Colab interface with the file 'DatasetDefinitivo.ipynb' open. The top bar includes 'Archivo', 'Editar', 'Ver', 'Insertar', 'Entorno de ejecución', 'Herramientas', and 'Ayuda'. The left sidebar shows a file explorer with 'Comandos', 'Código', 'Texto', and 'Ejecutar todas'. The main area displays the following content:

```
!pip install ultralytics roboflow
```

Collecting ultralytics  
Downloading ultralytics-8.4.131-py3-none-any.whl.metadata (45 kB)  
46.0/46.0 kB 3.2 MB/s eta 0:00:00

Collecting roboflow  
Downloading roboflow-1.4.1-py3-none-any.whl.metadata (11 kB)  
Requirement already satisfied: filelock>=3.16.1 in /usr/local/lib/python3.13/dist-packages (from ultralytics) (3.32.4)  
Requirement already satisfied: numpy>=1.23.0 in /usr/local/lib/python3.13/dist-packages (from ultralytics) (2.1.3)  
Requirement already satisfied: matplotlib>=3.3.0 in /usr/local/lib/python3.13/dist-packages (from ultralytics) (3.10.0)  
Requirement already satisfied: opencv-python!=4.13.0.90,>=4.7.0 in /usr/local/lib/python3.13/dist-packages (from ultralytics) (5.0.0.93)  
Requirement already satisfied: pillow>=7.1.2 in /usr/local/lib/python3.13/dist-packages (from ultralytics) (11.3.0)

Figura E.4. Entrenamiento del modelo YOLO11 en Google Colab con aceleración por GPU

### E.3. Conjunto de datos público (Roboflow)

El conjunto de datos aéreo desarrollado en este trabajo se ha publicado en abierto en la plataforma *Roboflow Universe* para que cualquiera pueda consultarlo, descargarlo o reutilizarlo en otros proyectos, tal y como se ilustra en la Figura E.5.

The screenshot shows the Roboflow Universe interface for the dataset 'dron-sar-model'. The left sidebar includes 'Object Detection', 'DATA', 'Versions', 'Analytics', 'Models', 'NAS', 'Test', 'DEPLOY', and 'Active Learning'. The main area displays the following information:

- Versions:** A list of dataset versions with timestamps and image counts. The selected version is 2026-08-27 11:36pm with 717 images.
- Dataset Split:** A table showing the distribution of images across different sets.
- Preprocessing:** Information about the preprocessing steps applied to the dataset.
- Augmentations:** Information about the augmentations applied to the dataset.

| Dataset Split | Train Set  | Valid Set | Test Set  |
|---------------|------------|-----------|-----------|
|               | 540 Images | 88 Images | 89 Images |

Preprocessing: Auto-Orient: Applied, Resize: Fit within 640x640

Augmentations: No augmentations were applied

Figura E.5. Estructura y particionado del conjunto de datos en la plataforma Roboflow Universe.

## ANEXO F. Declaración de uso de IAG

### DECLARACIÓN DE USO DE INTELIGENCIA ARTIFICIAL GENERATIVA (IAG) EN EL TRABAJO DE FIN DE GRADO (TFG)

He usado IAG en mi TFG. Marca lo que corresponda:

|                                        |                             |
|----------------------------------------|-----------------------------|
| <input checked="" type="checkbox"/> SÍ | <input type="checkbox"/> NO |
|----------------------------------------|-----------------------------|

#### Parte 1: declaración sobre comportamiento legal, ético y responsable

Se responde a las cuatro cuestiones del modelo. La opción marcada (☒) y sombreada es la aplicable a este trabajo.

|                                                                                                                                                                                                                                                                                                                                                                                              |                                                                                                    |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------|
| <b>En mi interacción con herramientas de IAG he facilitado datos de carácter CONFIDENCIAL contando siempre con la debida autorización de los interesados.</b>                                                                                                                                                                                                                                |                                                                                                    |
| <input type="checkbox"/> Sí, he usado estos datos con la autorización de los interesados.                                                                                                                                                                                                                                                                                                    | <input checked="" type="checkbox"/> NO, no he usado datos de carácter confidencial.                |
| <b>En mi interacción con herramientas de IAG he facilitado MATERIALES PROTEGIDOS POR DERECHOS DE AUTORÍA contando siempre con la autorización de los respectivos titulares.</b>                                                                                                                                                                                                              |                                                                                                    |
| <input type="checkbox"/> Sí, con autorización de los titulares o al amparo de una excepción legal (dominio público, licencia Creative Commons, derecho de cita con fines de investigación).                                                                                                                                                                                                  | <input checked="" type="checkbox"/> NO, no he usado materiales protegidos por derechos de autoría. |
| <b>En mi interacción con herramientas de IAG he facilitado DATOS DE CARÁCTER PERSONAL con la debida autorización de los interesados.</b>                                                                                                                                                                                                                                                     |                                                                                                    |
| <input type="checkbox"/> Sí, con autorización de los interesados y conforme a las instrucciones de la guía aprobada por la Universidad.                                                                                                                                                                                                                                                      | <input checked="" type="checkbox"/> NO, no he usado datos de carácter personal.                    |
| <b>Mi utilización de la herramienta de IAG ha respetado sus términos de uso, así como los principios éticos esenciales, no orientándola de manera maliciosa a obtener un resultado inapropiado para el trabajo presentado (que produzca una impresión contraria a la realidad de los resultados, que suplante mi propio trabajo o que pueda resultar en un perjuicio para las personas).</b> |                                                                                                    |
| <input checked="" type="checkbox"/> SÍ                                                                                                                                                                                                                                                                                                                                                       | <input type="checkbox"/> NO                                                                        |

#### Parte 2: declaración de uso técnico

Declaro haber hecho uso de los sistemas de IAG ChatGPT (OpenAI) y Claude (Anthropic) —este último también en su modalidad Claude Code para la asistencia en programación— para las finalidades indicadas. En todos los casos, las respuestas de las herramientas se han utilizado como apoyo al trabajo propio, revisándolas y adaptándolas antes de incorporarlas.

#### Documentación y redacción

- **Soporte a la reflexión en relación con el desarrollo del trabajo (análisis de alternativas y enfoques)**

Empleé la IAG para contrastar alternativas de diseño antes de decidir. Las aportaciones sirvieron de punto de partida; la valoración y la decisión final fueron propias y se justifican en la memoria a partir de la documentación técnica y de los resultados de las pruebas.

- **Revisión o reescritura de párrafos redactados previamente**

Utilicé la IAG para revisar párrafos ya redactados por mí, con el fin de mejorar su claridad y estructura, y detectar repeticiones. Todas las propuestas se revisaron y adaptaron antes de incorporarlas a la versión final, sin alterar el contenido técnico ni los datos.

- **Búsqueda de información o respuesta a preguntas concretas**

Recurrí a la IAG para resolver dudas puntuales sobre conceptos y aspectos prácticos del trabajo (por ejemplo, cómo resuelven algunos fabricantes de drones el enlace de radio propietario, o cómo alimentar la Raspberry Pi mediante un regulador UBEC). La información se contrastó con la documentación oficial, las técnicas de los fabricantes y la bibliografía.

## Desarrollar contenido específico

- **Asistencia en el desarrollo de líneas de código (programación y software)**

Se diferencia explícitamente el alcance de la asistencia entre el software de aviónica/IoT y las herramientas auxiliares:

- **Software de control, aviónica e inferencia a bordo (Núcleo de ingeniería):** La arquitectura general, la interacción con la Pixhawk para telemetría y navegación (pymavlink), la persistencia en un buffer local SQLite (*Store-and-forward*), la ingesta MQTT multidominio y el flujo de inferencia acelerada con YOLO11 en la NPU Hailo-8L han sido diseñados, integrados y testeados por mí. En esta capa, utilicé la IA y **Claude Code** como entorno de asistencia técnica avanzada para la refactorización de scripts en Python, la generación de estructuras de funciones iniciales, la depuración de excepciones (*debugging*) y la resolución de errores de compilación con las librerías de HailoRT y MAVLink. Todo el código generado fue revisado de forma crítica, adaptado e integrado tanto en el hardware físico como en el simulador SITL, complementando y asumiendo la responsabilidad íntegra por su correcto funcionamiento.
- **Plataforma web de difusión y captación (Material auxiliar y complementario):** El desarrollo del portal web ([drone-sar.gorostiditfg.com](https://drone-sar.gorostiditfg.com)), el formulario de colaboración y la bitácora técnica ([journal.html](#)) constituyen un recurso de apoyo secundario y estrictamente complementario al TFG. Estas páginas nacieron con un propósito instrumental muy acotado: servir como canal de divulgación abierta y ofrecer un soporte visual y atractivo para facilitar la captación ágil y voluntaria de vídeos reales destinados a la construcción del conjunto de datos. Al quedar fuera del núcleo de ingeniería del sistema (centrado en la aviónica, la inferencia Edge-AI y la arquitectura de comunicaciones), en este desarrollo *frontend* (HTML, CSS y JavaScript) se recurrió a un uso extensivo de **Claude Code** para acelerar la maquetación. La definición de requisitos, la orientación de contenidos, la configuración del servidor web en nginx y la gestión del dominio en Cloudflare se realizaron bajo mi supervisión directa.

- **Generación de esquemas, imágenes, audios o vídeos**

Empleé la IA para elaborar borradores de esquemas y diagramas explicativos, así como de las tablas y del cronograma de la sección de planificación, a partir de descripciones, del diario de seguimiento del proyecto y de indicaciones propias; todo ello se revisó y se adaptó al trabajo. Las fotografías del montaje y del vehículo, así como el material del conjunto de datos, son propias o proceden de fuentes de libre uso debidamente citadas.

- **Procesos de optimización**

Utilicé la IA como apoyo en tareas de depuración y en la resolución de incidencias de instalación, configuración y compilación del entorno de simulación e inferencia. Las mediciones y las conclusiones son de elaboración propia.

- ***Tratamiento de datos: recogida, análisis, cruce de datos...***

Empleé la IA para generar resúmenes de los resultados de las simulaciones y de los ensayos de rendimiento y para proponer formas de representarlos. El análisis, la discusión y las conclusiones son propios.

- ***Inspiración de ideas en el proceso creativo***

Una vez definidos los objetivos del TFG, utilicé la IAG para explorar posibles enfoques y alternativas. Las propuestas sirvieron como fuente de reflexión; las decisiones finales las adopté yo.

- ***Generación y ordenación de material auxiliar (glosario de abreviaturas)***

Se empleó la IAG como herramienta de apoyo para extraer, ordenar alfabéticamente y redactar el significado formal del listado inicial de acrónimos y términos técnicos del Anexo A (Glosario de abreviaturas), revisando y verificando manualmente cada término y definición contra la normativa y la bibliografía del proyecto antes de su inclusión definitiva.

### **Parte 3: reflexión sobre utilidad**

Durante el desarrollo de este Trabajo Fin de Grado, la IAG me ha sido útil como herramienta de apoyo para resolver dudas y reforzar conceptos. También me ayudó a explorar opciones antes de decidir. La usé en momentos de bloqueo durante la programación y para crear borradores de esquemas, lo que hizo el trabajo más visual.

Como puntos débiles, he visto que la información que ofrece no siempre es correcta ni actualizada, por lo que he tenido que compararla con la documentación oficial, la documentación técnica de los fabricantes, la bibliografía y, sobre todo, con el funcionamiento real del sistema y con los resultados de las pruebas. Además, la calidad de las respuestas depende en gran medida de cómo se formulan las preguntas. Esto me ha obligado a definir con claridad cada problema y a revisar críticamente lo que recibía.

Creo que la IAG ha sido una herramienta de apoyo útil, pero siempre como complemento del criterio y de los análisis propios. Esta experiencia mostró que usar bien la IAG requiere formular los requisitos con precisión y mantener un filtro crítico de manera constante.

En resumen, la herramienta ha servido para acelerar tareas de apoyo, pero no reemplaza el trabajo de ingeniería. El diseño arquitectónico, el desarrollo del software de aviónica e IoT, la integración de la NPU, la experimentación en campo y las conclusiones son el resultado de mi dedicación y de mi criterio técnico personal.